



BAZE DE DATE

Concepte de bază

Mihaela Elena Breabăn

© FII 2015-2016

Bază de date

- ▶ o colecție de date (operaționale) relaționate logic
- ▶ proiectată pentru a deservi necesarul de informații al unei organizații

Sistem de gestiune a bazelor de date (SGBD)

- ▶ **Ansamblu:**

- ▶ Hardware
- ▶ Software
- ▶ Date
- ▶ Utilizatori

- ▶ pune la dispoziție metode eficiente și sigure de regasire și furnizare a datelor către un număr mare de utilizatori

SGBD

Funcții

- ▶ Oferă
 - ▶ Securitate
 - ▶ Acces controlat la baza de date
 - ▶ Stocarea, regăsirea, actualizarea datelor
 - ▶ Integritate
 - ▶ Suport pentru tranzacții
 - ▶ Control concurent
 - ▶ Recuperare a datelor
 - ▶ Catalog (dicționarul de date)

SGBD

Hardware

- ▶ Datele au caracter persistent
- ▶ Volumul de date este ridicat
- ▶ Accesul se realizează rapid

- ▶ Poate varia de la un simplu PC la o rețea de calculatoare

SGBD Software

- ▶ Interacțiunea dintre utilizatori si sistem se realizează prin limbaje de interogare:
 - ▶ DDL (data definition language)
 - ▶ Definirea datelor - generează meta-date
 - ▶ DML (data manipulation language)
 - ▶ Regăsirea și actualizarea datelor
- ▶ abordare neprocedurală

SGBD

Utilizatori

- ▶ *Administratorul bazei de date*
- ▶ *Proiectantul bazei de date*
- ▶ *Programatorii de aplicații*
- ▶ *Utilizatorii finali*

SGBD

Arhitectura

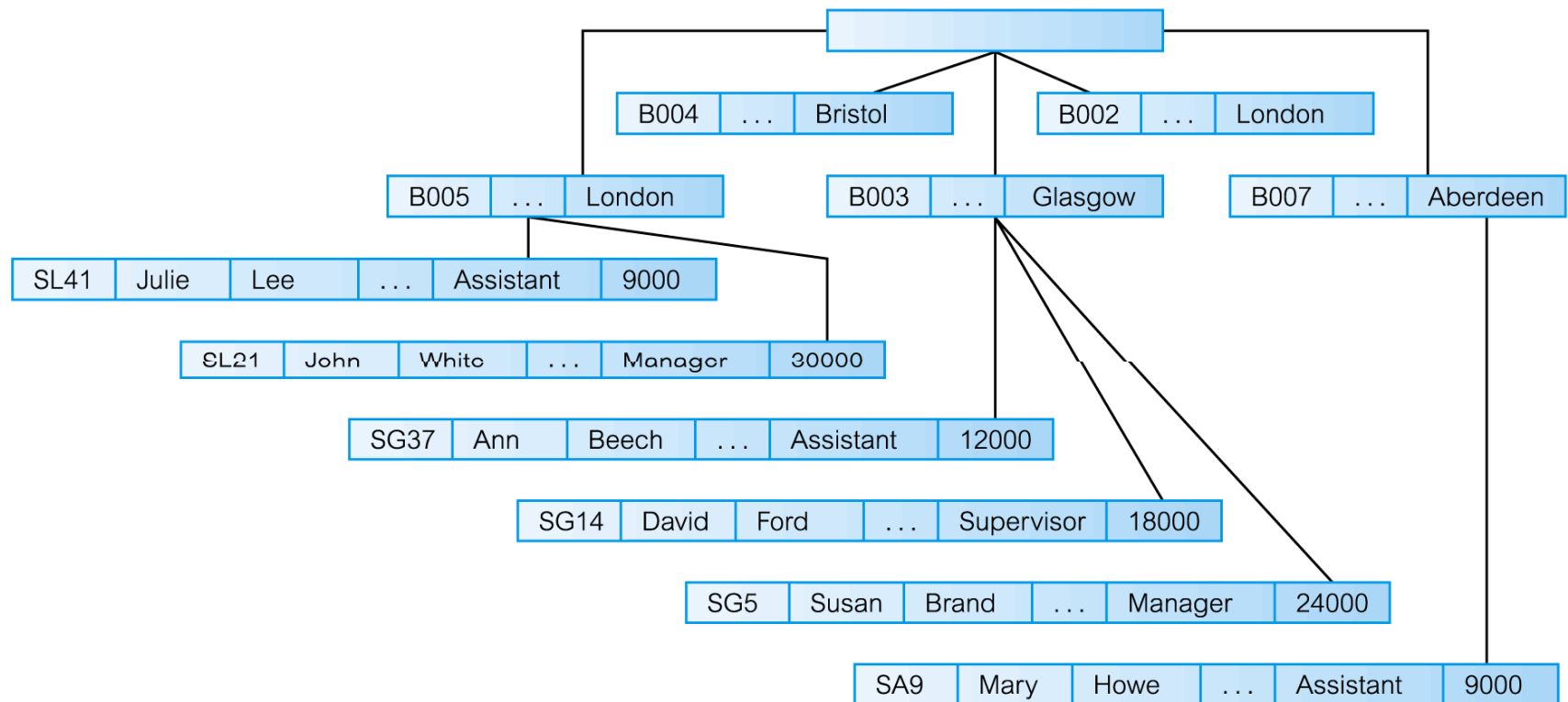
- ▶ **Funcțional:**
 - ▶ Managerul de memorie
 - ▶ Procesorul de interogări
 - ▶ Managerul de tranzacții (ACID)
- ▶ **La nivel de aplicație**
 - ▶ Client-server

SGBD

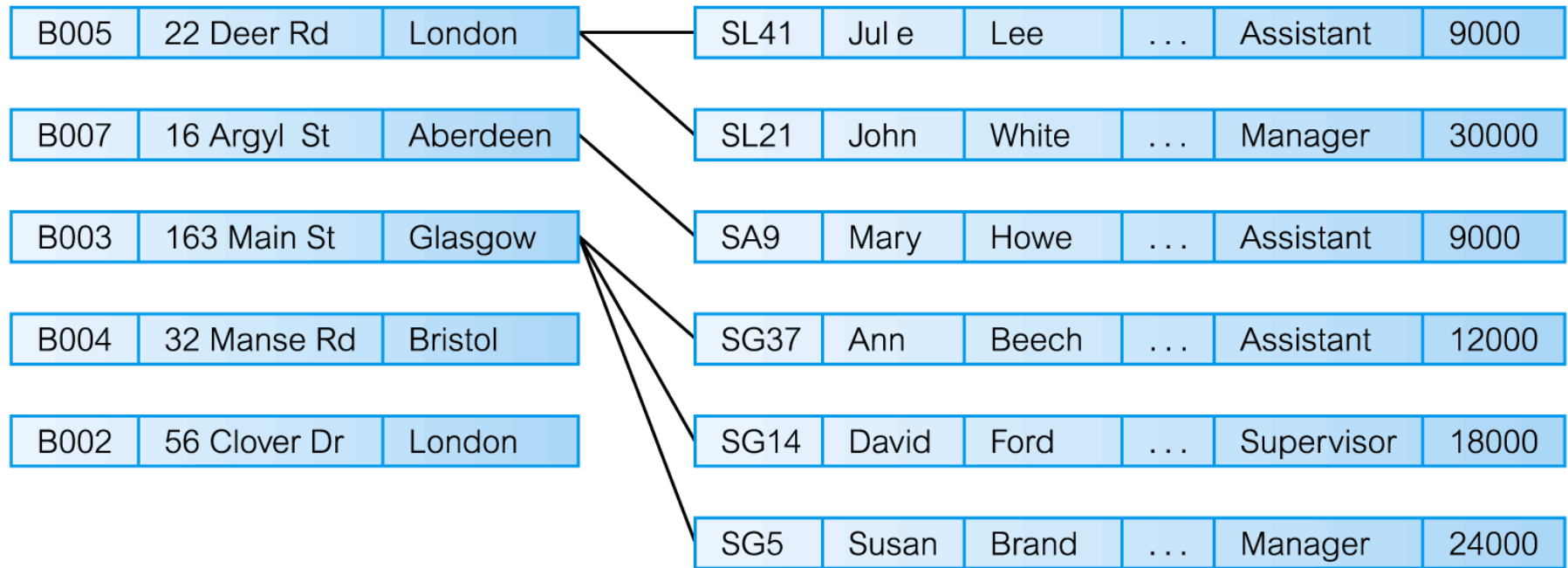
Istoric

- ▶ Modelele ierarhic (IBM's IMS, sf. '60)
- ▶ Modelul rețea (CODASYL 1971)
- ▶ Modelul relațional (Codd, '70)
- ▶ Modelul obiect-relațional ('90)

Modelul ierarhic (IBM's IMS, sf. '60)



Modelul rețea (Charles Bachman - CODASYL 1971)



II

Ex. preluat din Thomas Connolly, Caroline Begg: “Database Systems. A practical approach to design, implementation and management”. Ed. Addison Wesley

Modelul relațional (Edgar Frank Codd – ‘70)

Branch

branchNo	street	city	postCode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

- ☐ IBM's System R, SEQUEL
- ☐ Berkley's Ingres
- ☐ Oracle

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

Modelul relațional

► Componente:

- O clasă de structuri de date denumite tabele
- Constrângeri impuse asupra datelor din tabele
- Asocieri între tabele
- Metode pentru a construi noi tabele (operații în algebra relațională)

Baze de date relaționale

Terminologie

- ▶ **Relație = Tabel**
- ▶ **Atribute = Coloane = Câmpuri**
- ▶ **Domeniu** – mulțimea de valori permise pentru atribute
- ▶ **Tuplu = Înregistrare** – o linie dintr-o relație
- ▶ **Bază de date relațională** – o colecție de relații cu nume distincte
- ▶ **Schema unei relații** – o relație cu nume definită de perechi atribut-domeniu
- ▶ **Schema unei baze de date** relaționale - mulțime de scheme de relații
- ▶ **Instanță a bazei de date** – conținutul bazei de date la un anumit moment

Proprietăți ale relațiilor

- ▶ Numele relațiilor sunt unice în schema relațională
- ▶ Fiecare celulă a unei relații conține exact o valoare atomică
- ▶ Fiecare atribut are nume unic în cadrul unei relații
- ▶ Valorile unui atribut sunt toate din același domeniu
- ▶ Ordinea atributelor și a tuplelor nu are semnificație
- ▶ (Fiecare tuplu este distinct; nu există tuple duplicate)

Chei

- ▶ **Supercheie** – un atribut sau o mulțime de attribute care identifică unic un tuplu într-o relație
- ▶ **Cheie candidat** – o supercheie cu proprietatea că nici o submulțime proprie a sa nu este supercheie
- ▶ **Cheie primară** – o cheie candidat selectată pentru a identifica în mod unic tuplele într-o relație
- ▶ **Cheie alternativă** – Chei candidat care nu au fost selectate pentru a juca rolul de cheie primară
- ▶ **Cheie străină** – un atribut sau o submulțime de attribute dintr-o relație care face referință la o cheie candidat a altei relații

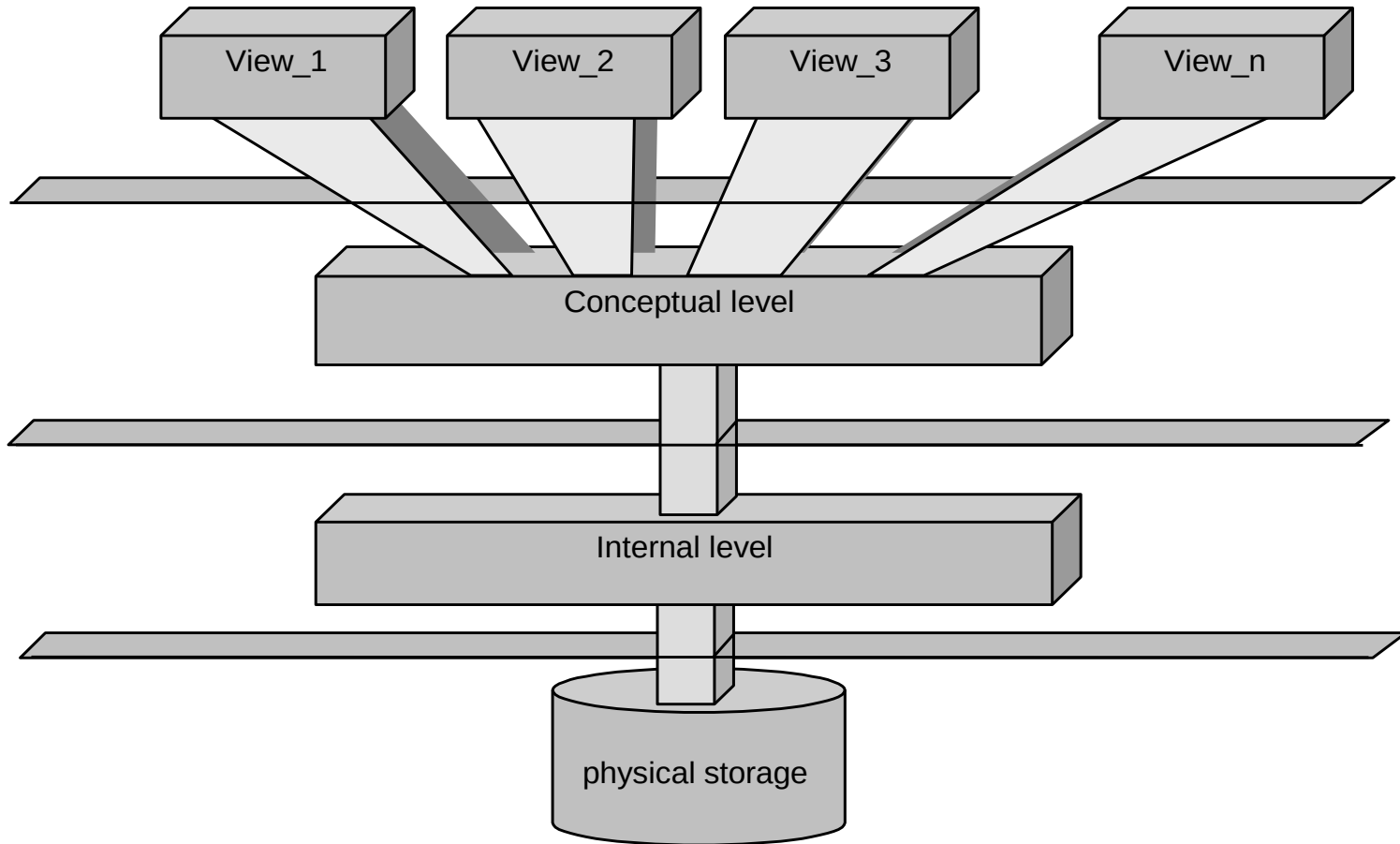
Constrângeri de integritate

- ▶ Nici un atribut al cheii primare nu poate fi NULL
- ▶ Valoarea cheii străine trebuie să se potrivească cu valoarea cheii candidat pentru măcar un tuplu din relația referențiată, altfel trebuie să aibă valoarea NULL.
- ▶ Alte constrângeri...

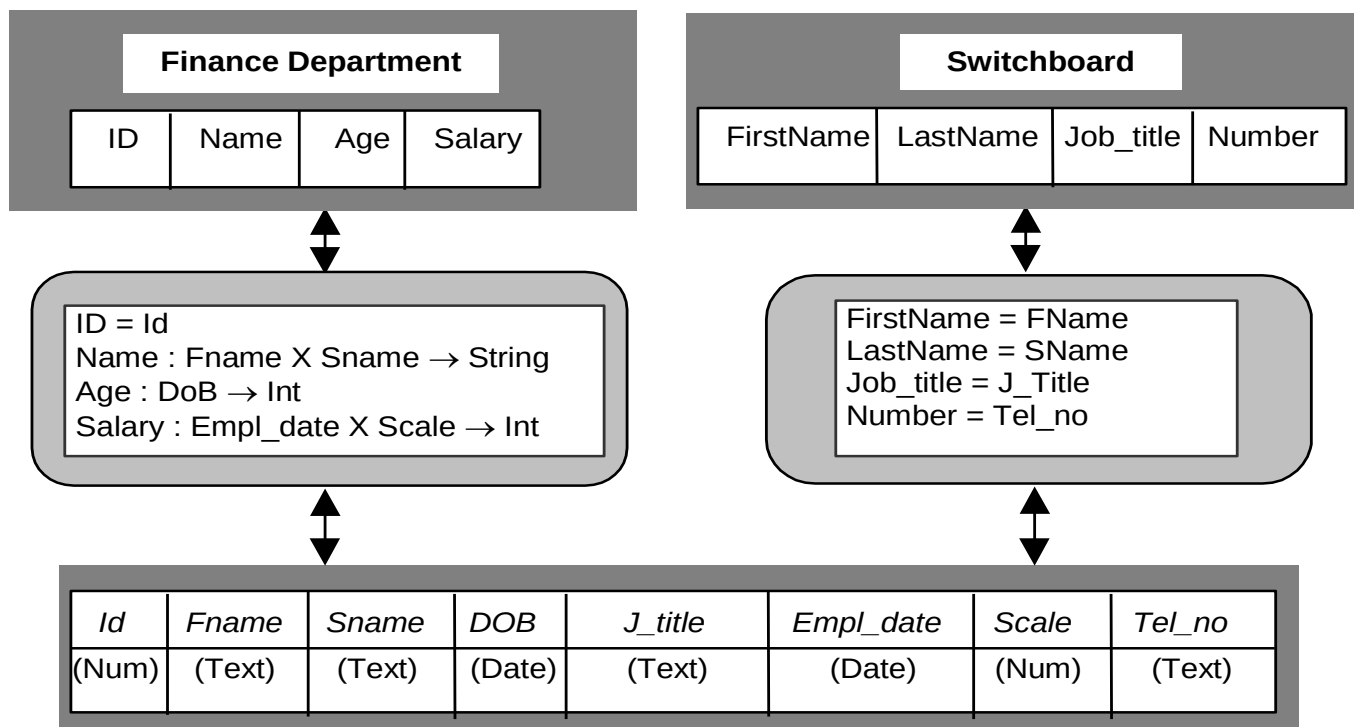
View-uri

- ▶ Relațiile de bază au tuplele stocate fizic în baza de date
- ▶ View-ul este rezultatul unor operații cu tabelele existente, nu e stocat efectiv în baza de date.

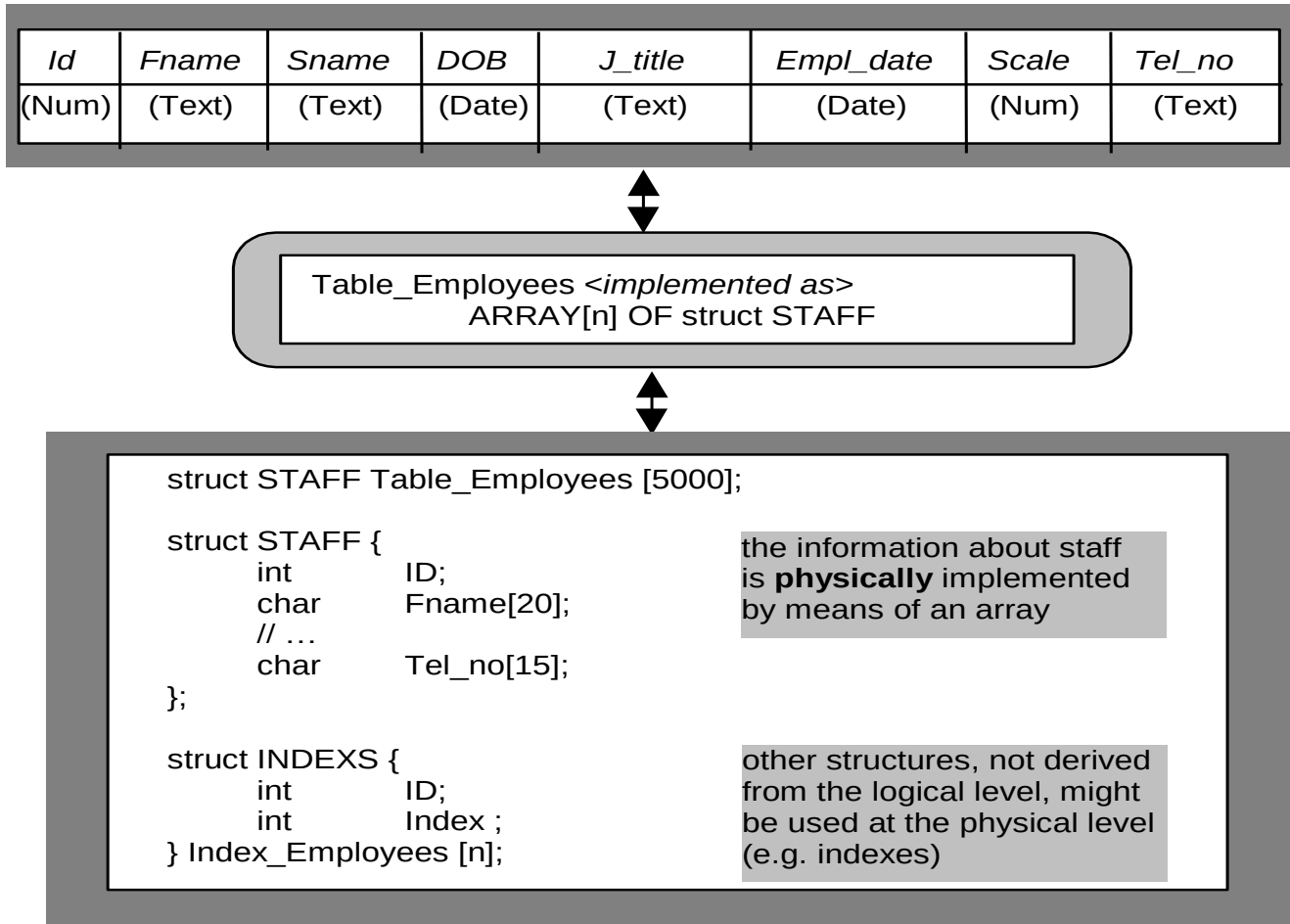
Arhitectura pe 3 nivele ANSI-SPARC



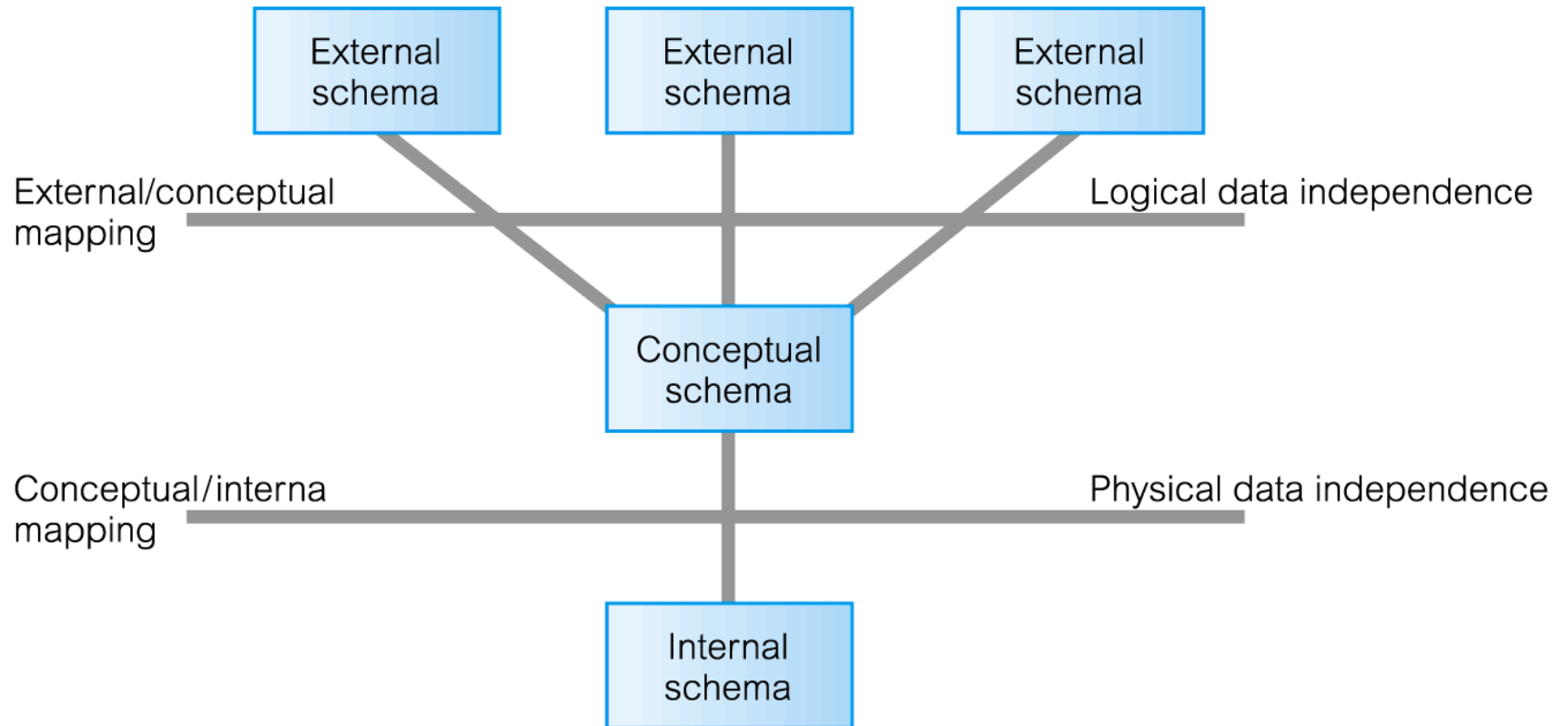
Mapare nivel extern/conceptual



Mapare nivel conceptual/intern



Arhitectura pe 3 nivele Scheme



SGBD – avantaje

- ▶ Consistența datelor
- ▶ Partajarea datelor
- ▶ Securitate
- ▶ Acces îmbunătățit
- ▶ Concurență crescută
- ▶ Servicii de backup și recuperare

Bibliografie

- ▶ E. F. Codd: *A Relational Model of Data for Large Shared Data Banks*. [CACM](#) [13](#)(6): 377-387 (1970)
- ▶ E. F. Codd(1985). "Is Your DBMS Really Relational?" and "Does Your DBMS Run By the Rules?" *ComputerWorld*, October 14 and October 21.
- ▶ E. F. Codd. 1990. *The Relational Model for Database Management:Version 2*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.
- ▶ Thomas Connolly, Caroline Begg:“*Database Systems.A practical approach to design, implementation and management*”. Ed. Addison Wesley



BAZE DE DATE

Proiectarea bazelor de date relaționale

Modelul entitate-asociere

Mihaela Elena Breabăn

© FII 2015-2016

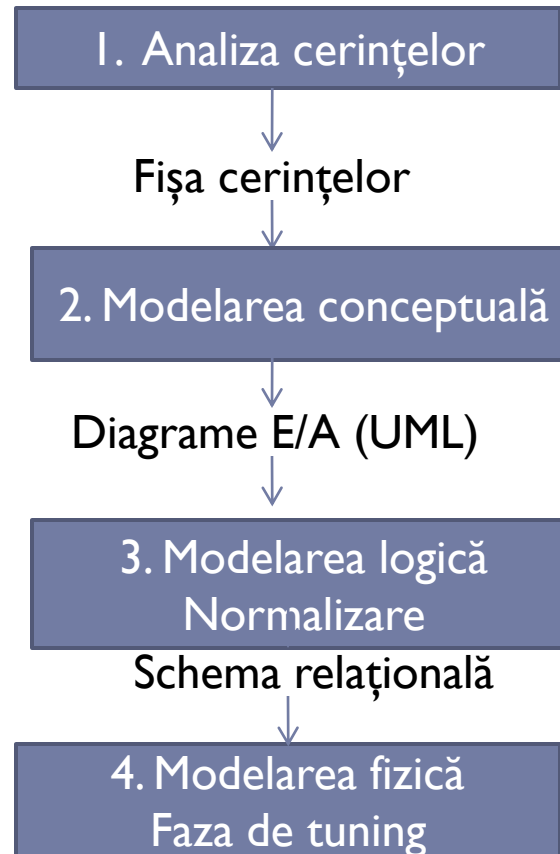
Modelul entitate-asociere (Entity/Relationship)

Obiective

- ▶ Concepte E/A de bază
- ▶ Modelarea constrângerilor
- ▶ Capcane de conectare
- ▶ Modelarea E/A înUML
- ▶ De la diagrame E/A (UML) la schema relațională

Proiectarea unei BD

Metodologie



Concepte E/A clasice (Chen 1976)

▶ Entitate

- ▶ Obiect ce trebuie reprezentat în baza de date
- ▶ Mulțime-entitate - corespunde unui grup de obiecte de același tip, deci unei mulțimi omogene de entități
- ▶ O instanță - entitate
 - ▶ O instanță unic identificabilă

▶ Asociere (Relationship)

- ▶ Conexiune/asociere între două sau mai multe entități (sau chiar a unei entități cu ea însăși)
- ▶ Gradul asocierii = nr. de entități participante
 - ▶ unare/recursive, binare, ternare...
- ▶ Mulțime-asociere – corespunde unei mulțimi omogene de asocieri

▶ Atribut

- ▶ Proprietate a unei entități
- ▶ Pentru asociere
 - ▶ Atribute ale entităților referențiate
 - ▶ Noi atribute

Diagrame E/A

- ▶ Reprezentare grafică a conceptelor E/A
 - ▶ Există mai multe standarde grafice, aici varianta Chen

Mulțime Entitate

Mulțime
Asociere

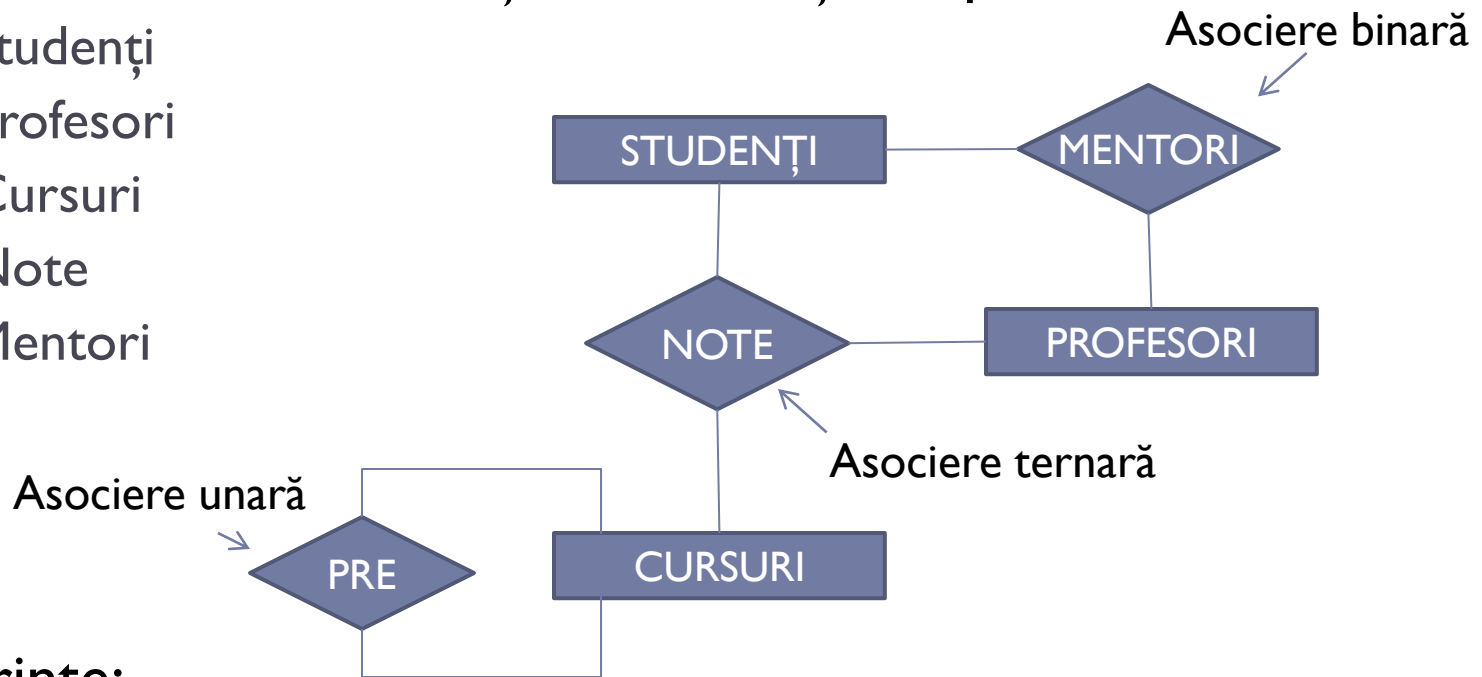
Atribut

- ▶ Un graf
 - ▶ Mulțimile-entitate, mulțimile-asociere și attributele sunt noduri
 - ▶ Există muchii doar între
 - ▶ noduri-entitate și noduri-asociere
 - ▶ noduri-entitate și noduri-atribute
 - ▶ noduri-asociere și noduri-atribute

Exemplu

► O bază de date ce conține informații despre:

- Studenți
- Profesori
- Cursuri
- Note
- Mentori



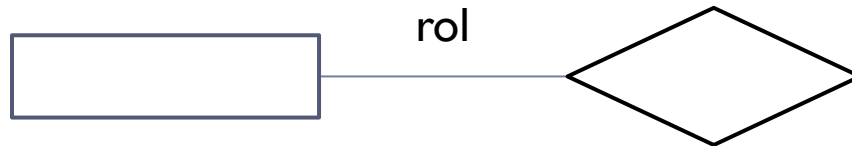
► Cerințe:

- Putem determina notele obținute, cursurile pe care le-a finalizat și creditele obținute de orice student
- Putem determina mentorul oricărui student

Alte concepte E/A

▶ Rol

- ▶ Explică semnificația entităților în asocieri



▶ Cheie primară

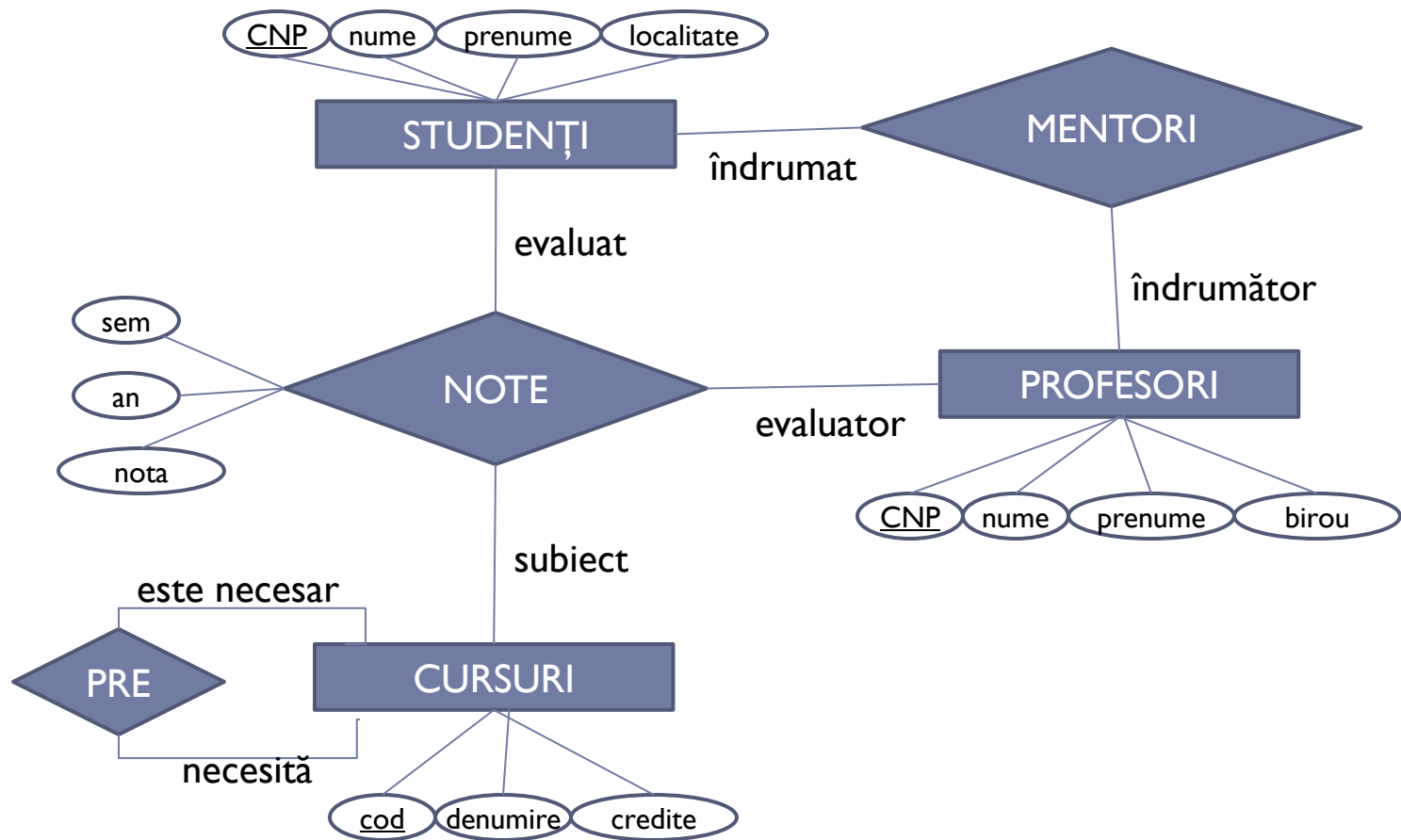
- ▶ Un atribut sau o submulțime minimală de attribute ce identifică unic o instanță-entitate sau o instanță-asociere
- ▶ Obligatorie pentru entități, pentru a indica care instanțe participă în asocieri

Cheie primară

▶ Cheie străină pentru o asociere

- ▶ Un atribut sau o mulțime de attribute care constituie cheie primară pentru entitățile implicate

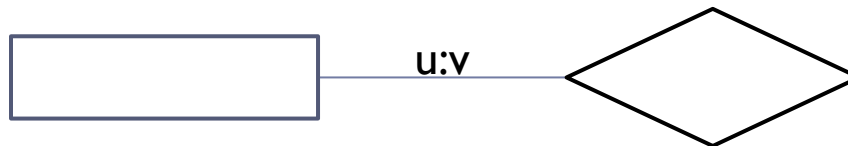
Exemplu



Care sunt cheile străine pentru cele trei mulțimi de asocieri?

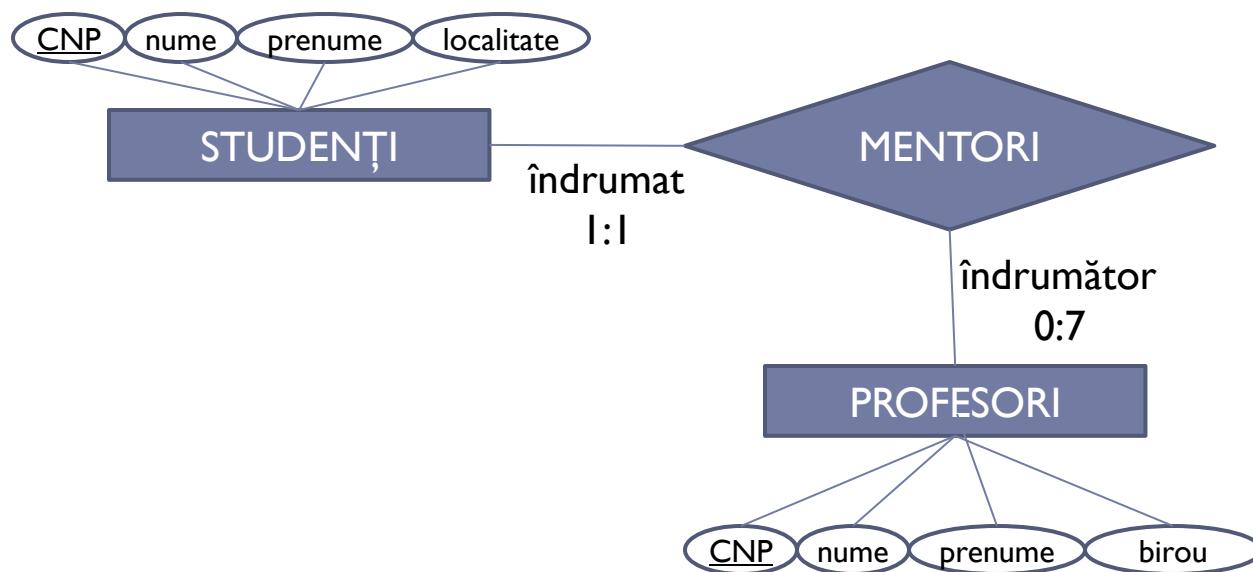
Constrângeri de conectivitate/participare

- ▶ Modelul E/A permite declararea de constrângeri asupra numărului de instanțe-asociere în care o instanță-entitate participă
- ▶ Fie R o mulțime-asociere între n mulțimi-entitate $E_i, i=1..n$. Baza de date satisface constrângerea (E_i, u,v,R) dacă fiecare instanță-entitate din E_i participă în cel puțin u și cel mult v instanțe-asociere din R .

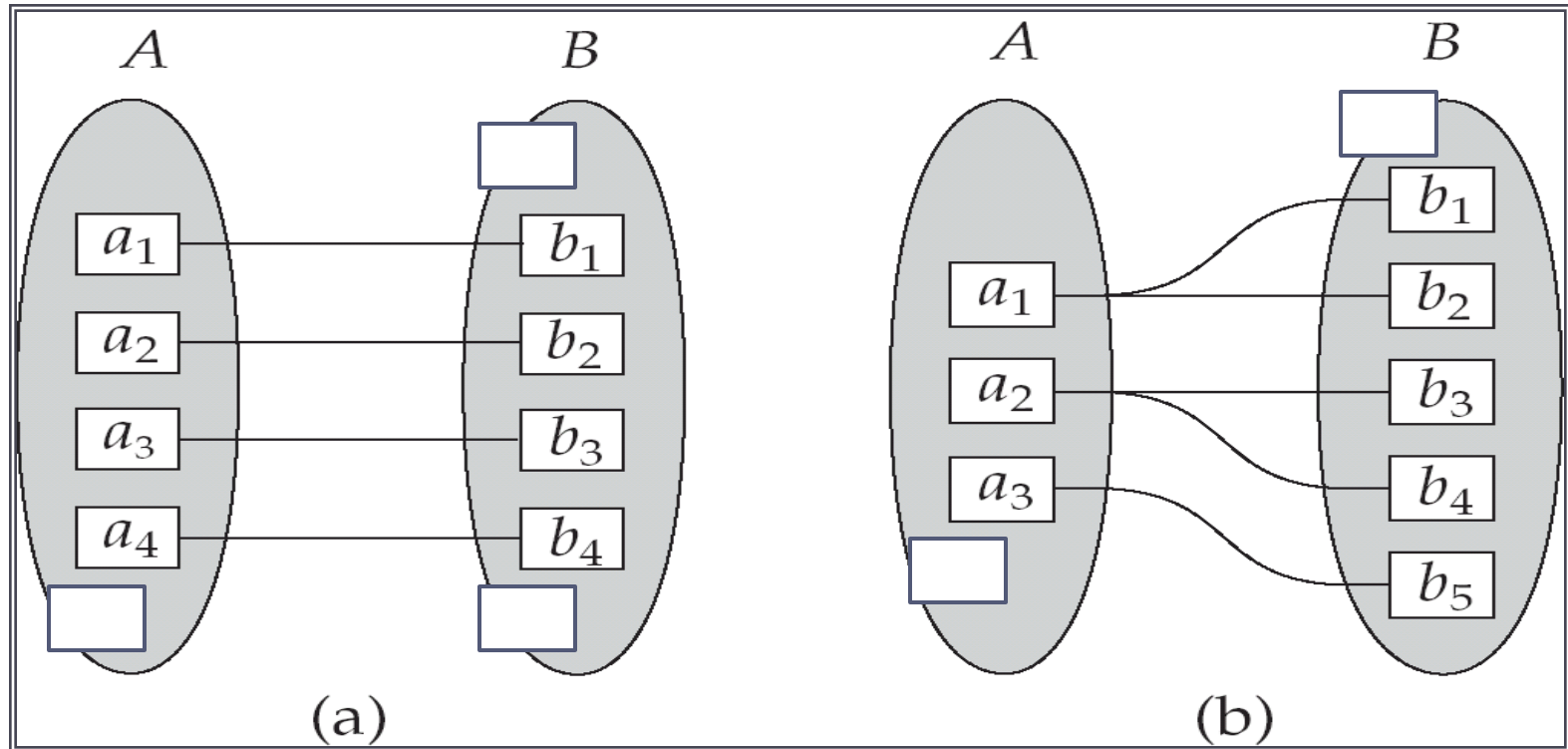


Exemplu

- ▶ (Studenti, 1, 1 Mentori)
- ▶ (Profesori, 0, 7, Mentori)
- ▶ Fiecare student are un singur profesor drept mentor iar un profesor poate fi mentor pentru cel mult 7 studenti



Constrângeri de conectivitate pentru asocieri binare (1)



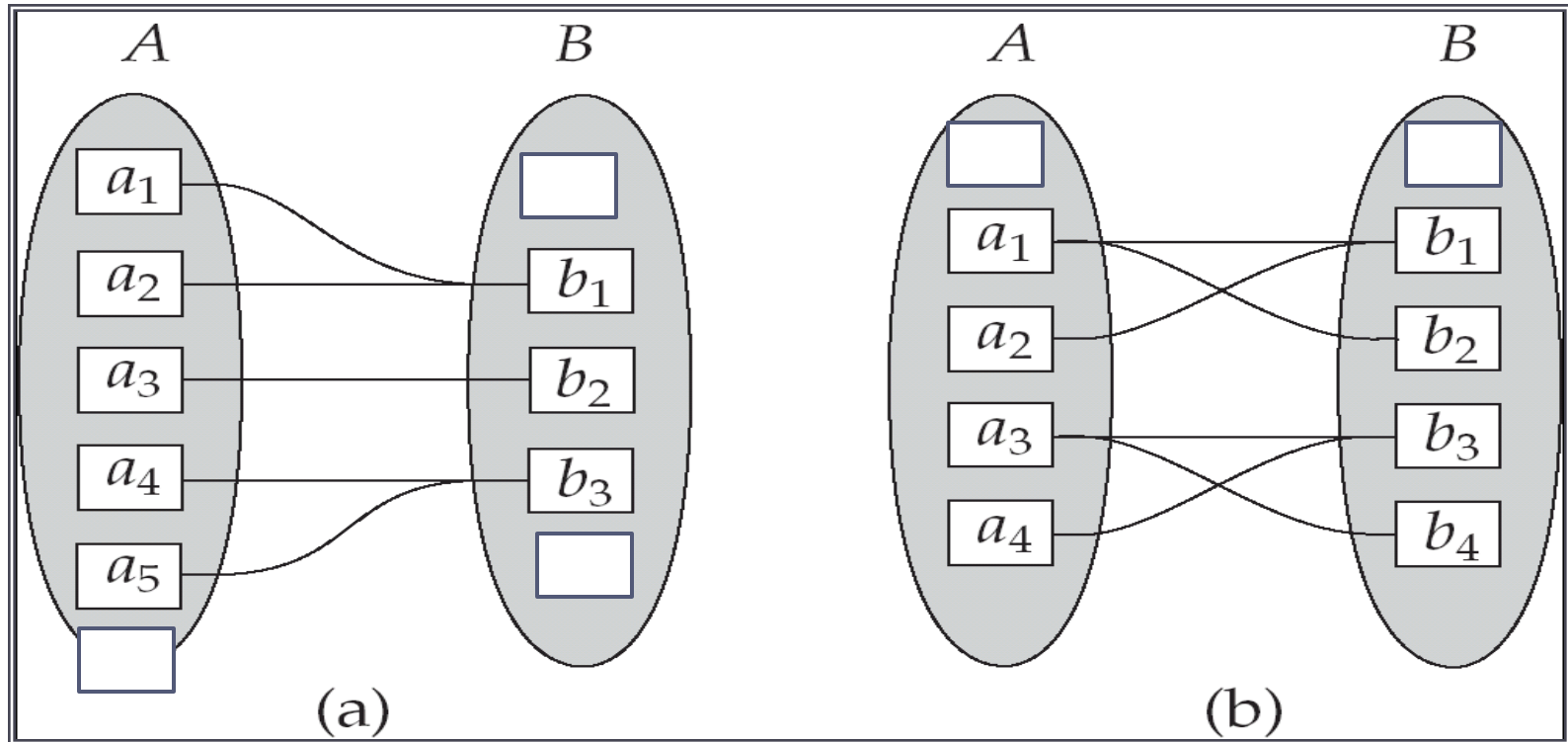
a) Asocierie unu la unu
(A,0,1,R) (B,0,1,R)



b) Asocierie unu la mulți
(A,0,n,R) (B,0,1,R), $n > 1$



Constrângeri de conectivitate pentru asocieri binare (2)



a) Asociere mulți la unu
(A,0,1,R) (B,0,n,R)

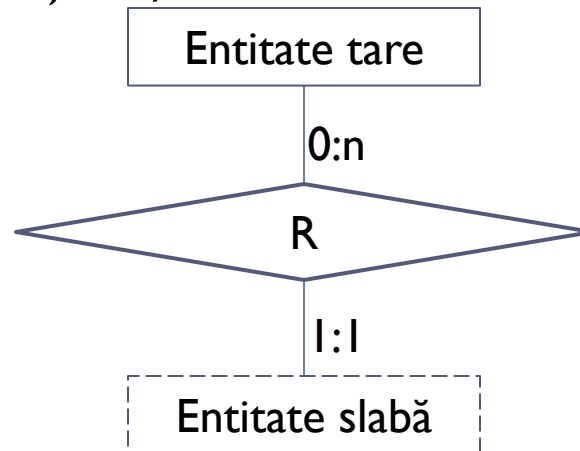


b) Asociere mulți la mulți
(A,0,m,R) (B,0,n,R), $m,n > 1$



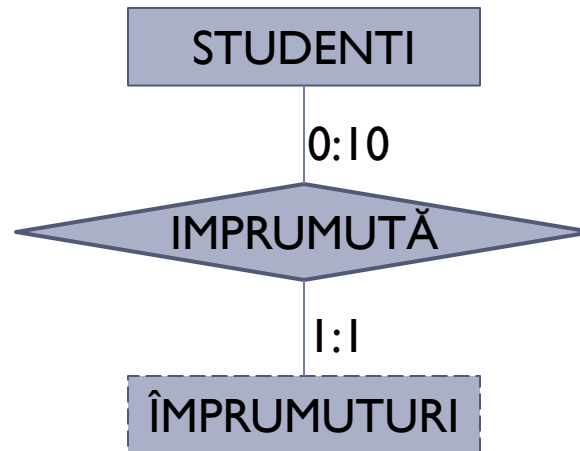
Entitate slabă

- ▶ O mulțime-entitate este slabă dacă existența instanțelor sale depinde de existența instanțelor altei mulțimi-entități (dependență existențială)

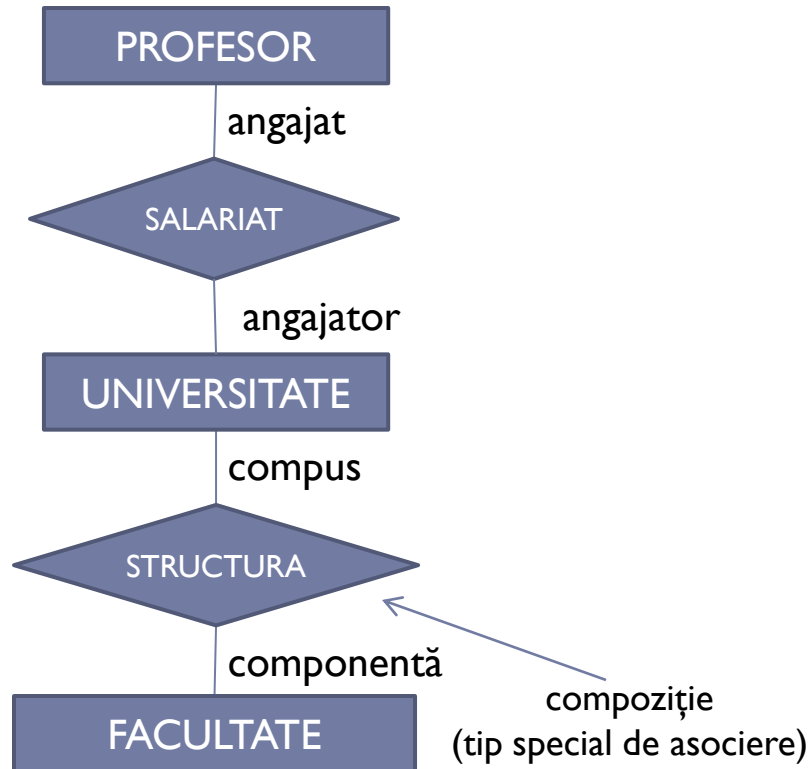
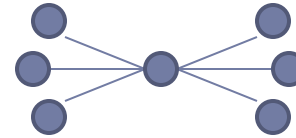


- ▶ Nu are cheie
- ▶ Satisface constrângerea de conectivitate (Entitate_slaba, 1, 1, R), deci participă într-o asociere de tip unu la mulți relativ la entitatea tare

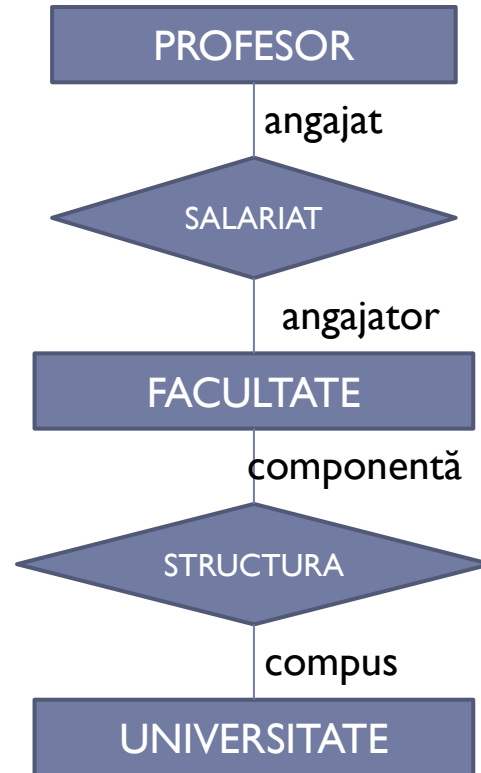
Exemplu



Capcane de conectare (Fan traps)

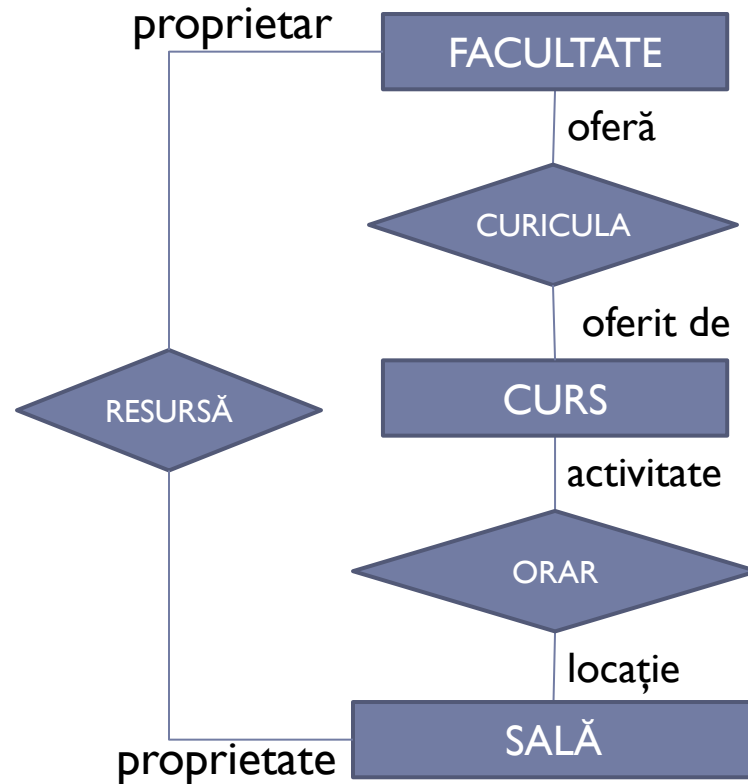
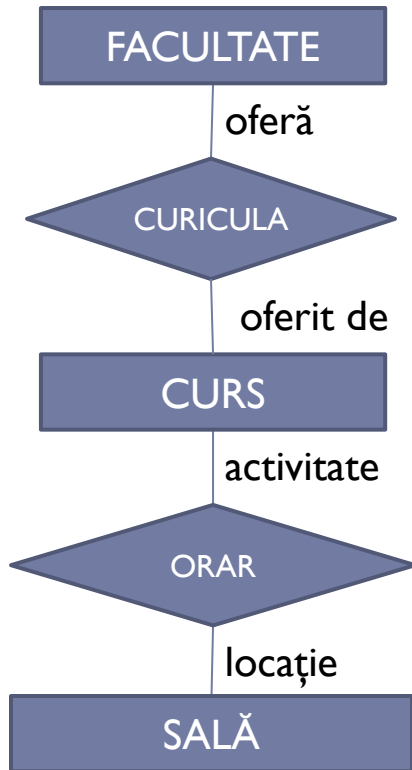
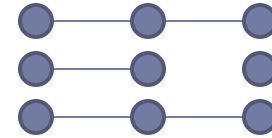


Problema:
La ce departament aparține profesorul X?



Soluția:
Model restructurat

Capcane de conectare (Chasm traps)



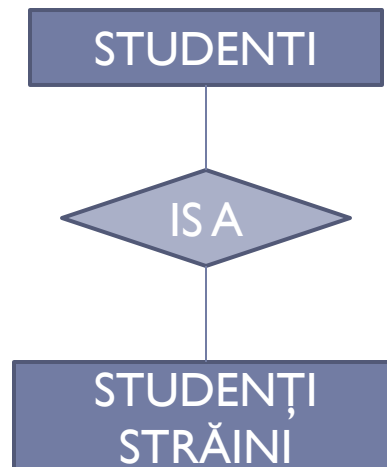
Problema:
Care sunt toate sălile ce aparțin unei facultăți?

Soluția:
Noi asocieri

Modelul E/A extins

Specializare

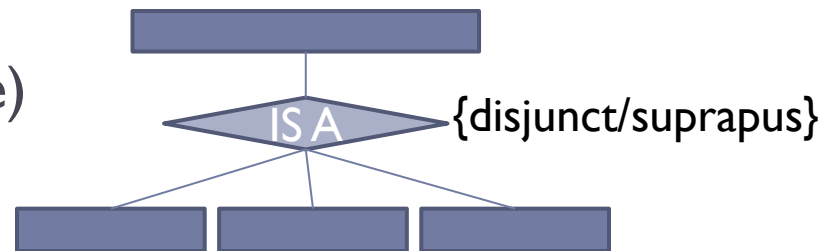
- ▶ Subgrupuri distinctive de instanțe-entități
 - ▶ Au în plus anumite atribute
 - ▶ Participă în asocieri la care nu participă toate instanțele-entități
 - ▶ Corespund unei mulțimi de entități specializate care se află într-o asociere de tip IS-A cu mulțimea de entități de bază



Constrângeri specifice specializării

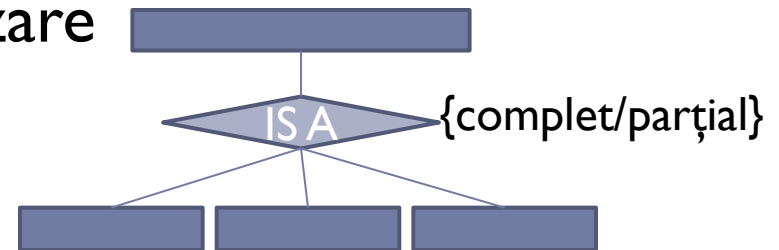
- ▶ Instanțele specializării moștenesc toate atributele și asocierile mulțimii de entități de bază, inclusiv cheia
- ▶ O instanță a unei mulțimi-entitate poate aparține la una sau la mai multe specializări

- ▶ Specializări disjuncte (exclusive)
- ▶ Specializări cu suprapunere



- ▶ O instanță a unei mulțimi-entitate trebuie sau nu să aparțină la cel puțin o specializare

- ▶ Complet
- ▶ Incomplet (parțial)



Modelare UML

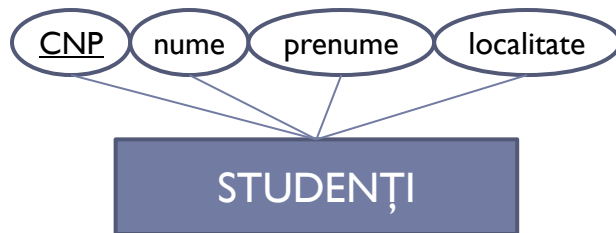
- ▶ **Unified Modeling Language**
 - ▶ Utilizat în ingineria software
 - ▶ Bazat pe concepte orientate obiect
 - ▶ Instrument de comunicare cu clientul în termenii utilizați în companie
 - ▶ Un limbaj foarte mare, utilizăm un set restrâns de elemente (diagrama de clase) pentru a modela o bază de date.

Mapare E/A – UML

E/R	UML
Mulțime-entitate cu atribute	Clasă
Mulțime-asociere fără atribute proprii	Asociere
Mulțime-asociere cu atribute proprii	Clasă de asociere
Specializare	Subclasă
	Compoziție și agregare

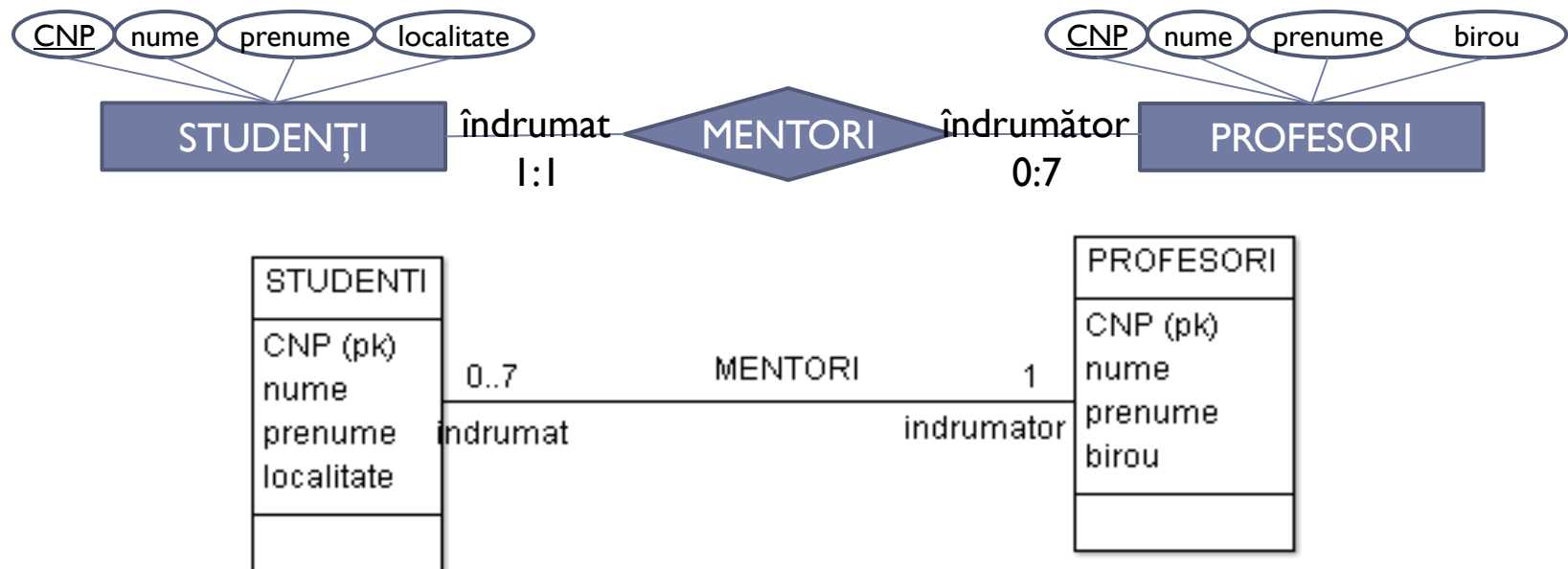
Clase

- ▶ Componente: nume, attribute, metode
- ▶ BD: nume, attribute (cheia primară)



Asocieri

- ▶ Exprimă asocierea dintre obiectele aparținând la **2** clase
- ▶ BD: asocierea dintre instanțele a două entități



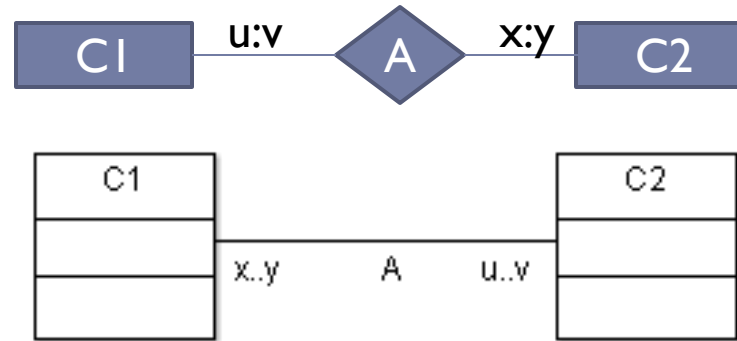
- ▶ Obs: constrângerile de cardinalitate se specifică invers decât în diagramele E/A

Asocieri

Constrângeri de conectivitate/multiplicitate

► Restricții

- $(C1, u, v, A)$
- $(C2, x, y, A)$



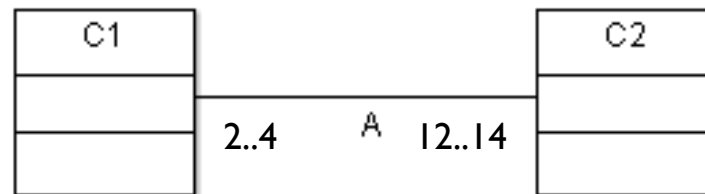
- Fiecare obiect din (instanță a entității) C1 este asociat cu cel puțin u și cel mult v obiecte din (instanțe ale entității) C2
- Fiecare obiect din (instanță a entității) C2 este asociat cu cel puțin x și cel mult y obiecte din (instanțe ale entității) C2

x..y	u..v	Tip asociere
0..1	0..1	unu la unu incompletă
1..1 (1)	1..1 (1)	unu la unu completă (implicită)
0..1	0..* (*)	unu la multi incompletă
...	...	...

???

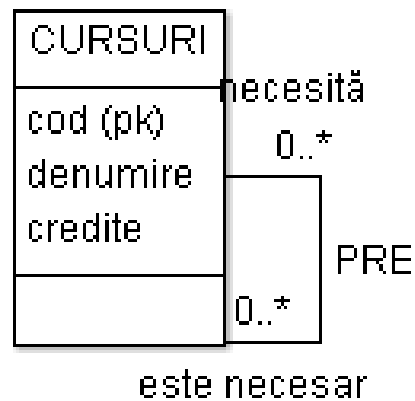
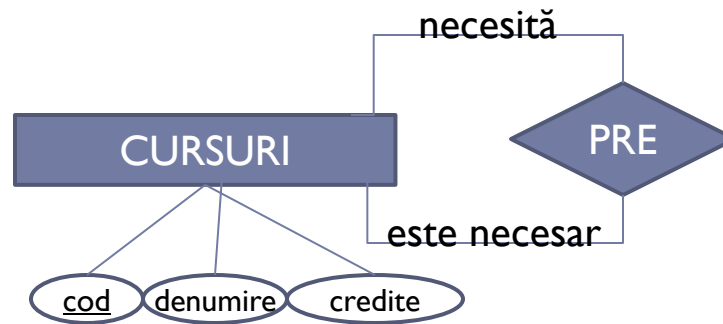
- ▶ Modelați asocierea dintre STUDENTI și UNIVERSITĂȚI. Un student poate studia la cel mult 2 universități și e necesar să studieze la cel puțin una. O universitate primește cel mult 10.000 studenți.

- ▶ Fie asocierea

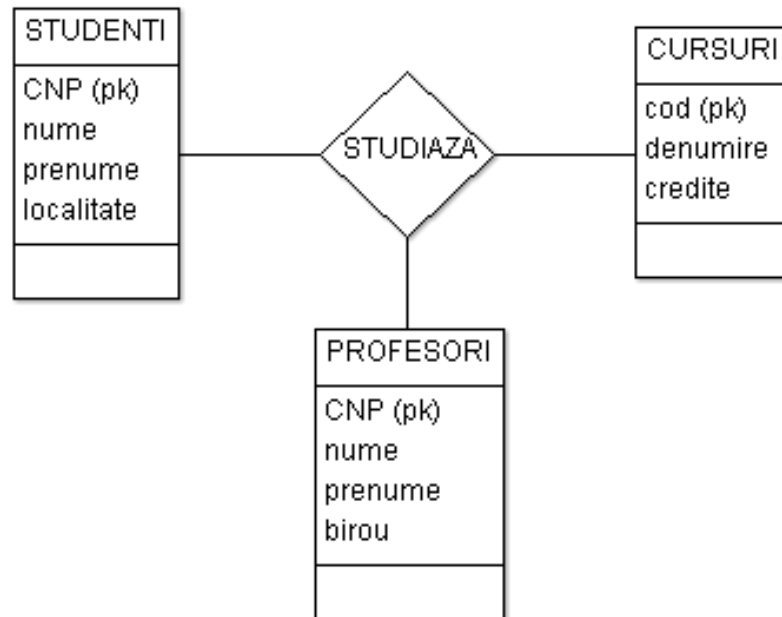


Care e numărul minim de instanțe pentru entitatea C1 și pentru C2?

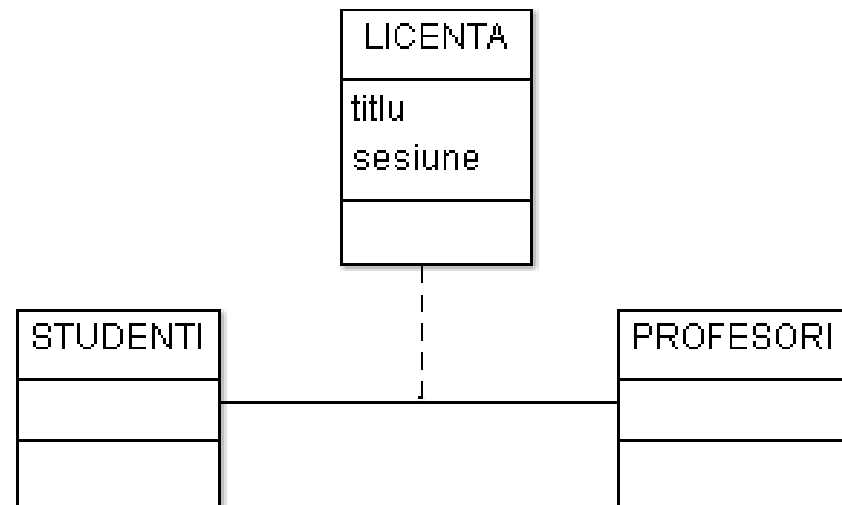
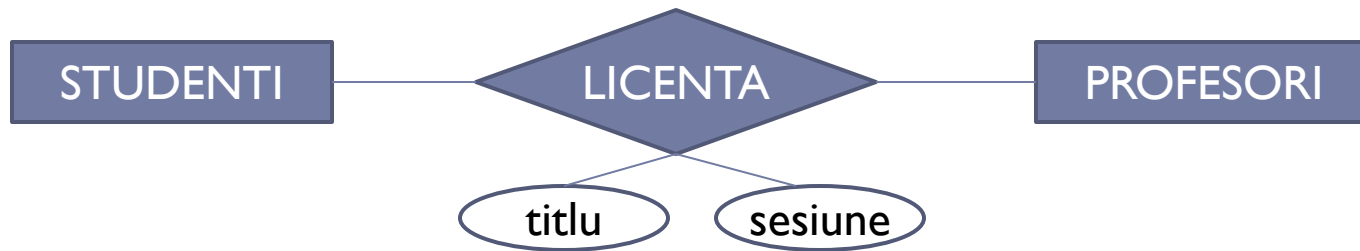
Asocieri recursive



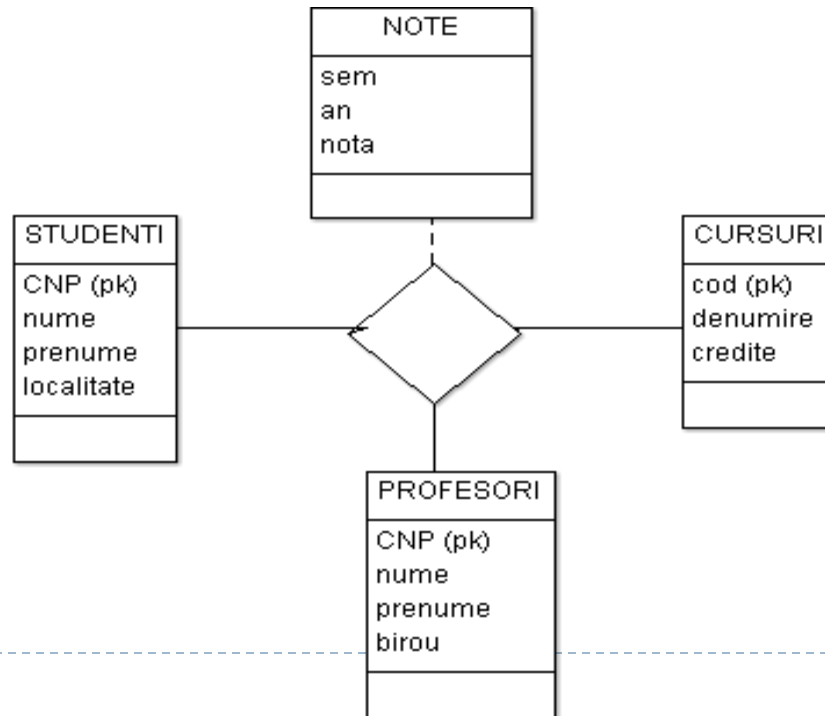
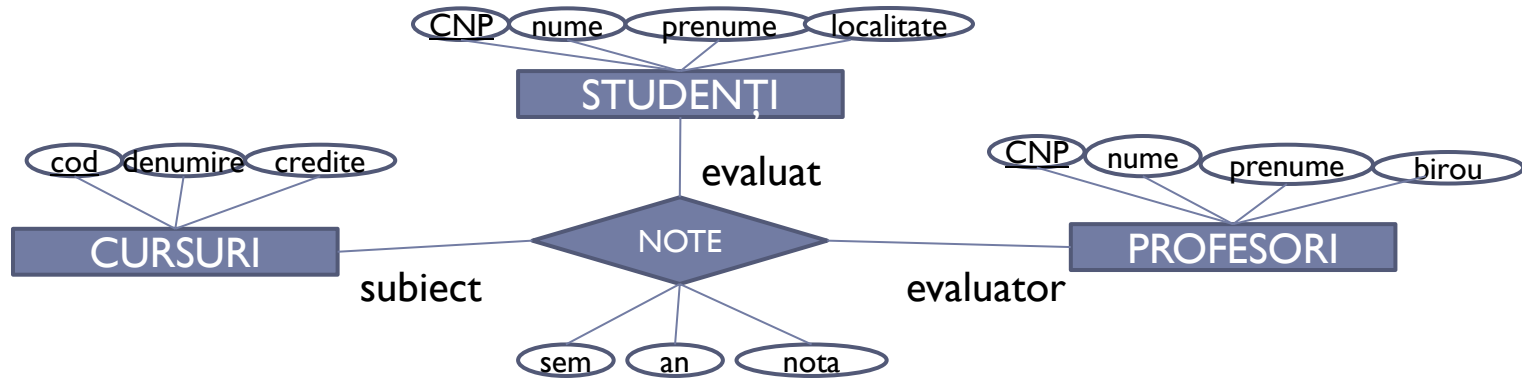
Asocieri n-are



Clase de asociere

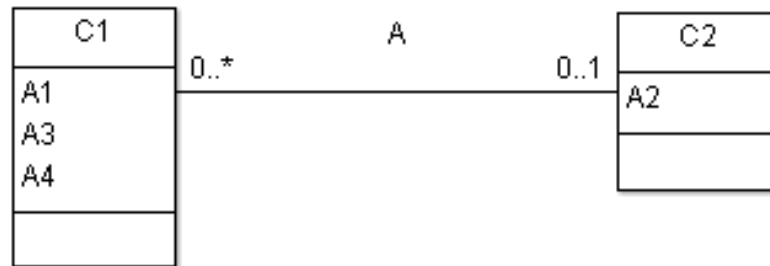
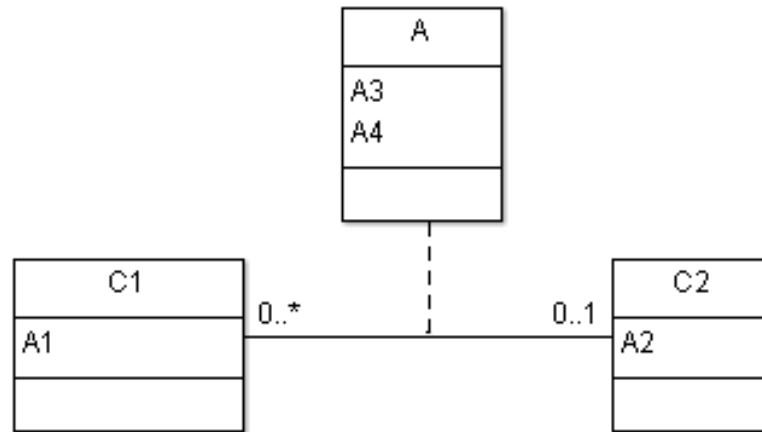


Clase de asociere

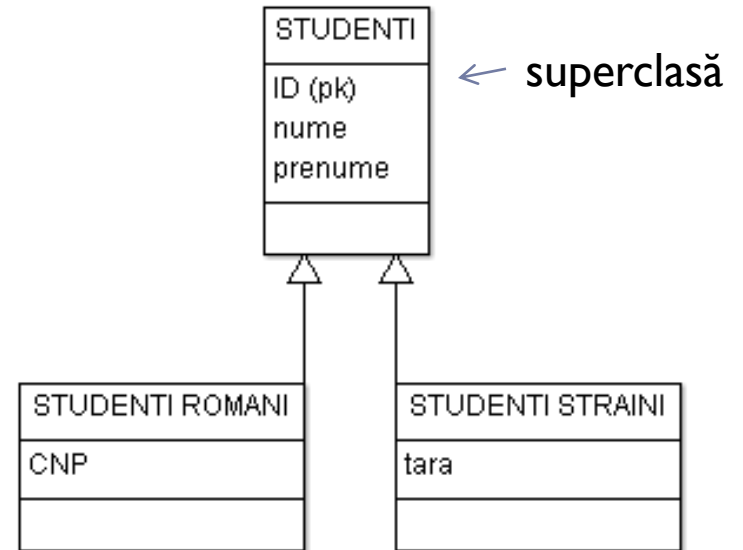
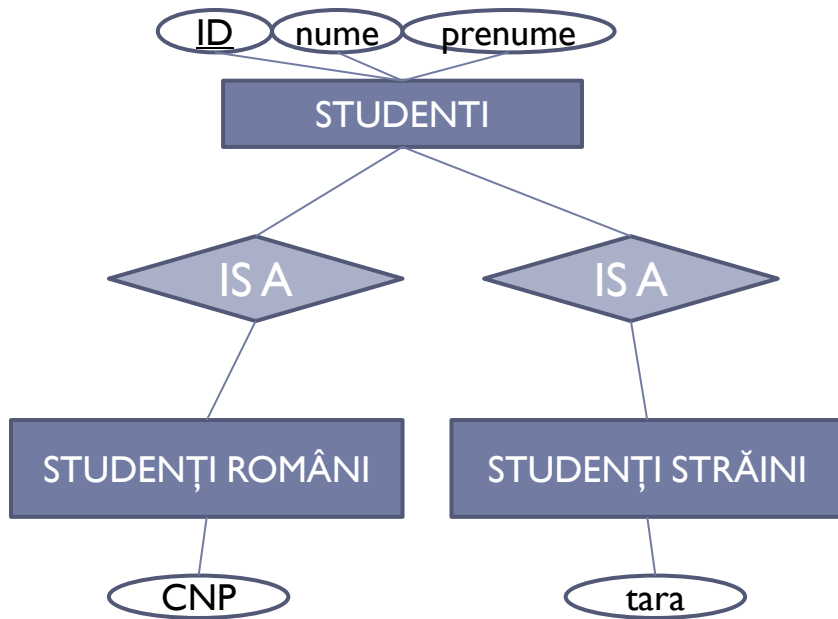


Eliminarea claselor de asociere

- Atunci când avem multiplicitate 0..1 sau 1..1

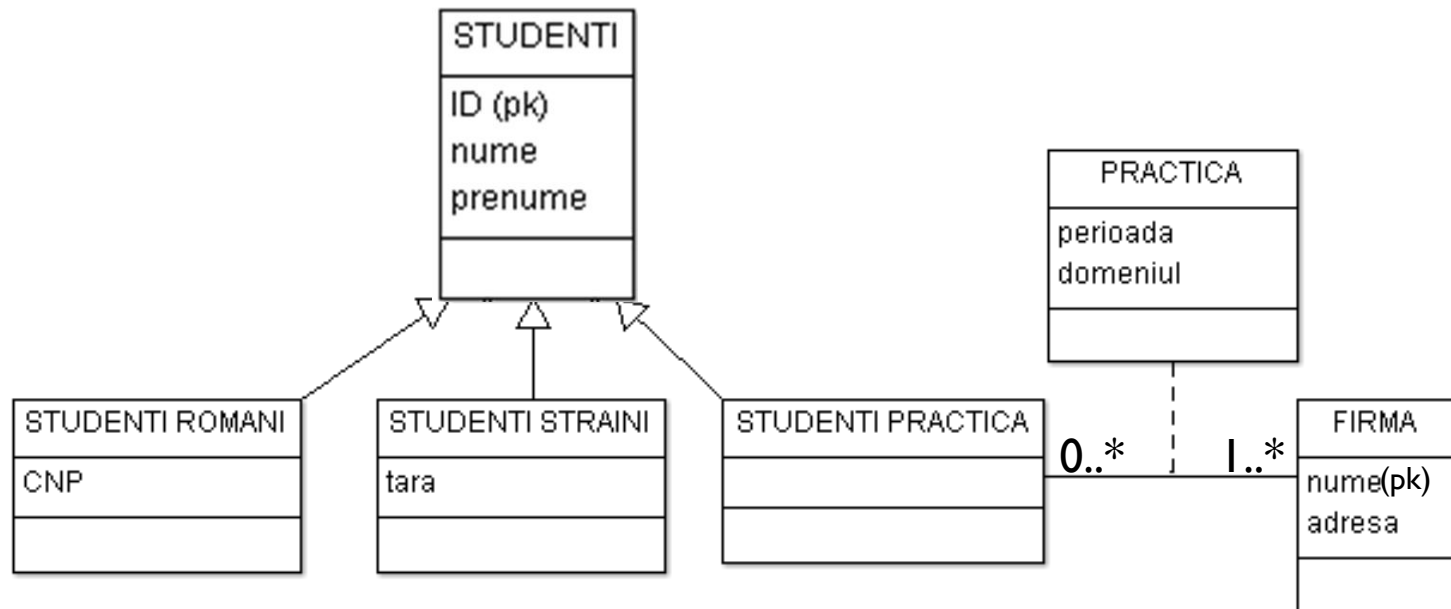


Subclasă (1)



Specializare completă, disjunctă

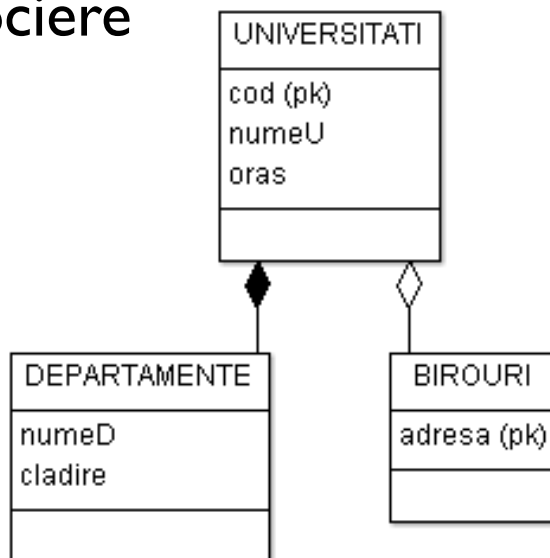
Subclasă (2)



Specializare completă, cu suprapunere

Compoziție și agregare

- ▶ Obiecte dintr-o clasă aparțin obiectelor din altă clasă
- ▶ Tipuri speciale de asociere

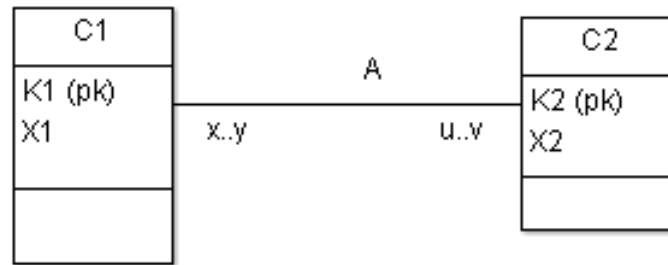


- ▶ Compoziția: **toate** obiectele unei clase *părți* aparțin obiectelor dintr-o clasă *compusă*; clasei *părți* îi corespunde de obicei o entitate slabă (multiplicitate 1..1; fără cheie primară);
- ▶ Agregarea: **unele** obiecte dintr-o clasă aparțin obiectelor din altă clasă (multiplicitate 0..1)

Mapare E/A, UML -> schema BD relațională

E/A	UML	Schema relațională
Mulțime-entitate cu atribut	Clasă	Relație cu cheie primară
Mulțime-asociere fără atribut proprii	Asociere	Relație cu chei străine
Mulțime-asociere cu atribut proprii	Clasă de asociere	Relație cu chei străine și alte atribute
Specializare	Subclasă	Relație cu cheie primară (cea a superclasei) și atribute particulare/specializate
	Compoziție și agregare	Relație cu cheie străină și atribute particulare

Mulțimi-entitate/clase și asocieri



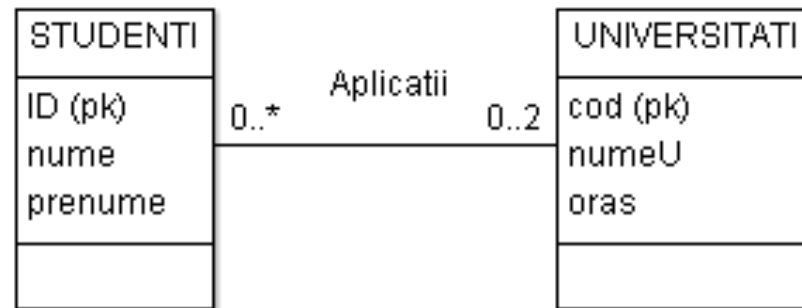
$\{C1(\underline{K1}, X1), C2(\underline{K2}, X2), A(K1, K2)\}$

► Cheia primară pentru asociere depinde de multiplicitate

x..y	u..v	Cheia primară pt A	Observații
0..1 1..1	*	K2	Nu e necesară relația A $\{C1(\underline{K1}, X1), C2(\underline{K2}, X2, K1)\}$
*	0..1 1..1	K1	Nu e necesară relația A $\{C1(\underline{K1}, X1, K2), C2(\underline{K2}, X2)\}$
*	*	(K1, K2)	

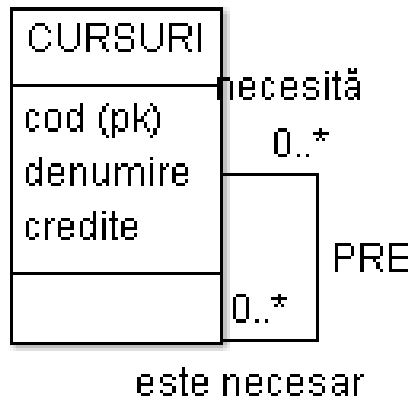
???

► Fie diagrama

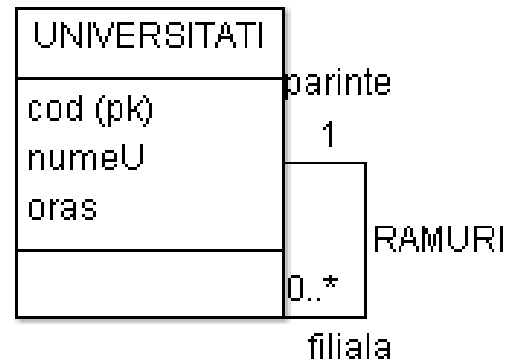


► Mai este posibilă renunțarea la relația corespunzătoare asocierii?

Asocieri recursive

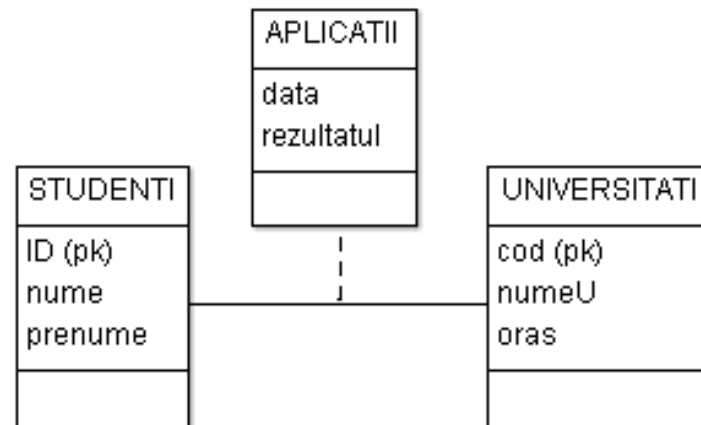


{CURSURI (cod, denumire, credite)
PRE (cod1, cod2)}



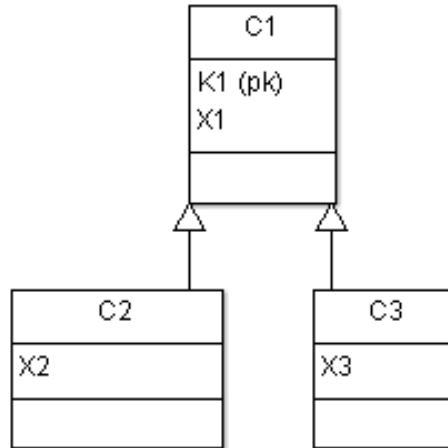
{UNIVERSITATI (cod, numeU, oras)
RAMURI (codFiliala, codParinte)}

Clase de asociere



{STUDENTI (ID, nume, prenume)
UNIVERSITATI (cod, numeU, oras)
APLICATII (ID, cod, data, rezultatul)}

Specializare / Subclase



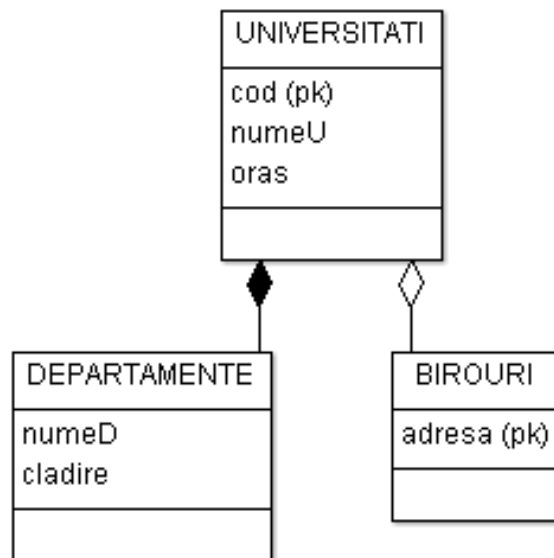
► Posibilități

- Relații subclasă ce conțin cheia superclasei și attributele specializate
 $C1(\underline{K1}, X1), C2(\underline{K1}, X2), C3(\underline{K1}, X3)$
- Relații subclasă ce conțin attributele superclasei (inclusiv atributul cheie) și attributele specializate; superclasa conține doar tuple nespecializate
 $C1(\underline{K1}, X1), C2(\underline{K2}, X1, X2), C3(\underline{K2}, X1, X3)$
- O singură relație ce conține attributele din superclasă și subclasă
 $C(\underline{K1}, X1, X2, X3)$

???

-
- Fie superclasa S cu un număr de subclase. Considerăm că relația de specializare este incompletă și cu suprapunere. Dacă n_1 , n_2 și n_3 reprezintă numărul total de tuple necesare fiecărei scheme de decodificare din cele 3 date anterior (în ordinea dată), care este relația dintre cele 3 valori?
- $n_1 < n_2 < n_3$
 - $n_1 \leq n_2 \leq n_3$
 - $n_3 < n_2 < n_1$
 - $n_3 \leq n_2 \leq n_1$

Compoziție și agregare



{ UNIVERSITATI(cod, numeU, oras)
DEPARTAMENTE(codU, numeD, cladire)
BIROURI (codU, adresa)}

← NU acceptă NULL
← acceptă NULL

Modelare EA/UML

Sumar

► PROS

- Tehnică populară de modelare conceptuală
- Construcții expresive, descriu punctul de vedere personal asupra aplicației
- Permite exprimarea unor tipuri de constrângeri (chei primare, străine, multiplicitate, exclusivitate...)

► CONS

- Tehnică subiectivă (entitate sau atribut, entitate sau asociere, subclasare sau nu, compoziție sau nu)
- Nu permite modelarea tuturor dependentelor
- Necesită utilizarea ulterioară a normalizării

Bibliografie

- ▶ Hector Garcia-Molina, Jeff Ullman, Jennifer Widom: *Database Systems: The Complete Book (2nd edition)*, Prentice Hall; (June 15, 2008)
- ▶ Thomas Connolly, Carolyn Begg: *Database Systems: A Practical Approach to Design, Implementation and Management*, (5th edition) Addison Wesley, 2009

- ▶ Unelte:
 - ▶ <https://createy.com> (diagrame EA, diagrame UML de clasă)
 - ▶ <http://diagramo.com/> (diagrame EA)
 - ▶ <http://argouml-downloads.tigris.org/nonav/argouml-0.32.2/ArgoUML-0.32.2.zip> (diagrame UML de clasă)

Baze de date relaționale / Dependențe

Nicolae-Cosmin Vârlan

October 31, 2015

Elemente ale modelului relațional

- ▶ U mulțime de atribute: $U = \{A_1, A_2, \dots, A_n\}$;
- ▶ $dom(A_i)$ - domeniul valorilor atributului A_i ;

Definim *uplu* peste U ca fiind funcția:

$$\varphi : U \rightarrow \bigcup_{1 \leq i \leq n} dom(A_i) \quad \text{a.i. } \varphi(A_i) \in dom(A_i), 1 \leq i \leq n$$

Fie valorile v_i astfel încât $v_i = \varphi(A_i)$.

Notăm cu $\{A_1 : v_1, A_2 : v_2, \dots, A_n : v_n\}$ asocierea dintre atributele existente în U și valorile acestora. În cazul în care sunt considerate mulțimi ordonate (de forma $(A_1, A_2, \dots, A_n)$), notația va fi de forma: $(v_1, v_2, \dots, v_n)$.

Elemente ale modelului relațional

Considerăm mulțimea ordonată $(A_1, A_2, \dots, A_n)$. Pentru orice uplu φ , există vectorul $(v_1, v_2, \dots, v_n)$ a.i. $\varphi(A_i) = v_i$, $1 \leq i \leq n$.

Pentru un vector $(v_1, v_2, \dots, v_n)$ cu $v_i \in \text{dom}(A_i)$, $1 \leq i \leq n$ există un uplu φ a.i. $\varphi(A_i) = v_i$.

În practică este considerată o anumită ordonare a atributelor.

Elemente ale modelului relațional

O mulțime de uple peste U se numește *relație* și se notează cu r .
 r poate varia în timp dar nu și în structură.

Exemplu:

$$r = \{(v_{11}, v_{12}, \dots, v_{1n}), (v_{21}, v_{22}, \dots, v_{2n}), \dots, (v_{m1}, v_{m2}, \dots, v_{mn})\}.$$

Structura relației se va nota cu $R[U]$ unde R se numește *numele relației* iar U este mulțimea de *atribute* corespunzătoare.

Notatii echivalente $R(U)$, $R(A_1, A_2, \dots, A_n)$, $R[A_1, A_2, \dots, A_n]$.

$R[U]$ se mai numește și *schemă de relație*.

Elemente ale modelului relațional

În practică, o relație r poate fi reprezentată printr-o matrice:

$$r : \begin{array}{cccc} & A_1 & A_2 & \dots & A_n \\ \hline & v_{11} & v_{12} & \dots & v_{1n} \\ & \dots & \dots & \dots & \dots \\ & v_{m1} & v_{m2} & \dots & v_{mn} \\ \hline \end{array}$$

unde $(v_{i1}, v_{i2}, \dots, v_{in})$ este un uplu din r , $1 \leq i \leq m$ și $v_{ij} \in \text{dom}(A_j)$, $1 \leq j \leq n$, $1 \leq i \leq m$

Vom nota cu t_i linia cu numărul i din matrice:

$$t_i = (v_{i1}, v_{i2}, \dots, v_{in})$$

Elemente ale modelului relațional

O mulțime finită D de scheme de relație se numește *schemă de baze de date*. Formal, $D = \{R_1[U_1], \dots, R_h[U_h]\}$ unde $R_i[U_i]$ este o schemă de relație, $1 \leq i \leq h$.

O *bază de date peste D* este o corespondență ce asociază fiecărei scheme de relație din D o relație.

Exemplu:

$r_1, r_2, \dots, r_h$ este o bază de date peste $D = \{R_1[U_1], \dots, R_h[U_h]\}$.

Considerând D ca fiind ordonată $D = (R_1[U_1], \dots, R_h[U_h])$, putem nota baza de date sub forma $(r_1, r_2, \dots, r_h)$

Operatii

Asupra unei multimi de relatii putem efectua o serie de operatii.
Exista doua categorii de operatori:

- ▶ operatori din teoria multimilor (reuniunea, intersectia, diferenta, produsul cartezian)
- ▶ operatori specifici algebrelor relationale (proiectia, selectia, joinul general, joinul natural)

Operații în cadrul modelului relațional - *reuniunea*

În cazul operațiilor pe mulțimi (cu excepția produsului cartezian), acestea se realizează între două relații. Trebuie să existe o compatibilitate, relativă la multimea de atribute peste care sunt construite cele două tuple, în sensul că acele atribute care apar în prima relație trebuie să apară și în a doua relație și reciproc.

Reuniunea a două relații r_1 și r_2 , ambele peste $R[U]$, este o relație notată cu $r_1 \cup r_2$ definită astfel:

$$r_1 \cup r_2 = \{t \mid t = \text{uplu}, \quad t \in r_1 \text{ sau } t \in r_2\}$$

În practică, acest lucru se realizează utilizând cuvântul cheie UNION. Studenții din anii 1 și 3 sunt selectați de interogarea:

```
SELECT * FROM studenti WHERE an=1  
UNION  
SELECT * FROM studenti WHERE an=3;
```

Operații în cadrul modelului relațional - *diferența*

Diferența a două relații r_1 și r_2 , ambele peste $R[U]$, este o relație notată cu $r_1 - r_2$ definită astfel:

$$r_1 - r_2 = \{t \mid t = \text{uplu}, \ t \in r_1 \text{ și } t \notin r_2\}$$

În practică, acest lucru se realizează utilizând cuvântul cheie MINUS. Pentru a-i selecta pe studenții din anul 2 fără bursă, putem să îi selectăm pe toți apoi să îi eliminăm pe cei cu bursă:

```
SELECT * FROM studenti WHERE an=2  
MINUS  
SELECT * FROM studenti WHERE bursa IS NOT NULL;
```

Se observă că, la fel ca în cazul reuniunii, cele două mulțimi de tuple peste care s-a făcut diferența sunt construite peste aceeași mulțime de atribute.

Operații în cadrul modelului relațional - *internsecția*

Internsecția a două relații r_1 și r_2 , ambele peste $R[U]$, este o relație notată cu $r_1 \cap r_2$ definită astfel:

$$r_1 \cap r_2 = \{t \mid t = uplu, \quad t \in r_1 \text{ si } t \in r_2\}$$

În practică, acest lucru se realizează utilizând cuvântul cheie INTERSECT. Putem afla care studenți din anul 2 au bursa rulant:

```
SELECT * FROM studenti WHERE an=2  
INTERSECT  
SELECT * FROM studenti WHERE bursa IS NOT NULL;
```

Operatorul de intersecție poate fi obținut din ceilalți doi:

$$r_1 \cap r_2 = r_1 - (r_1 - r_2)$$

Operații în cadrul modelului relațional - *produs cartezian*

Produsul cartezian a două relații r_1 definită peste $R_1[U_1]$ și r_2 definită peste $R_2[U_2]$ cu $U_1 \cap U_2 = \emptyset$ este o relație notată cu $r_1 \times r_2$ definită astfel:

$$r_1 \times r_2 = \{t \mid t = \text{uplu peste } U_1 \cup U_2, t[U_1] \in r_1 \text{ si } t[U_2] \in r_2\}$$

De aceasta data, cele doua relatii nu trebuie sa fie peste aceeasi multime de attribute. Rezultatul va fi o noua relatie peste o multime de attribute formata din attributele relatiilor initiale.

Operații în cadrul modelului relațional - *produs cartezian*

Daca un atribut s-ar repeta, el va fi identificat diferit. Spre exemplu, chiar daca tabelele note si cursuri au un acelasi camp, nu se face nici o sincronizare dupa acesta ci vor avea ca efect crearea a doua attribute diferite: r1.camp, r2.camp.

Produsul cartezian intre aceste tabele, in practica, se obtine executand interogarea:

```
SELECT * FROM cursuri, note;
```

Operații specifice algebrelor relaționale

Operațiile pe mulțimi aveau ca elemente tuplele. Uneori aceste tuple nu sunt compatibile (de exemplu nu putem reuni o relație peste $R_1[U_1]$ cu $R_2[U_2]$).

Pentru a opera asupra atributelor ce definesc tuplele din rezultat, avem nevoie de o serie de operatori specifici algebrelor relationale: proiecția, și diverse variații ale join-ului.

Operații în cadrul modelului relațional - *proiecția*

Considerăm:

- ▶ $R[U]$ = schemă de relație;
- ▶ $X \subseteq U$;
- ▶ t = uplu peste $R[U]$ ($t \in r$).

Se numește *proiecția lui t relativă la X* și notată cu $t[X]$, restricția lui t la mulțimea de atribute X .

Exemplu:

Dacă $U = (A_1, A_2, \dots, A_n)$ atunci $t = (v_1, v_2, \dots, v_n)$.

Considerăm $X = (A_{i_1}, A_{i_2}, \dots, A_{i_k})$, $1 \leq i_1 < i_2 < \dots < i_k \leq n$.

atunci $t[X] = (v_{i_1}, v_{i_2}, \dots, v_{i_k})$;

Operații în cadrul modelului relațional - *proiecția*

Dacă r este o relație peste $R[U]$ și $X \subseteq U$, atunci *proiecția lui r relativă la X* este $r[X] = \{t[X] \mid t \in r\}$

Exemplu:

Dacă $U = (A_1, A_2, \dots, A_n)$ atunci

$r = \{(v_{11}, v_{12}, \dots, v_{1n}), (v_{21}, v_{22}, \dots, v_{2n}), \dots, (v_{m1}, v_{m2}, \dots, v_{mn})\}$.

Considerăm $X = (A_{i_1}, A_{i_2}, \dots, A_{i_k})$, $1 \leq i_1 < i_2 < \dots < i_k \leq n$.

atunci

$r[X] = \{(v_{1_{i_1}}, v_{1_{i_2}}, \dots, v_{1_{i_k}}), (v_{2_{i_1}}, \dots, v_{2_{i_k}}), \dots, (v_{m_{i_1}}, \dots, v_{m_{i_k}})\}$

În practică, proiecția se realizează selectând doar anumite campuri ale tabelului (anumite atribute):

SELECT nume, prenume **FROM** studenti;

Operații în cadrul modelului relațional - *proiecția*

Ca si exemplu, vom scrie o interogare care sa returneze toate persoanele care trec pragul Facultatii (studenti si profesori):

```
SELECT nume, prenume FROM studenti  
UNION  
SELECT nume, prenume FROM profesori;
```

În cazul în care câmpurile cele două câmpuri (nume, prenume) din cele două tabele au același tip (de exemplu nume este de tip VARCHAR2(10) în ambele tabele), interogarea va afișa toate persoanele ce “trec pragul Facultatii”.

Observație: Pentru a modifica tipul nume din tabela profesori la VARCHAR2(10) executați comanda:
ALTER TABLE profesori MODIFY nume VARCHAR2(10);

Operații în cadrul modelului relațional - *join* (natural)

Considerăm:

- ▶ r_1 relație peste $R_1[U_1]$;
- ▶ r_2 relație peste $R_2[U_2]$;

Se numește *join* (sau *unire*) a relațiilor r_1 și r_2 , relația $r_1 * r_2$ peste $U_1 \cup U_2$ definită prin:

$$r_1 * r_2 = \{t \mid t \text{ uplu peste } U_1 \cup U_2, t[U_i] \in r_i, i = 1, 2\}$$

Dacă R este un nume pentru relația peste $U_1 \cup U_2$ atunci $r_1 * r_2$ este definită peste $R[U_1 \cup U_2]$

Pentru simplitate vom nota $U_1 \cup U_2$ cu U_1U_2 .

Operații în cadrul modelului relațional - *join* (natural)

Exemplu:

Fie $R_1[A, B, C, D]$, și $R_2[C, D, E]$ și r_1, r_2 a.i.:

$r_1 :$	A	B	C	D	$r_2 :$	C	D	E
	0	1	0	0		1	1	0
	1	1	0	0		1	1	1
	0	0	1	0		0	0	0
	1	1	0	1		1	0	0
	0	1	0	1		1	0	1

Atunci: $r_1 * r_2 :$

A	B	C	D	E
0	1	0	0	0
1	1	0	0	0
0	0	1	0	0
0	0	1	0	1

Operații în cadrul modelului relațional - *join* (natural)

Urmatoarea interogare identifica cui apartine fiecare nota din tabelul note. Joinul se face dupa campul nr_matricol intre tabelele studenti si note:

```
SELECT nume, valoare FROM studenti  
JOIN note ON studenti.nr_matricol = note.nr_matricol
```

Se poate observa ca daca din produsul cartezian am elimina acele cazuri in care campul "nr_matricol" nu este identic in ambele tabele, am obtine, de fapt, acelasi rezultat. Din acest motiv, joinul de mai sus poate fi scris si sub forma:

```
SELECT nume, valoare FROM studenti,note  
WHERE studenti.nr_matricol = note.nr_matricol
```

Proprietăți *join* (natural)

- ▶ $r_1 * r_2[U_1] \subseteq r_1$
- ▶ $r_2 * r_1[U_2] \subseteq r_2$

Dacă $X = U_1 \cap U_2$ și:

$r'_1 = \{t_1 | t_1 \in r_1, \exists t_2 \in r_2 \text{ a.i. } t_1[X] = t_2[X]\}$ și $r_1'' = r_1 - r'_1$,

$r'_2 = \{t_2 | t_2 \in r_2, \exists t_1 \in r_1 \text{ a.i. } t_1[X] = t_2[X]\}$ și $r_2'' = r_2 - r'_2$,

atunci: $r_1 * r_2 = r'_1 * r'_2$, $r_1 * r_2[U_1] = r'_1$, $r_2 * r_1[U_2] = r'_2$.

Dacă $\overline{r_1} \subseteq r_1$, $\overline{r_2} \subseteq r_2$ și $\overline{r_1} * \overline{r_2} = r_1 * r_2$ atunci $r'_1 \subseteq \overline{r_1}$ și $r'_2 \subseteq \overline{r_2}$

Dacă $U_1 \cap U_2 = \emptyset$ atunci $r_1 * r_2 = r_1 \times r_2$.

Extindere *join* (natural)

Fie r_i relație peste $R_i[U_i]$, $i = \overline{1, h}$ atunci:

$$r_1 * r_2 * \dots * r_h = \{t \mid t \text{ uplu peste } U_1, \dots, U_h, \text{ a.i. } t[U_i] \in r_i, i = \overline{1, h}\}$$

Notății echivalente:

- ▶ $r_1 * r_2 * \dots * r_h$
- ▶ $\bowtie \langle r_i, i = 1, h \rangle$
- ▶ $* \langle r_i, i = 1, h \rangle$

Operația join este asociativă.

Operații în cadrul modelului relațional - *join* (oarecare)

Fie r_i peste $R_i[U_i]$, $i = \overline{1, 2}$ cu $A_{\alpha_1}, A_{\alpha_2}, \dots, A_{\alpha_k} \in U_1$ și $B_{\beta_1}, B_{\beta_2}, \dots, B_{\beta_k} \in U_2$ și θ_i operator de comparație între elementele lui $dom(A_{\alpha_i})$ și cele ale lui $dom(B_{\beta_i})$

θ_i este relație binară peste $dom(A_{\alpha_i}) \times dom(B_{\beta_i})$, $1 \leq i \leq k$.

Join-ul oarecare a două relații r_1 și r_2 , notat cu $r_1 \bowtie_{\theta} r_2$, este definit prin:

$$r_1 \bowtie_{\theta} r_2 = \{(t_1, t_2) | t_1 \in r_1, t_2 \in r_2, t_1[A_{\alpha_i}] \theta_i t_2[B_{\beta_i}], i = \overline{1, k}\}$$

unde $\theta = (A_{\alpha_1} \theta_1 B_{\beta_1}) \wedge (A_{\alpha_2} \theta_2 B_{\beta_2}) \wedge \dots \wedge (A_{\alpha_k} \theta_k B_{\beta_k})$

Observație: un join oarecare cu condiția TRUE pentru orice combinație de tuple este un produs cartezian.

Observatie2: Joinul oarecare poate fi considerat ca fiind o filtrare după anumite criterii ale rezultatelor unui produs cartezian.

Operații în cadrul modelului relațional - *join* (oarecare)

Atunci când dorim să selectăm doar anumite rânduri (WHERE), se poate face un join oarecare între tabela pe care trebuie făcută selecția și tabela dual.

Vor fi selectate doar acele câmpuri care ne interesează (nu și cele din DUAL) și acestea se vor conforma condițiilor puse.

```
SELECT nume FROM studenti, dual  
WHERE studenti.bursa IS NOT NULL;
```

Operații în cadrul modelului relațional - *selecția*

Fie r o relație peste $R[U]$.

Considerăm pentru început **expresiile elementare** de selecție:
 $A\theta B$, $A\theta c$, $c\theta B$, unde $A, B \in U$ și c este o constantă.

Dacă e_1 și e_2 sunt expresii de selecție (elementare sau nu), atunci următoarele sunt expresii de selecție: (e_1) , $e_1 \wedge e_2$, $e_1 \vee e_2$, $(e_1 \wedge e_2)$, $(e_1 \vee e_2)$.

Operații în cadrul modelului relațional - *selecția*

Fie E o expresie de selecție. Atunci:

- ▶ când $E = A\theta B$, t satisface E dacă $t[A] \theta t[B]$,
- ▶ când $E = A\theta c$, t satisface E dacă $t[A] \theta c$,
- ▶ când $E = c\theta B$, t satisface E dacă $c \theta t[B]$,
- ▶ când $E = e_1 \wedge e_2$, t satisface E dacă t satisface atât pe e_1 cât și pe e_2 ,
- ▶ când $E = e_1 \vee e_2$, t satisface E dacă t satisface măcar pe unul dintre e_1 și e_2 .

Dacă F este o expresie de selecție atunci *selecția* se notează cu $\sigma_F(r)$ și este definită ca:

$$\sigma_F(r) = \{t \mid t = \text{tuplupeste } R[U], t \text{ satisface } F\}$$

Exerciții:

Utilizand schema de baze de date de la laborator, scrieti in algebra relationala urmatoarele:

- ▶ Cursurile din facultate impreuna cu numele profesorilor care le tin.
- ▶ Numele si prenumele studentilor din anul 1 si care au bursa mai mare de 300 ron.
- ▶ Prenumele studentilor care au acelasi nume de familie ca macar unul din profesori.
- ▶ Numele si prenumele studentilor, cursurile pe care le-au urmat si notele pe care le-au obtinut.

Scrieti interogările SQL asociate formulelor din algebra relationala scrise mai sus.

Notatii (alternative) operatori alg. relationala

Proiectia ($r_1[U]$): $\pi_U(r_1)$

Join natural ($r_1 * r_2$): $r_1 \bowtie r_2$

Join oarecare: $r_1 \bowtie_{\theta} r_2$

Selectia : $\sigma_{\theta}(r_1)$ [obs: $r_1 \bowtie_{\theta} r_2 = \sigma_{\theta}(r_1 \times r_2)$]

Join la stanga: $r_1 \triangleright \circ \triangleleft_L r_2$

Join la dreapta: $r_1 \triangleright \circ \triangleleft_R r_2$

Full outer join : $r_1 \triangleright \circ \triangleleft r_2$

Redenumirea: Daca r este definit peste $B_1, B_2, \dots, B_n$ si vrem sa redenumim numele atributelor, vom folosi operatorul de redenumire $\rho : r' = \rho(r)_{A_1, A_2, \dots, A_n}$ - redenumirea atributelor lui r in $A_1, A_2, \dots, A_n$

Dependențe funcționale

Dependențe funcționale

Fie $X, Y \subseteq U$. Vom nota o dependență funcțională cu $X \rightarrow Y$.

O relație r peste U satisface **dependența funcțională** $X \rightarrow Y$ dacă:

$$(\forall t_1, t_2)(t_1, t_2 \in r)[t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]]$$

$X = \emptyset$ avem $\emptyset \rightarrow Y$ dacă $(\forall t_1, t_2)(t_1, t_2 \in r)[t_1[Y] = t_2[Y]]$

$Y = \emptyset$ atunci orice $\forall r$ peste U avem că $X \rightarrow \emptyset$

Dacă r satisface $X \rightarrow Y$, atunci există o funcție $\varphi : r[X] \rightarrow r[Y]$ definită prin $\varphi(t) = t'[Y]$, unde $t' \in r$ și $t'[X] = t \in r[X]$.

Dacă r satisface $X \rightarrow Y$ spunem că X determină funcțional pe Y în r .

Proprietăți ale dependențelor funcționale

FD1. (**Reflexivitate**) Dacă $Y \subseteq X$, atunci r satisface $X \rightarrow Y$, $\forall r \in U$.

FD2. (**Extensie**) Dacă r satisface $X \rightarrow Y$ și $Z \subseteq W$, atunci r satisface $XW \rightarrow YZ$.

FD3. (**Tranzitivitate**) Dacă r satisface $X \rightarrow Y$ și $Y \rightarrow Z$, atunci r satisface $X \rightarrow Z$.

FD4. (**Pseudotranzitivitate**) Dacă r satisface $X \rightarrow Y$ și $YW \rightarrow Z$, atunci r satisface $XW \rightarrow Z$.

Proprietăți ale dependențelor funcționale

FD5. (**Uniune**) Dacă r satisface $X \rightarrow Y$ și $X \rightarrow Z$, atunci r satisface $X \rightarrow YZ$.

FD6. (**Descompunere**) Dacă r satisface $X \rightarrow YZ$, atunci r satisface $X \rightarrow Y$ și $X \rightarrow Z$.

FD7. (**Proiectabilitate**) Dacă r peste U satisface $X \rightarrow Y$ și $X \subset Z \subseteq U$, atunci $r[Z]$ satisface $X \rightarrow Y \cap Z$

FD8. (**Proiectabilitate inversă**) Dacă $X \rightarrow Y$ este satisfăcută de o proiecție a lui r , atunci $X \rightarrow Y$ este satisfăcută de r .

Dependențe funcționale - consecință și acoperire

Dacă Σ este o mulțime de dependențe funcționale peste U atunci spunem că $X \rightarrow Y$ *este consecință din Σ* dacă orice relație ce satisface toate consecințele din Σ satisface și $X \rightarrow Y$.

Notăție: $\Sigma \models X \rightarrow Y$

Fie $\Sigma^* = \{X \rightarrow Y \mid \Sigma \models X \rightarrow Y\}$. Fie $\Sigma_1 =$ mulțime de dependențe funcționale. Σ_1 constituie o *acoperire* pentru Σ^* dacă $\Sigma_1^* = \Sigma^*$.

Proprietăți ale dependențelor funcționale

Propoziție

Pentru orice mulțime Σ de dependențe funcționale există o acoperire Σ_1 pentru Σ^ , astfel încat toate dependențele din Σ_1 sunt de forma $X \rightarrow A$, A fiind un atribut din U .*

Propoziție

$\Sigma \models X \rightarrow Y$ dacă și numai dacă $\Sigma \models X \rightarrow B_j$ pentru $j = \overline{1, h}$, unde $Y = B_1 \dots B_h$.

Reguli de deducere

Fie $\mathcal{R}$ o mulțime de formule de deducere pentru dependențe funcționale și Σ o mulțime de dependențe funcționale. Spunem că $X \rightarrow Y$ este o **demonstrație** în Σ utilizând regulile $\mathcal{R}$ și vom nota $\Sigma \vdash_{\mathcal{R}} X \rightarrow Y$, dacă există șirul $\sigma_1, \sigma_2, \dots, \sigma_n$, astfel încât:

- ▶ $\sigma_n = X \rightarrow Y$ și
- ▶ pentru $\forall i = \overline{1, n}$, $\sigma_i \in \Sigma$ sau există în $\mathcal{R}$ o regulă de forma $\frac{\sigma_{j_1}, \sigma_{j_2}, \dots, \sigma_{j_k}}{\sigma_i}$, unde $j_1, j_2, \dots, j_k < i$.

Reguli de deducere

Conform proprietăților FD1-FD5 putem defini regulile:

$$\text{FD1f: } \frac{Y \subseteq X}{X \rightarrow Y}$$

$$\text{FD4f: } \frac{X \rightarrow Y, YW \rightarrow Z}{XW \rightarrow Z}$$

$$\text{FD2f: } \frac{X \rightarrow Y, Z \subseteq W}{XW \rightarrow YZ}$$

$$\text{FD5f: } \frac{X \rightarrow Y, X \rightarrow Z}{X \rightarrow YZ}$$

$$\text{FD3f: } \frac{X \rightarrow Y, Y \rightarrow Z}{X \rightarrow Z}$$

$$\text{FD6f: } \frac{X \rightarrow YZ, X \rightarrow YZ}{X \rightarrow Y, X \rightarrow Z}$$

Propoziție

Regulile FD4f, FD5f, FD6f se exprimă cu ajutorul regulilor FD1f, FD2f, FD3f.

Notăm cu $\mathcal{R}_1 = \{\text{FD1f, FD2f, FD3f}\}$,
și cu $\mathcal{R}_2 = \mathcal{R}_1 \cup \{\text{FD4f, FD5f, FD6f}\}$

Propoziție

Regulile FD4f, FD5f, FD6f se exprimă cu ajutorul regulilor FD1f, FD2f, FD3f.

Idei de demonstrație:

- ▶ FD4f: Se aplica FD2f pentru $X \rightarrow Y$ și $W \subseteq W$ iar din rezultat și din $YW \rightarrow Z$ prin FD3f se obține rezultatul;
- ▶ FD5f: Se aplica FD2f pentru $X \rightarrow Y$ și $X \subseteq X$ și la fel pentru $X \rightarrow Z$ și $Y \subseteq Y$ apoi FD3f (tranzitivitatea) între rezultate;
- ▶ FD6f: din FD1f avem ca $YZ \rightarrow Y$ și $YZ \rightarrow Z$ și din FD3f rezulta $X \rightarrow Y$ și $X \rightarrow Z$

Axiomele lui Armstrong

Armstrong a definit (în *Dependency structures of database relationships* Proc. IFIP 74, Amsterdam, 580-583) următoarele reguli de inferență (numite *Axiomele lui Armstrong*):

$$A1: \frac{}{A_1 \dots A_n \rightarrow A_i}, i = \overline{1, n}$$

$$A2: \frac{A_1, \dots, A_m \rightarrow B_1, \dots, B_r}{A_1 \dots A_m \rightarrow B_j}, j = \overline{1, r}$$

$$\frac{A_1, \dots, A_m \rightarrow B_j, j = \overline{1, r}}{A_1 \dots A_m \rightarrow B_1, \dots, B_r}$$

$$A3: \frac{A_1, \dots, A_m \rightarrow B_1, \dots, B_r, \quad B_1, \dots, B_r \rightarrow C_1, \dots, C_p}{A_1 \dots A_m \rightarrow C_1, \dots, C_p}$$

unde A_i, B_j, C_k sunt atribute. Notăm $\mathcal{R}_A = \{A1, A2, A3\}$.

Obs: regula A3 este de fapt FD3f (tranzitivitatea).

Propoziție

Regulele din $\mathcal{R}_1$ se exprimă prin cele din $\mathcal{R}_A$ și invers.

Notatie:

$$\Sigma_{\mathcal{R}}^+ = \{X \rightarrow Y \mid \Sigma \vdash_{\mathcal{R}} X \rightarrow Y\}$$

Propoziție

Fie $\mathcal{R}'_1$ și $\mathcal{R}'_2$ două mulțimi de reguli astfel încât $\mathcal{R}'_1$ se exprima prin $\mathcal{R}'_2$ și invers. Atunci $\Sigma_{\mathcal{R}'_1}^+ = \Sigma_{\mathcal{R}'_2}^+$ pentru orice mulțime Σ de dependente functionale.

Consecința: $\Sigma_{\mathcal{R}_1}^+ = \Sigma_{\mathcal{R}_A}^+$

Fie $X \subseteq U$ și $\mathcal{R}$ o mulțime de reguli de inferență. Notăm cu

$$X_{\mathcal{R}}^+ = \{A \mid \Sigma \vdash_{\mathcal{R}} X \rightarrow A\}$$

Lema

$\Sigma \vdash_{\mathcal{R}} X \rightarrow Y$ dacă și numai dacă $Y \subseteq X_{\mathcal{R}_1}^+$.

Lema

Fie Σ o multime de dependente functionale si $\sigma : X \rightarrow Y$ o dependenta functionala astfel incat $\Sigma \not\models_{\mathcal{R}_1} X \rightarrow Y$. Atunci exista o relatie r_σ ce satisface toate dependentele functionale din Σ si r_σ nu satisface $X \rightarrow Y$.

Theorem

Fie Σ o multime de dependente functionale. Atunci exista o relatie r_0 ce satisface exact elementele lui $\Sigma_{\mathcal{R}_1}^+$, adica:

- ▶ r_0 satisface τ , $\forall \tau \in \Sigma_{\mathcal{R}_1}^+$ si
- ▶ r_0 nu satisface γ , $\forall \gamma \notin \Sigma_{\mathcal{R}_1}^+$

Dependențe multivaluate

Exemplu

Presupunem că persoana cu $CNP = 1$ a fost admisă la două facultăți și are permis de conducere pentru categoriile A și B :

	CNP	Admis la facult.	Are permis categ.
$r :$	1	Informatică	A
	1	Matematică	B

Deși anumite rânduri nu sunt scrise în tabelă, putem să intuim că persoana cu $CNP = 1$ a dat la Facultatea de Informatică și are permis de conducerea categoria B . Deci, deși în r nu există t -uplul $\langle 1, \text{Informatica}, B \rangle$, ar trebui să existe și el (pentru că poate fi dedus din cele existente).

Care alt t -uplu mai poate fi dedus ?

Exemplu

	CNP	Admis la facult.	Are permis categ.
$r :$	1	Informatică	A
	1	Matematică	B
	1	Informatică	B
	1	Matematică	A

t -uplele marcate cu roșu ar putea lipsi, ele fiind redundante deoarece pot fi obținute din primele două t -uple.

Prin intermediul dependențelor funcționale pot afla la care coloane pot renunța astfel încât să le pot reface ulterior.

Prin intermediul dependențelor multivaluate pot afla la care linii pot renunța astfel încât să le pot reface ulterior.

Dependențe multivaluate - definiție

Fie $X, Y \subseteq U$. O dependență multivaluată este notată cu $X \twoheadrightarrow Y$.

Definition

Relația r *peste* U *satisface dependența multivaluată* $X \twoheadrightarrow Y$ dacă pentru oricare două tuple $t_1, t_2 \in r$ și $t_1[x] = t_2[x]$, există tuplele t_3 și t_4 din r , astfel încât:

- ▶ $t_3[X] = t_1[X], t_3[Y] = t_1[Y], t_3[Z] = t_2[Z];$
- ▶ $t_4[X] = t_2[X], t_4[Y] = t_2[Y], t_4[Z] = t_1[Z]$

unde $Z = U - XY$ (Z mai este denumită și *rest*).

Exemplul 2 (mai formal)

$r :$	A	B	C	D			
	a_1	b_1	c_1	d_1	t_1		t_1''
	a_1	b_2	c_2	d_2	t_2		
	a_1	b_1	c_1	d_2	t_3		t_2''
	a_1	b_2	c_2	d_1	t_4		
	a_2	b_3	c_1	d_1		t'_1, t'_4	
	a_2	b_3	c_1	d_2		t'_2, t'_3	

r satisface $A \twoheadrightarrow BC$

Intrebare: cum alegem t_3'' , t_4'' ?

Deoarece atunci când $t_1[A] = t_2[A]$ avem că:

$t_3[A] = t_1[A], t_3[BC] = t_1[BC], t_3[D] = t_2[D]$ și

$t_4[A] = t_2[A], t_4[BC] = t_2[BC], t_4[D] = t_1[D]$

Definiție echivalentă

Relația r peste U satisface dependența multivaluată $X \twoheadrightarrow Y$, dacă pentru orice $t_1, t_2 \in r$ cu $t_1[X] = t_2[X]$ avem că

$$M_Y(t_1[XZ]) = M_Y(t_2[XZ])$$

unde $M_Y(t[XZ]) = \{t'[Y] | t' \in r, t'[XZ] = t[XZ]\}$ = valorile lui Y din diferite tuple in care XZ sunt egale (cu XZ -ul din parametru).

	A	B	C	D	
	a_1	b_1	c_1	d_1	$= t_1$
	a_1	b_2	c_2	d_2	$= t_2$
$r :$	a_1	b_1	c_1	d_2	
	a_1	b_2	c_2	d_1	
	a_2	b_3	c_1	d_1	
	a_2	b_3	c_1	d_2	
	$M_Y(t_1[AD]) = M_Y(t_2[AD]) = \{(b_1, c_1), (b_2, c_2)\}$				

Observații

- ▶ Dacă r satisface dependența funcțională $X \rightarrow Y$, atunci pentru orice $t \in r$, avem $M_Y(t[XZ]) = \{t[Y]\}$.
- ▶ Dacă r satisface dependența funcțională $X \rightarrow Y$, atunci r satisface și dependența multivaluată $X \twoheadrightarrow Y$.
- ▶ Dacă r satisface dependența multivaluată $X \twoheadrightarrow Y$, atunci putem defini o funcție $\psi : r[X] \rightarrow \mathcal{P}(r[Y])$, prin $\psi(t[X]) = M_Y(t[XZ]), \forall t \in r$ (returnează valorile diferite din proiecția pe Y). Când r satisface $X \rightarrow Y$, atunci $\psi : r[X] \rightarrow r[Y]$ (deoarece valorile pe Y nu sunt diferite în cadrul dependenței funcționale).

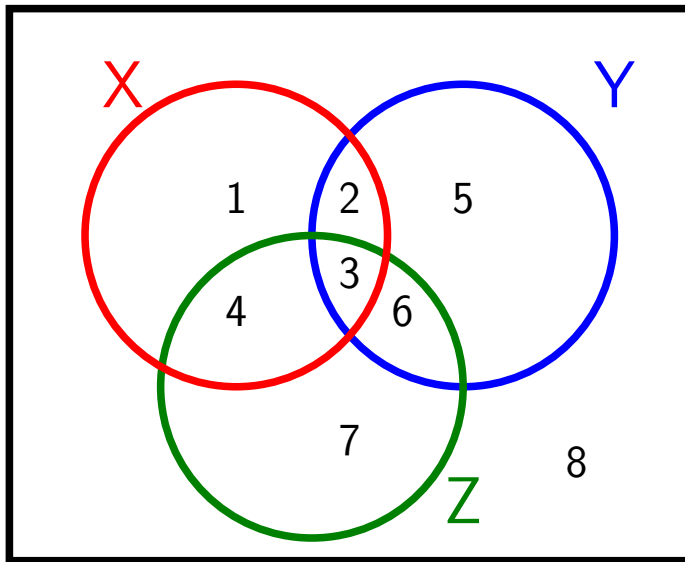
Proprietăți ale dependențelor multivaluate

MVD0 (**Complementarizare**) Fie $X, Y, Z \subseteq U$, astfel încât $XYZ = U$ și $Y \cap Z \subseteq X$. Dacă r satisface $X \twoheadrightarrow Y$, atunci r satisface $X \twoheadrightarrow Z$.

MVD1 (**Reflexivitate**) Dacă $Y \subseteq X$, atunci orice relație r satisface $X \twoheadrightarrow Y$.

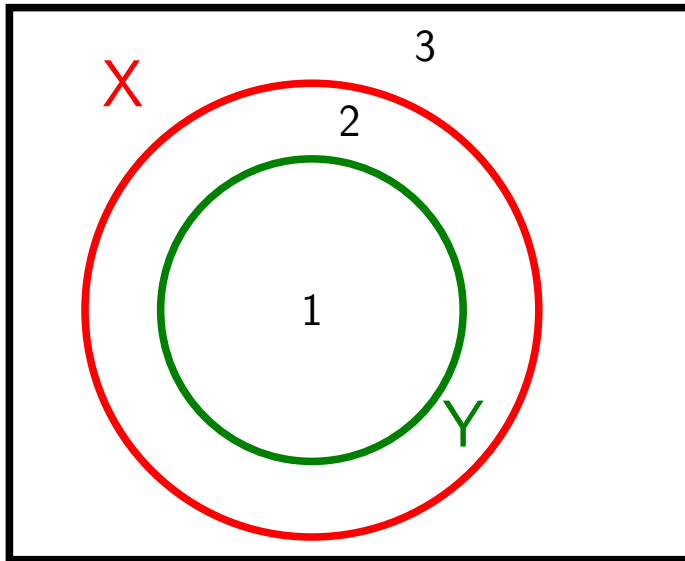
MVD2 (**Extensie**) Fie $Z \subseteq W$ și r satisface $X \twoheadrightarrow Y$. Atunci r satisface $XW \twoheadrightarrow YZ$.

MVD3 (**Tranzitivitate**) Dacă r satisface $X \twoheadrightarrow Y$ și $Y \twoheadrightarrow Z$, atunci r satisface $X \twoheadrightarrow Z$.



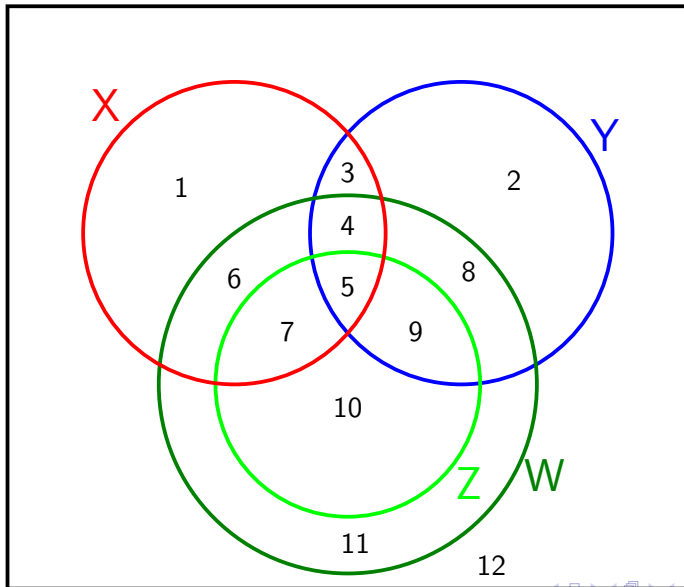
MVD0

U



MVD1

MVD2



U

Proprietăți ale dependențelor multivaluate

MVD4 (**Pseudotranzitivitate**) Dacă r satisface $X \twoheadrightarrow Y$ și $YW \twoheadrightarrow Z$, atunci r satisface și $XW \twoheadrightarrow Z - YW$.

MVD5 (**Uniune**) Dacă r satisface $X \twoheadrightarrow Y$ și $X \twoheadrightarrow Z$ atunci r satisface $X \twoheadrightarrow YZ$.

MVD6 (**Descompunere**) Dacă r satisface $X \twoheadrightarrow Y$ și $X \twoheadrightarrow Z$, atunci r satisface $X \twoheadrightarrow Y \cap Z$, $X \twoheadrightarrow Y - Z$, $X \twoheadrightarrow Z - Y$

Proprietăți mixte ale dependențelor multivaluate

FD-MVD1. Dacă r satisface $X \rightarrow Y$, atunci r satisface și $X \twoheadrightarrow Y$.

FD-MVD2. Dacă r satisface $X \twoheadrightarrow Z$ și $Y \rightarrow Z'$, cu $Z' \subseteq Z$ și $Y \cap Z = \emptyset$, atunci r satisface $X \rightarrow Z'$.

FD-MVD3. Dacă r satisface $X \twoheadrightarrow Y$ și $XY \twoheadrightarrow Z$, atunci r satisface $X \rightarrow Z - Y$.

Reguli de inferență

$$\text{MVD0f: } \frac{XYZ=U, Y \cap Z \subseteq X, X \twoheadrightarrow Y}{X \twoheadrightarrow Z}$$

$$\text{MVD1f: } \frac{Y \subseteq X}{X \twoheadrightarrow Y}$$

$$\text{MVD2f: } \frac{Z \subseteq W, X \twoheadrightarrow Y}{XW \twoheadrightarrow YZ}$$

$$\text{MVD3f: } \frac{X \twoheadrightarrow Y, Y \twoheadrightarrow Z}{X \twoheadrightarrow Z - Y}$$

$$\text{MVD4f: } \frac{X \twoheadrightarrow Y, YW \twoheadrightarrow Z}{XW \twoheadrightarrow Z - YW}$$

Reguli de inferență

$$\text{MVD5f: } \frac{X \twoheadrightarrow Y, X \twoheadrightarrow Z}{X \twoheadrightarrow YZ}$$

$$\text{MVD6f: } \frac{X \twoheadrightarrow Y, X \twoheadrightarrow Z}{X \twoheadrightarrow Y \cap Z, X \twoheadrightarrow Y - Z, X \twoheadrightarrow Z - Y}$$

$$\text{FD-MVD1f: } \frac{X \rightarrow Y}{X \twoheadrightarrow Y}$$

$$\text{FD-MVD2f: } \frac{X \rightarrow Z, Y \rightarrow Z', Z' \subseteq Z, Y \cap Z = \emptyset}{X \twoheadrightarrow Z'}$$

$$\text{FD-MVD3f: } \frac{X \rightarrow Y, XY \rightarrow Z}{X \twoheadrightarrow Z - Y}$$

Propoziție

Fie $\mathcal{R}$ o multime de reguli valide si γ o regula $\frac{\alpha_1, \alpha_2, \dots, \alpha_k}{\beta}$, astfel incat $\{\alpha_1, \dots, \alpha_k\} \vdash_{\mathcal{R}} \beta$, atunci si regula γ este valida.

Propoziție

Fie $\mathcal{R}_{FM} = \{FD1f - FD3f^1, MVD0f - MVD3f, FD - MVD1f - FD - MVD3f\}$. Avem:

- ▶ $FD - MVD3f$ se exprima cu celelalte regulid din $\mathcal{R}_{FM}$ si FD
- ▶ $MVD2f$ se exprima prin celelalte reguli din $\mathcal{R}_{FM}$.

Propoziție

Regulile $MVD4f - MVD6f$ se exprima cu ajutorul regulilor $MVD0f - MVD3f$

¹cele de la dependente functionale

Theorem

Fie Σ o multime de dependente functionale sau multivaluate si X o submultime de attribute. Atunci exista o partitie a lui $U - X$ notata prin $Y_1 \dots Y_k$, astfel incat pentru $Z \subseteq U - X$ avem $\Sigma \vdash_{\mathcal{R}_{FM}} X \twoheadrightarrow Z$ daca si numai daca Z este reuniunea unui numar de multimi din partitia $\{Y_1, \dots Y_k\}$

Definition

Pentru Σ o multime de dependente functionale sau multivaluate si X o submultime de attribute, numim **baza de dependenta pentru X cu privire la Σ** partitia $B(\Sigma, X) = \{\{A_1\} \dots \{A_h\}, Y_1 \dots Y_k\}$, unde $X = A_1, \dots A_h$, iar $Y_1, \dots Y_k$ este partitia construita in teorema precedenta.

Observatii

- ▶ Avem $\Sigma \vdash_{\mathcal{R}_{FM}} X \twoheadrightarrow Z$ dacă și numai dacă Z este o reuniune de elemente din partiția $B(\Sigma, X)$.
- ▶ Fie $X_{\Sigma}^* = \{A \mid \Sigma \vdash_{\mathcal{R}_{FM}} X \rightarrow A\}$. Atunci pentru orice $A \in X_{\Sigma}^*$ avem $\{A\} \in B(\Sigma, X)$.



BAZE DE DATE

Proiectarea bazelor de date relaționale

Normalizare

Mihaela Elena Breabăn

© FII 2015-2016

Normalizare

- ▶ Dependențe funcționale (revizitat)
- ▶ 1NF, 2NF, 3NF
- ▶ Forma normală Boyce-Codd (BCNF)
- ▶ Dependențe multivaluate (revizitat)
- ▶ Forma normală 4 (4NF)
- ▶ Denormalizare

Proiectarea schemei

- ▶ De obicei mai multe variante de proiectare
 - ▶ Unele sunt (mult) mai bune decât altele
 - ▶ Cum alegem?
-
- ▶ Teorie pentru proiectarea bazelor de date relaționale cu fundamente în algebra relațională, introdusă de Codd între '70-'74
 - ▶ Eliminarea anomaliilor la modificări în date
 - ▶ Minimizarea necesității reproiectării când sunt necesare extensii ale structurii
 - ▶ Evitarea avantajării anumitor interogări

Exemplu

Schemă cu anomalii

- ▶ Informații cu privire la candidatii la admitere
 - ▶ CNP și nume
 - ▶ Universitatea la care s-a candidat
 - ▶ Liceele de la care provin candidații (și orașele)
 - ▶ Hobby-urile candidaților

Candidati(CNP, aNume, uNume, lIceu, lOraș, hobby)

Ioana cu CNP-ul **2810605222111** a studiat la **Negruzzi** în **Iași**, candidează la **Cuza, Asachi** și la **Babes-Bolyai**, îi place să joace **tenis** și să cânte la **chitară**

Câte tuple sunt necesare a fi inserate în relația **Candidati** pentru a păstra toate informațiile despre **Ioana**?

Anomalii de proiectare

- ▶ Redundanță
- ▶ Anomalii de actualizare
- ▶ Anomalii la ștergere

Exemplu

Schemă fără anomalii

- ▶ Informații cu privire la aplicațiile de admitere
 - ▶ CNP și nume
 - ▶ Universitatea la care s-a candidat
 - ▶ Liceele de la care provin candidații (și orașele)
 - ▶ Hobby-urile candidaților

`Abso1vent(CNP, aNume)`

`Candidat(CNP, uNume)`

`Liceu(CNP, codLiceu)`

`LocatieLiceu(codLiceu, lNume, lOraș)`

`Hobbies(CNP, hobby)`

???

-
- ▶ Informații cu privire la cursurile luate de studenți
 - ▶ Studenții au id-uri unice și nume (nu sunt unice)
 - ▶ Cursurile au număr de identificare unic și titlu (nu unic)
 - ▶ Studenții iau un curs într-un anumit an și primesc o notă
 - ▶ Care e schema recomandată?
 - ▶ Studiază(sID, nume, cID, titlu, an, notă)
 - ▶ Curs(cID, titlu, an), Studiază(sID, cID, notă)
 - ▶ Student(sID, nume), Curs(cID, titlu), Studiază(sID, cID, an, notă)
 - ▶ Student(sID, nume), Curs(cID, titlu), Studiază(nume, titlu, an, notă)

Proiectarea prin descompunere

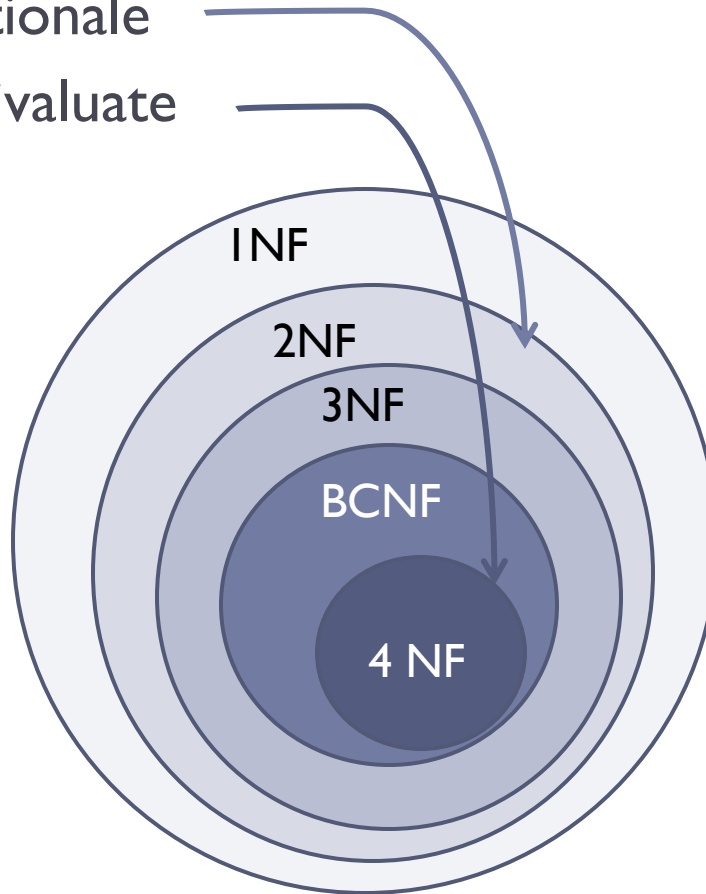
- ▶ Se pleacă de la “mega-relații” ce conțin tot
- ▶ Se descompune în relații mai mici ce păstrează toate informațiile
- ▶ Se poate realiza automat
 - ▶ Mega-relații + *proprietăți ale datelor*
 - ▶ Descompunerea se realizează pe baza proprietăților
 - ▶ Setul final de relații satisface anumite *forme normale*
 - ▶ **Fără anomalii**
 - ▶ **Fără pierdere de informații**

Proprietăți și forme normale

► Proprietăți

- Dependențe funcționale
- Dependențe multivaluate

► Forme normale



Dependențe funcționale

- ▶ Concepte folosite pentru
 - ▶ Stocarea datelor – compresie
 - ▶ Optimizarea interogărilor

$X \rightarrow Y$ dacă

$$\forall t_1, t_2 \in r, t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$$

r – relație peste mulțimea de atribute U

X, Y – submulțimi ale lui U

De ce funcțional?

Exemplu

Dependențe funcționale

Absolvent(CNP, aNume, adresa,
lCod, lNume, lOras, medie,
prioritate)

Candidat(CNP, uNume, uOras, data,
specializare)

- ▶ Valorile atributului **prioritate** sunt determinate de valorile atributului **medie**

medie → *prioritate*

Care sunt dependențele funcționale pentru relația Absolvent?

Dar pentru relația Candidat?

Ce constrângere este impusă de {CNP,uNume → data}

???

-
- ▶ $R(A,B,C,D,E,F)$
 - ▶ $ABC \rightarrow D$
 - ▶ $DE \rightarrow F$
 - ▶ Fiecare din attributele A,B,C,E are cel mult 3 valori diferite.
 - ▶ Care este numărul maxim de valori diferite pe care îl poate lua E ? (3,9,27,81?)

Dependențe funcționale

Reguli de inferență

- ▶ Reflexivitatea (FD1)
 - ▶ (A1)
 - ▶ dependențe triviale
- ▶ Descompunerea (FD6)
 - ▶ A(21)
 - ▶ Se poate descompune și membrul stâng?
- ▶ Uniunea (FD5)
 - ▶ (A22)
- ▶ Tranzitivitatea (FD3)
 - ▶ (A3)
- ▶ Teorema de completitudine
 - ▶ o dependență funcțională este consecință a unei mulțimi de dependențe funcționale d.d. are demonstrație utilizând regulile de mai sus (Axiomele lui Armstrong)

Dependențe funcționale și chei

▶ Dependențe funcționale (d.f.)

- ▶ Valorile unei submulțimi de attribute determină (identifică) valorile unei alte submulțimi de attribute
 - ▶ Formulate pe baza cunoașterii lumii reale
 - ▶ Toate instanțele relației trebuie să le satisfacă
 - ▶ *Se specifică un set minimal netrivial al.î. toate dependențele satisfăcute de relație se obțin ca și consecințe ale acestei mulțimi*

▶ Supercheie

- ▶ Valorile unei submulțimi de attribute identifică în mod unic un tuplu => valorile tuturor atributelor
- ▶ O cheie este submulțime minimală cu proprietatea de mai sus
- ▶ Garantează o relație fără duplicate

Dependențele funcționale sunt o generalizare a noțiunii de cheie

Închideri

- ▶ *Închiderea unei mulțimi de d.f. Σ notată Σ^+*
 - ▶ Mulțimea d.f. Σ împreună cu toate d.f. consecințe din Σ
- ▶ *Închiderea unei mulțimi de attribute X notată X^+ relativ la un set de d.f. Σ*
 - ▶ Mulțimea tuturor atributelor B pentru care există $X \rightarrow B \in \Sigma^+$
- ▶ O dependență funcțională $X \rightarrow B$ este consecință a unei mulțimi de dependențe funcționale d.d. $B \in X^+$

Algoritm de calcul a închiderii lui X ?

Închideri și chei

- ▶ Date schema de relație $R(U)$ și un set de d.f. Σ satisfăcute de R , submulțimea de attribute X este cheie pentru R d.d.
 $X^+ = U$ și $\forall X' \subset X, X'^+ \neq U$

- ▶ Exemplu

`Absolvent(CNP, aNume, adresa,
 lCod, lNume, lOras, medie, prioritate)`

`CNP → aNume, adresa, medie`

`medie → prioritate`

`lCod → lNume, lOras`

`Perechea {CNP, lCod} este cheie`

Data o mulțime de d.f. cum putem determina toate cheile?

???

-
- ▶ $R(A,B,C,D,E)$
 - ▶ $AB \rightarrow C$
 - ▶ $AE \rightarrow D$
 - ▶ $D \rightarrow B$
 - ▶ Care sunt cheile pt. R?

Attribute (ne)prime

- ▶ **Atribut prim**

- ▶ Există o cheie care să-l conțină

- ▶ **Atribut neprim**

- ▶ Nu aparține nici unei chei

- ▶ **Exemplu**

- ▶ $R(A, B, C, D)$

- ▶ $F = \{AB \rightarrow C, B \rightarrow D, BC \rightarrow A\}.$

- ▶ *AB și BC sunt singurele chei cu privire la F, deci A, B, C sunt attribute prime*

- ▶ *D este atribut neprim.*

Dependențe pline

- ▶ Fie dată o schemă de relație R cu multimea de atribute U și F o multime de dependente functionale. O dependenta functională $X \rightarrow B \in F^+$ ($X \subset U$, $B \in U$, $B \notin X$) se numeste o dependenta plina a lui R (sau B este dependent plin de X sub F), daca nu exista nici o submultime proprie $X' \subset X$, astfel incat $X' \rightarrow B \in F^+$.
- ▶ Exemplu
 - ▶ $R(A, B, C, D)$
 - ▶ $F = \{AB \rightarrow C, B \rightarrow D, BC \rightarrow A\}$.
 - ▶ *Toate dependențele din F sunt pline.*
 - ▶ *$AB \rightarrow D \in F^+$ nu este dependență plină*

Atribut tranzitiv dependent

- ▶ Fie R o schema cu multimea de attribute U si F o multime de dependente functionale. Un atribut B din U se numeste *tranzitiv dependent* de X ($X \subset U$, $B \notin X$), daca exista $Y \subset U$, astfel incat:
 - ▶ $B \in U - Y$,
 - ▶ $X \rightarrow Y \in F^+$,
 - ▶ $Y \rightarrow B \in F^+$,
 - ▶ $Y \rightarrow X \notin F^+$.

1NF

- ▶ O schemă de relație este în 1NF dacă domeniile de valori ale tuturor atributelor sunt elementare (indivizibile) deci diferite de multimi, de tuple de valori dintr-un anumit domeniu. În general numim valoare elementară o valoare pentru care în aplicații nu se utilizează părți ale ei

2NF

- ▶ O schema de relatie R situata in 1NF, impreuna cu o multime de dependente functionale F este in a doua forma normala daca orice atribut neprim din R este dependent plin de orice cheie a lui R .
- ▶ Obs: Orice relație ce nu are chei multivaluate este în 2NF.
- ▶ Exemplu
 - ▶ $R(A, B, C, D)$
 - ▶ $F = \{AB \rightarrow C, B \rightarrow D, BC \rightarrow A\}$.
 - ▶ AB si BC sunt singurele chei
 - ▶ D este atribut neprim
 - ▶ $B \rightarrow D \in F^+$, deci D nu este dependent plin nici de AB , nici de BC . In concluzie, aceasta schema impreuna cu F nu este in 2NF.

???

-
- ▶ Absolventi(CNP, aNume, hobby)
 - ▶ Un Absolvent, identificat prin CNP, are un singur nume însă mai multe hobbyuri
 - ▶ Absolventi nu este în 2 NF. Ce anomalii apar din nerespectarea 2NF?
-
- ▶ Olimpici(concurs,an,CNP,nume)
 - ▶ Într-un anumit an există un singur câștigător (olimpic) la un anumit concurs. Câștigătorul e identificat prin CNP și are asociat un singur nume.
 - ▶ Este Olimpici in 2NF?

3NF

- ▶ Schema de relatie R impreuna cu multimea de dependente functionale F este in forma a treia normala (notata 3NF) daca este in a doua forma normala si orice atribut neprim din R **NU** este tranzitiv dependent de nici o cheie a lui R .
- ▶ Exemplu
 - ▶ $R(O, S, C)$
 - ▶ $F = \{OS \rightarrow C, C \rightarrow O\}$
 - ▶ OS si SC sunt chei.
 - ▶ toate attributele sunt prime, deci schema este in 2NF si 3NF.

???

-
- ▶ Olimpici(concurs,an,CNP,nume)
 - ▶ Într-un anumit an există un singur câștigător (olimpic) la un anumit concurs. Câștigătorul e identificat prin CNP și are asociat un singur nume.
 - ▶ Olimpici nu este in 3NF. Ce probleme de inconsistență a datelor pot să apară?

???

-
- ▶ Candidat(CNP,uNume,data,specializare)
 - ▶ Un absolvent identificat prin CNP poate candida la orice universitate (identificata prin uNume) la o singură specializare din cadrul acesteia, deci o singura data
 - ▶ Universitățile au date de aplicație care nu se suprapun
 - ▶ Este Candidat în 3NF relativ la regulile specificate mai sus?

BCNF

- ▶ O schemă de relație R împreună cu o mulțime de dependențe funcționale F este în BCNF dacă este în 1NF și pentru orice dependență funcțională netrivială $X \rightarrow A \in F^+$ X este (super)cheie pentru R
- ▶ Orice schemă de relație în BCNF este în 3NF
 - ▶ **Demonstrație?**
- ▶ Proiectarea unei scheme de BD în BCNF are la bază descompunerea:
 - ▶ Intrare: o mega-relație împreună cu un set de dependențe funcționale
 - ▶ Ieșire: un set de relații în BCNF care în urma reasamblării produc informațiile originale

???

-
- ▶ Candidat(CNP,uNume,data,specializare)
 - ▶ Un absolvent poate aplica la o universitate la o singură specializare
 - ▶ Universitățile au date de aplicație care nu se suprapun
 - ▶ Este Candidat în BCNF?

Descompunerea schemelor de relatie

- ▶ Fie schema de relatie $R[A_1, A_2, \dots, A_n]$.
- ▶ $\rho = \{R_1, \dots, R_k\}$, unde $R_i[A_{i1}, \dots, A_{ihi}]$ este o *descompunere* a lui R dacă
$$\bigcup_{i=1}^k \bigcup_{j=1}^{h_i} A_{ij} = \{A_1, \dots, A_n\}$$
- ▶ ρ este o *descompunere de tip join fără pierdere* a lui R *cu privire la o mulțime de d.f.* F , dacă pentru orice relație r peste R ce satisface F , avem $r = r[R_1] \bowtie \dots \bowtie r[R_k]$ – deci r se obține în urma joinului natural peste descompunerea ρ .

Exemplu

Descompunere

- ▶ Absolvent(CNP,aNume,adresa,ICod,INume,IOras,medie,prioritate)
- ▶ $\rho_1 = \{S_1(\text{CNP}, \text{aNume}, \text{adresa}, \underline{\text{ICod}}, \text{medie}, \text{prioritate}), S_2(\underline{\text{ICod}}, \text{INume}, \text{IOras})\}$
- ▶ $\rho_2 = \{S_1(\text{CNP}, \underline{\text{aNume}}, \text{adresa}, \text{ICod}, \underline{\text{INume}}, \text{IOras}), S_2(\underline{\text{aNume}}, \underline{\text{INume}}, \text{medie}, \text{prioritate})\}$
- ▶ ρ_1 - de tip join fără pierdere
- ▶ ρ_2 - NU e de tip join fără pierdere

Descompuneri de tip join fără pierdere

► Teoremă

- Dacă $\rho = (R_1, R_2)$ este o descompunere a lui R și F este o multime de d.f., atunci ρ este o descompunere join fără pierdere cu privire la F d.d. $R_1 \cap R_2 \rightarrow R_1 - R_2 \in F^+$ sau $R_1 \cap R_2 \rightarrow R_2 - R_1 \in F^+$.

(prin intersectia schemelor se intelege intersectia multimilor de atribute)

► Exemplu

- $R(A, B, C)$
- $F = \{A \rightarrow B\}$.
- $\rho_1 = (R_1(A, B), R_2(A, C))$
- $AB \cap AC = A, AB - AC = B, A \rightarrow B \in F^+$
- ρ_1 este de tip join fără pierdere

- $\rho_2 = (R_1(A, B), R_2(B, C))$.
- $AB \cap BC = B, AB - BC = A, B \rightarrow A \notin F^+$,
- $AB \cap BC = B, BC - AB = C, B \rightarrow C \notin F^+$,
- ρ_2 nu este de tip join fără pierdere cu privire la F .

Descompunere de tip join fara pierdere in BCNF

▶ Intrare:

- ▶ Schema de relatie R cu dependentele functionale F.

▶ Iesire:

- ▶ Descompunerea $\rho = (R_1, \dots, R_k)$, astfel incat ρ este de tip join fara pierdere cu privire la F si (R_i, F_i) este in BCNF $\forall i = 1, k$.

▶ Pasul 1.

- ▶ $\rho = R = R_1$
- ▶ Calculăm F^+ și cheile necesare verificării formei BCNF

▶ Pasul 2.

- ▶ Fie R_i o schema de relatie din ρ , pentru care (R_i, F_i) nu este in BCNF.
- ▶ Exista $X \rightarrow A \in F_i^+, A \notin X$ si X nu include o cheie.
- ▶ Construim $S_1 = X \cup \{A\}, S_2 = R_i - A$
- ▶ Înlocuim R_i in ρ prin S_1, S_2 . $k = k + 1$.
- ▶ Calculăm $F_{S_1}^+ \cup F_{S_2}^+$ și cheile pt. S_1, S_2 necesare verificării formei BCNF

▶ Pasul 3.

- ▶ Repetăm pasul 2, pana cand obținem toate $(R_i, F_i), i = 1, k$ in BCNF.

Exemplu

Descompunere în BCNF

Abso1vent(CNP, aNume, adresa,
1Cod, 1Nume, 1Oras, medie, prioritate)

CNP → aNume, adresa, medie

medie → prioritate

1Cod → 1Nume, 1Oras

{R1(1Cod, 1Nume, 1Oras),
R2(medie, prioritate),
R3(CNP, aNume, adresa, medie),
R4(CNP, 1Cod)}

este o descompunere de tip join fără pierdere în
BCNF

- Pentru o schemă de relație R pot exista mai multe descompuneri de tip join fără pierdere în BCNF?

Garantează desc. în BCNF o schemă bună?

- ▶ Poate fi reconstruită relația originală?
- ▶ Elimină redundanța?
 - ▶ `Candidat(CNP, uNume, hobby)`
 - ▶ d.f.? NU
 - ▶ Chei? Toate attributele
 - ▶ BCNF? DA
 - ▶ Schemă bună? ...

Dependențe multivaluate

► Reguli generatoare de tuple

$X \twoheadrightarrow Y$ dacă

$\forall t_1, t_2 \in r, t_1[X] = t_2[X], \text{ exista } t_3, t_4 \in r \text{ astfel încât}$

(i) $t_3[X] = t_1[X], t_3[Y] = t_1[Y] \text{ și } t_3[Z] = t_2[Z]$

(ii) $t_4[X] = t_2[X], t_4[Y] = t_2[Y] \text{ și } t_4[Z] = t_1[Z]$

r – relație peste mulțimea de attribute U

X, Y – submulțimi ale lui U

$Z = U - XY$

► Orice d.f. este d.mv.

Exemplu

Dependențe multivaluate

- ▶ `Candidat(CNP, uNume, hobby)`

- ▶ Cerințe:

- ▶ Aceleași hobbyuri la toate univ

- ▶ **Regula corespunzătoare:**

- ▶ **$CNP \twoheadrightarrow uNume$**

▶ Exemplu extins

- ▶ `Candidat(CNP, uNume, data, specializare, hobby)`

- ▶ Cerințe:

- ▶ Hobbyurile sunt introduse selectiv în funcție de universitate

- ▶ Un Absolvent candidează într-o singură zi la o anumită universitate

- ▶ Un Absolvent poate aplica la mai multe specializări (hobbyurile declarate la o univ. trebuie să fie vizibile la specializarile de la acea univ.)

- ▶ **Regulile corespunzătoare:**

- ▶ **$CNP, uNume \rightarrow data$**

- ▶ **$CNP, uNume, data \twoheadrightarrow specializare$**

???

-
- ▶ Fie $R(A,B,C)$ și $A \rightarrow \rightarrow B$
 - ▶ A ia cel puțin 3 valori diferite iar fiecare valoare a lui A este asociată cu cel puțin 4 valori diferite pentru B și cel puțin 5 valori diferite pentru C.
 - ▶ Care este numărul minim de tuple în R?

Dependențe multivaluate

Reguli

- ▶ Dependențe triviale
 - ▶ Reflexivitate (MVDI)
 - ▶ $X \twoheadrightarrow Y$ unde $XY=U$
- ▶ Complementariere (MVD0)
- ▶ Tranzitivitatea (!=d.f.)
- ▶ Intersecția

4NF

- ▶ O schemă de relație R și o mulțime de dependențe multivaluate D este în 4NF dacă este în 1NF și pentru orice dependență multivaluată netrivială $X \twoheadrightarrow A \in D^+$ X este (super)cheie pentru R
- ▶ Orice schemă de relație în 4NF este în BCNF
- ▶ Proiectarea unei scheme de BD în 4NF are la bază descompunerea:
 - ▶ Intrare: o mega-relație împreună cu un set de dependențe funcționale și multivaluate
 - ▶ Ieșire: un set de relații în 4NF care în urma reasamblării produc informațiile originale

Descompunere de tip join fara pierdere in 4NF

▶ Intrare:

- ▶ Schema de relatie R cu dependentele functionale F și dependențele multivaluate MV

▶ Iesire:

- ▶ Descompunerea lui $\rho = (R_1, \dots, R_k)$, astfel incat ρ este de tip join fara pierdere cu privire la F si (R_i, F_i, MV_i) este in 4NF $\forall i = 1, k$.

▶ Pasul 1.

- ▶ $\rho = R = R_1$
- ▶ Calculăm $M = \{F^+, MV^+\}$ și cheile necesare verificării formei 4NF

▶ Pasul 2.

- ▶ Fie R_i o schema de relatie din ρ , pentru care (R_i, F_i, MV_i) nu este in 4NF.
- ▶ Exista $X \twoheadrightarrow A \in M$ netrivială si X nu include o cheie.
- ▶ Construim $S_1 = X \cup \{A\}$, $S_2 = R_i - A$
- ▶ Înlocuim R_i in ρ prin S_1, S_2 . $k = k + 1$.
- ▶ Calculăm d.mv și cheile pt. S_1, S_2 necesare verificării formei 4NF

▶ Pasul 3.

- ▶ Repetam pasul 2, pana cand obtinem toate $(R_i, F_i), i = 1, k$ in NF.

Exemplu

Descompunere în 4NF

- ▶ $\text{Candidat}(\text{CNP}, \text{uNume}, \text{hobby})$
- ▶ $\text{CNP} \twoheadrightarrow \text{uNume}$
- ▶ $\rho = \{A_1(\text{CNP}, \text{uNume}), A_2(\text{CNP}, \text{hobby})\}$ este descompunere în 4NF de tip join fără pierdere
- ▶ $u * h, u + h$

▶ Exemplu extins

- ▶ $\text{Candidat}(\text{CNP}, \text{uNume}, \text{data}, \text{specializare}, \text{hobby})$
- ▶ $\text{CNP}, \text{uNume} \rightarrow \text{data}$
- ▶ $\text{CNP}, \text{uNume}, \text{data} \twoheadrightarrow \text{specializare}$
- ▶ $\rho = \{A_1(\text{CNP}, \text{uNume}, \text{data}), A_2(\text{CNP}, \text{uNume}, \text{specializare}), A_3(\text{CNP}, \text{uNume}, \text{hobby})\}$
este în 4NF de tip join fără pierdere

Neajunsuri ale normalizării

Exemplu 1

- ▶ $\text{Candidat}(\text{CNP}, \text{uNume}, \text{data}, \text{specializare})$
- ▶ $\text{CNP}, \text{uNume} \rightarrow \text{data}, \text{specializare}$
- ▶ $\text{data} \rightarrow \text{uNume}$

- ▶ $\{\text{A1}(\text{data}, \text{uNume}), \text{A2}(\text{CNP}, \text{data}, \text{specializare})\}$ în 4NF este o schemă mai bună?

Neajunsuri ale normalizării

Exemplu 2

- ▶ $\text{Absolvent}(\text{CNP}, \text{INume}, \text{medie}, \text{prioritate})$
- ▶ $\text{CNP} \rightarrow \text{medie}$
- ▶ $\text{medie} \rightarrow \text{prioritate}$
- ▶ $\text{CNP} \rightarrow \text{prioritate}$

- ▶ $\{S1(\text{CNP}, \text{prioritate}), S2(\text{CNP}, \text{medie}), S3(\text{CNP}, \text{INume})\}$ în 4NF este o schemă bună?

Neajunsuri ale normalizării

- ▶ Supra-descompunere
- ▶ Interogări supra-încărcate

- ▶ Ca soluție se poate aplica denormalizarea

Bibliografie

- ▶ Victor Felea: *Baze de date relationale. Dependente*. Editura Universitatii “Al.I.Cuza” Iasi, 1996
- ▶ Hector Garcia-Molina, Jeff Ullman, **Jennifer Widom**: *Database Systems: The Complete Book*, Prentice Hall; 2nd edition (June 15, 2008)
- ▶ Unele exemple sunt **adaptate** dupa cursul public de Baze de date de la Stanford (autor Jennifer Widom)



BAZE DE DATE

Implementarea constrângerilor
Declanșatoare (Triggers)
Vederi (Views)

Mihaela Elena Breabăn

© FII 2015-2016

Conținut

- ▶ Constrângeri de integritate
- ▶ Declanșatoare
- ▶ Views

Constrângeri de integritate (statice)

(1)

- ▶ **Restricționează stările posibile ale bazei de date**
 - ▶ Pentru a elimina posibilitatea introducerii eronate de valori la operația de inserare
 - ▶ Pentru a satisface corectitudinea la actualizare/ștergere
 - ▶ Forțează consistența
 - ▶ Transmit sistemului informații utile stocării, procesării interogărilor
- ▶ **Tipuri**
 - ▶ Non-null
 - ▶ Chei
 - ▶ Integritate referențială
 - ▶ Bazate pe atribut și bazate pe tuplu
 - ▶ Aserțiuni generale

Constrângeri de integritate (2)

▶ Declaraire

- ▶ Odată cu schema (comanda CREATE)
- ▶ După crearea schemei (comanda ALTER)

▶ Realizare

- ▶ Verificare la fiecare comandă de modificare a datelor
- ▶ Verificare la final de tranzacție

Constrângeri de integritate peste 1 variabilă

Implementare *inline*

CREATE TABLE *tabel* (

a1 tip **not null**, -- acceptă doar valori nenule

a2 tip **unique**, --cheie candidat formată dintr-un singur atribut

a3 tip **primary key**, -- cheie primară formată dintr-un singur atribut, implicit {not null, unique}

a4 tip **references** *tabel2* (*b1*), --cheie străină formată dintr-un singur atribut

a5 tip **check** (*condiție*) -- condiția e o expresie booleană formulată peste atributul *a5*: (*a5*<11 and *a5*>4), (*a5* between 5 and 10), (*a5* in (5,6,7,8,9,10))...

)

Constrângeri de integritate peste n variabile

Implementare *out-of-line*

CREATE TABLE *tabel* (

a1 tip,

a2 tip,

a3 tip,

a4 tip,

primary key (*a1,a2*), --cheie primară formată din 2 (sau mai multe)
atribute

unique(*a2,a3*), -- cheie candidat formată din 2 (sau mai multe) atribute

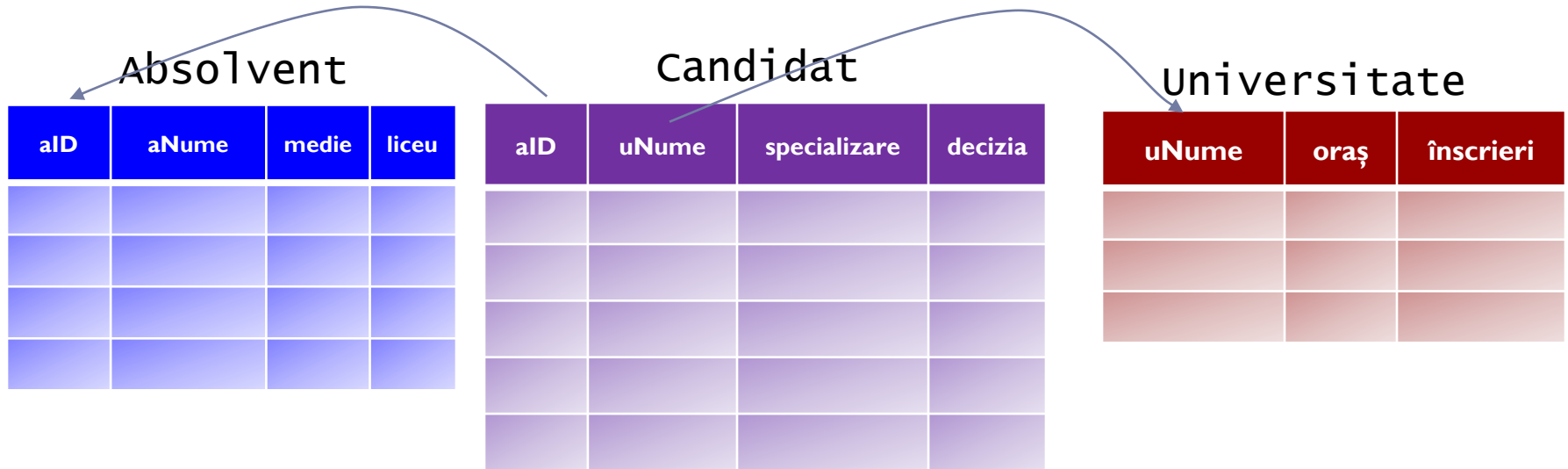
check (*condiție*), -- expresie booleană peste variabile declarate
anterior: $((a1+a3)/2 \geq 5)$

foreign key (*a3,a4*) **references** *tabel2*(*b1,b2*) -- cheie străină multi-
atribut

)

Integritate referențială

Definiții



- ▶ *Integritate referențială de la R.A la S.B:*
 - ▶ fiecare valoare din coloana A a tabelului R trebuie să apară în coloana B a tabelului S
 - ▶ A se numește cheie străină
 - ▶ B trebuie să fie cheie primară pentru S sau măcar declarat unic
 - ▶ sunt permise chei străine multi-atribut

Integritate referențială

Realizare

- ▶ Comenzi ce pot genera încălcarea restricțiilor:
 - ▶ inserări în R
 - ▶ ștergeri în S
 - ▶ actualizări pe R.A sau S.B
- ▶ Acțiuni speciale:
 - ▶ la ștergere din S:
ON DELETE RESTRICT (implicit) | **SET NULL** | **CASCADE**
 - ▶ la actualizări pe S.B:
ON UPDATE RESTRICT (implicit) | **SET NULL** | **CASCADE**

Integritate referențială

oul sau găina?

```
CREATE TABLE chicken (cID INT PRIMARY KEY,  
                        eID INT REFERENCES egg(eID));  
CREATE TABLE egg(eID INT PRIMARY KEY,  
                  cID INT REFERENCES chicken(cID));
```

```
CREATE TABLE chicken(cID INT PRIMARY KEY, eID INT);  
CREATE TABLE egg(eID INT PRIMARY KEY, cID INT);
```

```
ALTER TABLE chicken ADD CONSTRAINT chickenREFegg  
    FOREIGN KEY (eID) REFERENCES egg(eID)  
    DEFERRABLE INITIALLY DEFERRED; -- Oracle  
ALTER TABLE egg ADD CONSTRAINT eggREFchicken  
    FOREIGN KEY (cID) REFERENCES chicken(cID)  
    DEFERRABLE INITIALLY DEFERRED; -- Oracle
```

```
INSERT INTO chicken VALUES(1, 2);  
INSERT INTO egg VALUES(2, 1);  
COMMIT;
```

Cum rezolvați problema inserării dacă
verificarea constrângerii se efectuează
imediat după fiecare inserare?
Dar problema ștergerii tabelelor?

Aserțiuni

create assertion Key

check ((select count(distinct A) from T) =
(select count(*) from T));

create assertion ReferentialIntegrity

check (not exists (select * from Candidat
where aID not in (select aID from Student)));

Constrângeri de integritate

Abateri de la standardul SQL

- ▶ Postgres, SQLite, Oracle, MySQL(innodb) implementează și validează toate constrângerile anterioare
- ▶ Standardul SQL permite utilizarea de interogări în clauza check însă nici un SGBD nu le suportă
- ▶ Nici un SGBD nu a implementat aserțiunile din standardul SQL, funcționalitatea lor fiind furnizată de declanșatoare

...DEMO...
(fișierul *constrângeri.sql*)

Declanșatoare (constrangeri dinamice)

- ▶ Monitorizează schimbările în baza de date, verifică anumite condiții și inițiază acțiuni
- ▶ Reguli eveniment-condiție-acțiune
 - ▶ Introduc elemente din logica aplicației în SGBD
 - ▶ Forțează constrângeri care nu pot fi exprimate altfel
 - ▶ Sunt expresive
 - ▶ Pot întreprinde acțiuni de reparare
 - ▶ implementarea variază în funcție de SGBD, exemplele de aici urmăresc standardul SQL

Declanșatoare

Implementare

Create Trigger *nume*
Before|After|Instead Of *evenimente*
[*variabile-referențiate* **]**
[For Each Row] -- acțiune se execută pt fiecare linie modificată (tip row vs. statement)
[When (condiție)] -- ca o condiție WHERE din SQL
acțiune -- în standardul SQL e o comandă SQL, în SGBD-uri poate fi bloc procedural

- ▶ *evenimente*:
 - ▶ **INSERT ON** *tabel*
 - ▶ **DELETE ON** *tabel*
 - ▶ **UPDATE [OF a1,a2,...] ON** *tabel*
- ▶ *variabile-referențiate* (după declarare pot fi utilizate în *condiție* și *acțiune*):
 - ▶ **OLD TABLE AS** *var*
 - ▶ **NEW TABLE AS** *var*
 - ▶ **OLD ROW AS** *var* — *pentru ev. DELETE, UPDATE*
 - ▶ **NEW ROW AS** *var* — *pentru ev. INSERT, UPDATE*

} doar pentru triggere de
tip row

Declanșatoare

Exemplu (1)

- ▶ integritate referențială de la R.A la S.B cu ștergere în cascadă

Create Trigger Cascade
After Delete On S
Referencing Old Row As O
For Each Row
~~[fără condiții]~~
Delete From R Where A = O.B

Create Trigger Cascade
After Delete On S
Referencing Old Table As OT
~~[For Each Row]~~
~~[fără condiții]~~
Delete From R Where
A in (select B from OT)

Declanșatoare

Capcane

- ▶ mai multe declanșatoare activate în același timp: care se execută primul?
- ▶ acțiunea declanșatorului activează alte declanșatoare: înlănțuire sau auto-declanșare ce poate duce la ciclare

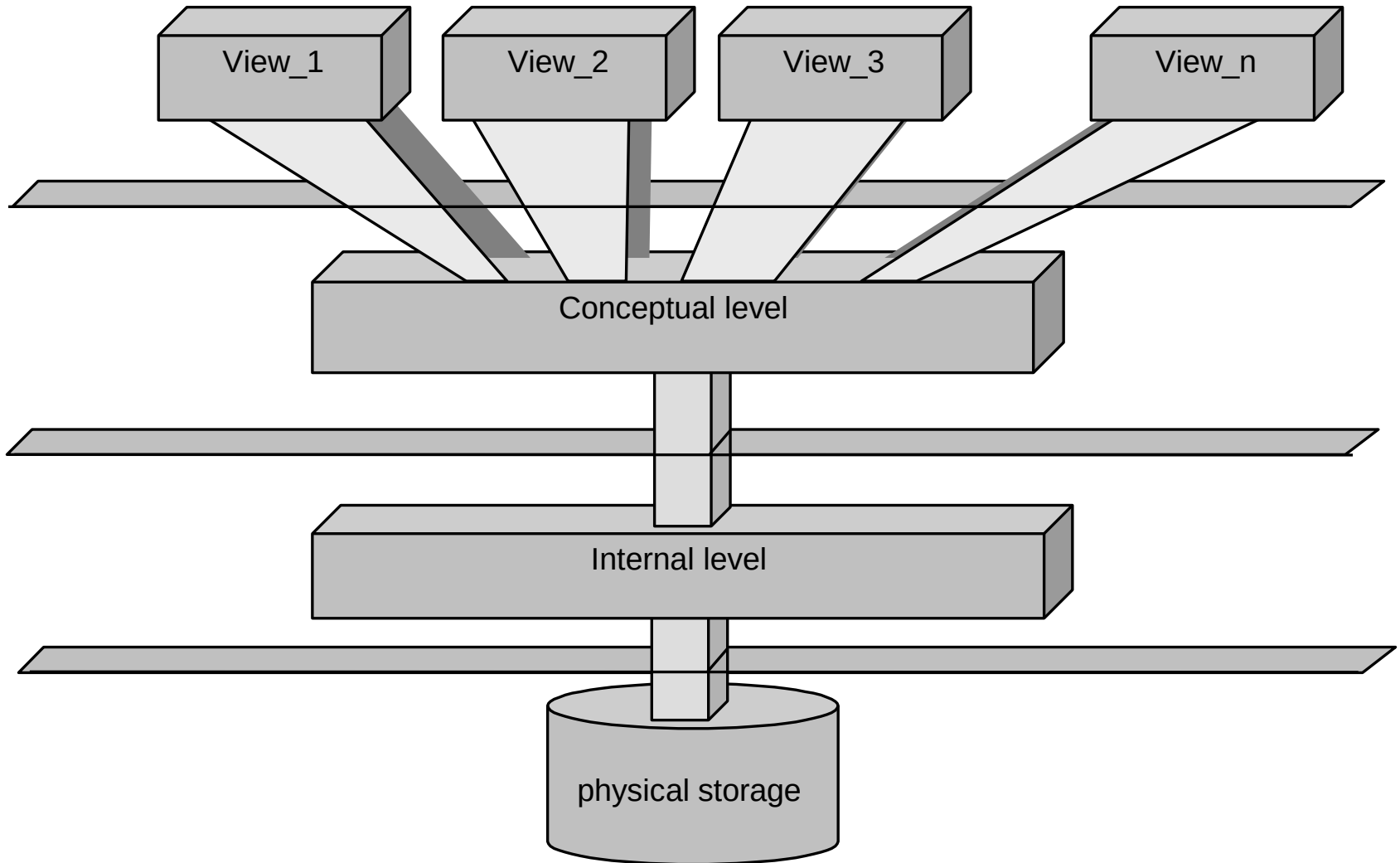
Declanșatoare

Abateri de la standardul SQL

- ▶ **Postgres**
 - ▶ cel mai apropiat de standard
 - ▶ implementează row+statement, old/new+row/table
 - ▶ sintaxa suferă abateri de la standard
- ▶ **SQLite**
 - ▶ doar tip row (fără old/new table)
 - ▶ se execută imediat, după modificarea fiecărei linii (abatere comportamentală de la standard)
- ▶ **MySQL**
 - ▶ doar tip row (fără old/new table)
 - ▶ se execută imediat, după modificarea fiecărei linii (abatere comportamentală de la standard)
 - ▶ permite definirea unui singur declanșator / eveniment asociat unui tabel
- ▶ **Oracle**
 - ▶ implementează standardul: row+statement cu modificări ușoare de sintaxă
 - ▶ tipul instead-of e permis numai pt. view-uri
 - ▶ permite inserarea de blocuri procedurale
 - ▶ introduce restricții pentru a evita ciclarea
 - ▶ **aprofundate la laborator**

...DEMO...
(fișierul *declansatoare.sql*)

View-uri - Vederi



Motivație

- ▶ acces modular la baza de date
- ▶ ascunderea unor date față de unii utilizatori
- ▶ ușurarea formulării unor interogări

- ▶ aplicațiile reale tind să utilizeze foarte multe view-uri

Definire și utilizare

- ▶ Un view este în esență o interogare stocată formulată peste tabele sau alte view-uri
- ▶ Schema view-ului este cea a rezultatului interogării
- ▶ Conceptual, un view este interogat la fel ca orice tabel
- ▶ În realitate, interogarea unui view este rescrisă prin inserarea interogării ce definește view-ul urmată de un proces de optimizare specific fiecărui SGBD
- ▶ Sintaxa

Create View *numeView* [*a1,a2,...*] **As** <*frază_select*>

Modificarea view-urilor

- ▶ View-urile sunt în general utilizate doar în interogări însă pentru utilizatorii externi ele sunt tabele: trebuie să poată suporta comenzi de manipulare/modificare a datelor
- ▶ Soluția: modificări asupra view-ului trebuie să fie rescrise în comenzi de modificare a datelor în tabelele de bază
 - ▶ de obicei este posibil
 - ▶ uneori există mai multe variante
- ▶ Exemplu
 - ▶ $R(A,B), V(A)=R[A]$, Insert into V values(3)
 - ▶ $R(N), V(A)=avg(N)$, update V set A=7

Modificarea view-urilor

Abordări

- ▶ creatorul view-ului rescrie toate comenzile de modificare posibile cu ajutorul declanșatorului de tip **INSTEAD OF**
 - ▶ acoperă toate cazurile
 - ▶ garantează corectitudinea?
- ▶ standardul SQL prevede existența de view-uri inerent actualizabile (updatable views) dacă:
 - ▶ view-ul e creat cu comanda select fără clauza **DISTINCT** pe o singură tabelă T
 - ▶ attributele din T care nu fac parte din definiția view-ului pot fi **NULL** sau iau valoare default
 - ▶ subinterogările nu fac referire la T
 - ▶ nu există clauza **GROUP BY** sau altă formă de agregare

View-uri materializate

Create Materialized View *V* [*a1,a2,...*] **As** <*frază_select*>

- ▶ are loc crearea unui nou tabel *V* cu schema dată de rezultatul interogării
- ▶ tuplele rezultat al interogării sunt inserate în *V*
- ▶ interogările asupra lui *V* se execută ca pe orice alt tabel
- ▶ Avantaje:
 - ▶ specifice view-urilor virtuale + crește viteza interogărilor
- ▶ Dezavantaje:
 - ▶ *V* poate avea dimensiuni foarte mari
 - ▶ orice modificare asupra tabelelor de bază necesită refacerea lui *V*
 - ▶ problema modificării tabelelor de bază la modificarea view-ului rămâne

Cum alegem ce materializăm

- ▶ dimensiunea datelor
- ▶ complexitatea interogării
- ▶ numărul de interogări asupra view-ului
- ▶ numărul de modificări asupra tabelelor de bază ce afectează view-ul și posibilitatea actualizării incrementale a view-ului
- ▶ punem în balanță timpul necesar execuției interogărilor și timpul necesar actualizării view-ului

...DEMO...
(fișierul *views.sql*)

Bibliografie

- ▶ Hector Garcia-Molina, Jeff Ullman, **Jennifer Widom**:
Database Systems: The Complete Book (2nd edition), Prentice Hall; (June 15, 2008)
- ▶ Oracle:
 - ▶ http://docs.oracle.com/cd/B28359_01/server.111/b28310/general005.htm
 - ▶ <http://www.oracle-base.com/articles/9i/MutatingTableExceptions.php>
 - ▶ http://www.dba-oracle.com/t_avoiding_mutating_table_error.htm



BAZE DE DATE

Indecși

Mihaela Elena Breabăn

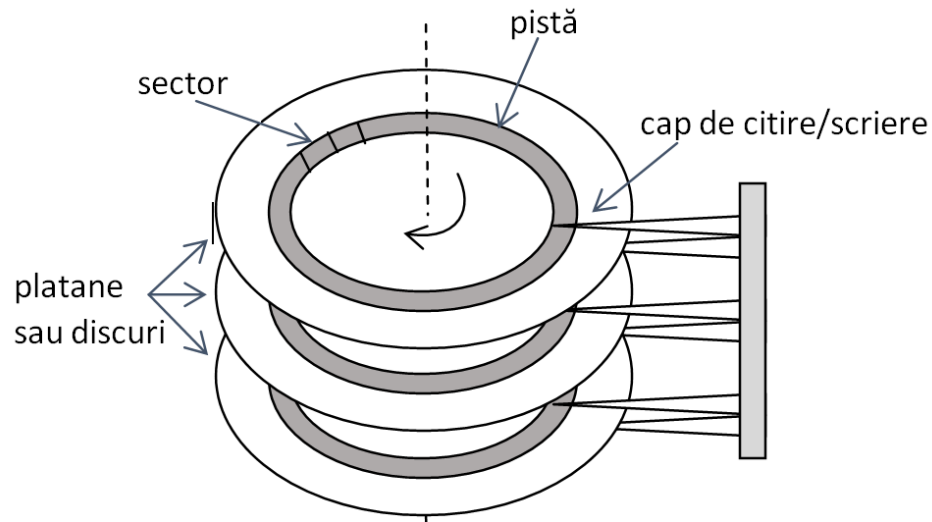
© FII 2015-2016

Indecși

Cuprins

- ▶ Stocarea fizica a datelor
- ▶ Indexare - motivatie
- ▶ Indecși ordonați
 - ▶ Indecși secvențiali
 - ▶ B⁺-Arbori
- ▶ Hashing
 - ▶ Hashing static
 - ▶ Hashing dinamic
- ▶ Acces multi-cheie și Indecși Bitmap
- ▶ Definirea indecșilor în SQL
- ▶ Indecși în Oracle

Stocarea fizica a datelor



- ▶ Viteza de citire a unui bloc este data de:
 - ▶ Viteza de miscare a bratului (pozitionare pe cilindru)
 - ▶ Viteza de rotatie a platanelor
 - ▶ Timpul de transfer

Indexare - Motivație

- Bazele de date consumă mult timp căutând

```
SELECT * FROM Student  
WHERE sID=40;
```

Cheie de căutare

Cum putem regăsi rezultatul în următoarele situații:

a) Ordine aleatoare a datelor

sID	sNume	medie
20	Ioana	9.5
40	Andrei	8.66
10	Tudor	8.55
30	Maria	8.33
70	Alex	9.33

b) Date secvențiale/ordonate

sID	sNume	medie
10	Tudor	8.55
20	Ioana	9.5
30	Maria	8.33
40	Andrei	8.66
70	Alex	9.33

Indexare – Motivație

- ▶ Căutarea binară
 - ▶ Complexitate: $\log_2(N)$
($\log_2(100\ 000)=17$)

```
SELECT * FROM Student  
WHERE sNume='Ioana';
```

Cum putem rezolva si aceasta interogare eficient?

- ▶ Solutia: Construirea unei structuri auxiliare care să ajute la localizarea unei înregistrări
 - ▶ Mecanismele de indexare sunt utilizate pentru a mări viteza de acces la datele dorite

Concepte de bază

- ▶ Un index este asociat cu o **cheie de căutare** = atribut sau set de attribute dintr-un fișier/tabel/relație utilizate pentru a căuta înregistrări în fișier
- ▶ **Fișier index** – constă din **înregistrări index** de forma

valoare cheie de căutare	pointer
--------------------------	---------

- ▶ **Fișier de date** – secvența de blocuri de memorie ce conține înregistrările unui tabel
- ▶ Sortare:
 - ▶ a indexului pe baza cheii de căutare
 - ▶ a fișierului/tabelei/relației stocate -> **cheie de sortare** = atribut care da ordonarea fișierului de date
- ▶ Un fișier de date poate avea asociați mai mulți indecși
- ▶ Fișierele index sunt de obicei de dimensiuni mult mai mici decât fișierul original cu date

Metrice de evaluare a indecșilor

- ▶ Timpul de acces
- ▶ Timpul de inserare
- ▶ Timpul de ștergere
- ▶ Spațiul necesar
- ▶ Tipurile de acces suportate eficient – influențează alegerea indexului
 - ▶ Înregistrări cu o valoare specificată a atributului
 - ▶ Înregistrări cu valoarea atributului inclusă într-un interval specificat

Tipuri de indecși

- ▶ **Indecși ordonați**: valorile cheii de căutare sunt stocate într-o anumită ordine
- ▶ **Indecși hash**: valorile cheii de căutare sunt distribuite uniform în bucket-uri cu ajutorul unei funcții hash
 - ▶ **Bucket**: o unitate de stocare conținând una sau mai multe înregistrări
- ▶ **Indecși bitmap**: asociați cheilor de căutare de tip attribute discrete, codifica distribuția valorilor sub formă de matrice binară

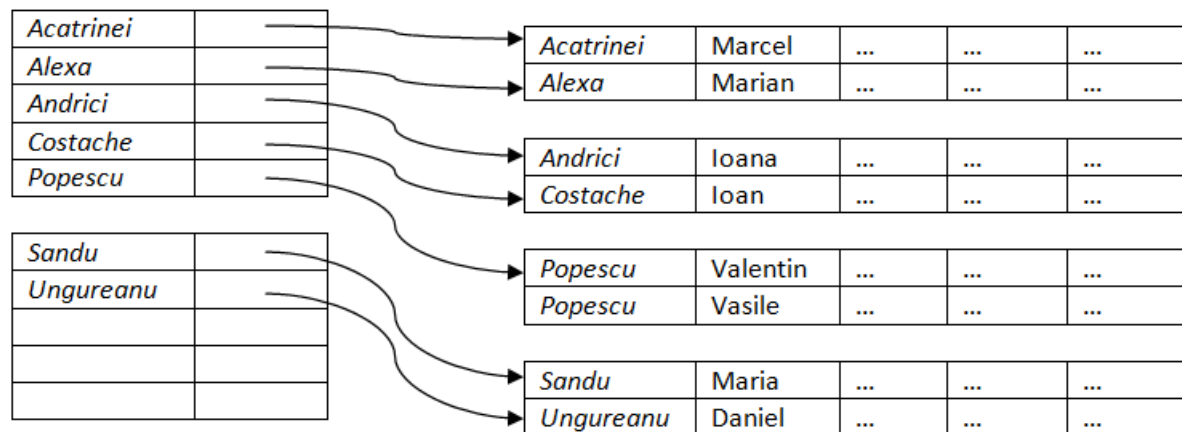
Indecsi ordonati: fisiere secventiale

Indecși secvențiali (fișiere secventiale)

- ▶ Intrările index sunt sortate după cheia de căutare
 - ▶ Ex: catalogul cu autori într-o bibliotecă
- ▶ **Index primar**: indexul a cărui cheie de căutare definește și ordonarea secvențială a fișierului/tabelei/relației
 - ▶ denumit și **index de grupare** (clustering/clustered index)
 - ▶ cheia de căutare a unui index primar este de obicei cheia primară dar nu e obligatoriu
 - ▶ o tabelă poate avea cel mult un index primar
- ▶ **Index secundar** (nonclustering/nonclustered): index a cărui cheie de căutare specifică o ordonare diferită de ordonarea secvențială a fișierului cu date

Fișiere index dense

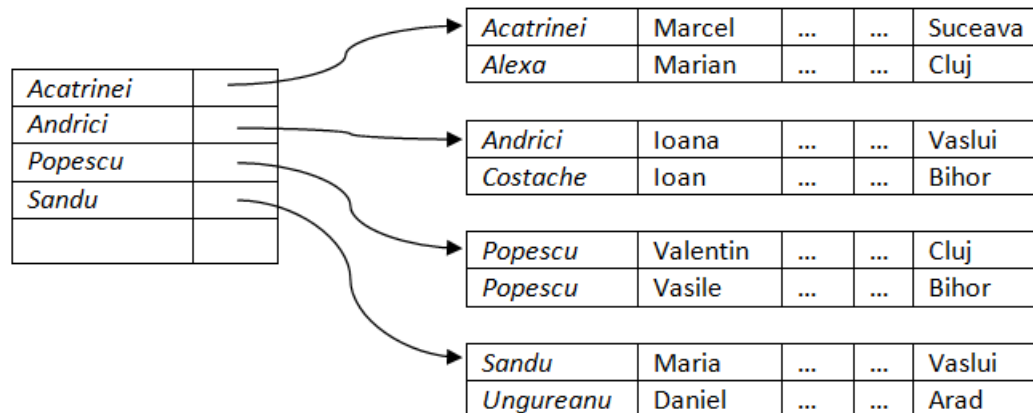
- ▶ **Index dens:** există înregistrări index pentru fiecare valoare a cheii de căutare în fișier/tabel/relație
- ▶ Dacă indexul e primar va păstra câte un singur pointer-doar la prima intrare cu valoarea respectivă
- ▶ Dacă indexul e secundar vor fi necesari mai mulți pointeri la o singură valoare a cheii de căutare



Index dens primar: cheia de cautare coincide cu cheia de sortare a fisierului de date

Fișiere index rare

- ▶ **Index rar**: conține intrări doar pentru unele valori a cheii de căutare
 - ▶ Aplicabil doar când înregistrările sunt ordonate secvențial după cheia de căutare
 - ▶ Balansul timp-spațiu
 - ▶ De obicei o intrare în index corespunde unui bloc din fisierul de date
- ▶ Pentru a localiza o înregistrare cu valoarea k a cheii de căutare:
 - ▶ Se determină înregistrarea index cu cea mai mare valoare a cheii de căutare $< k$
 - ▶ Se caută secvențial în fisier începând cu înregistrarea spre care indică înregistrarea index

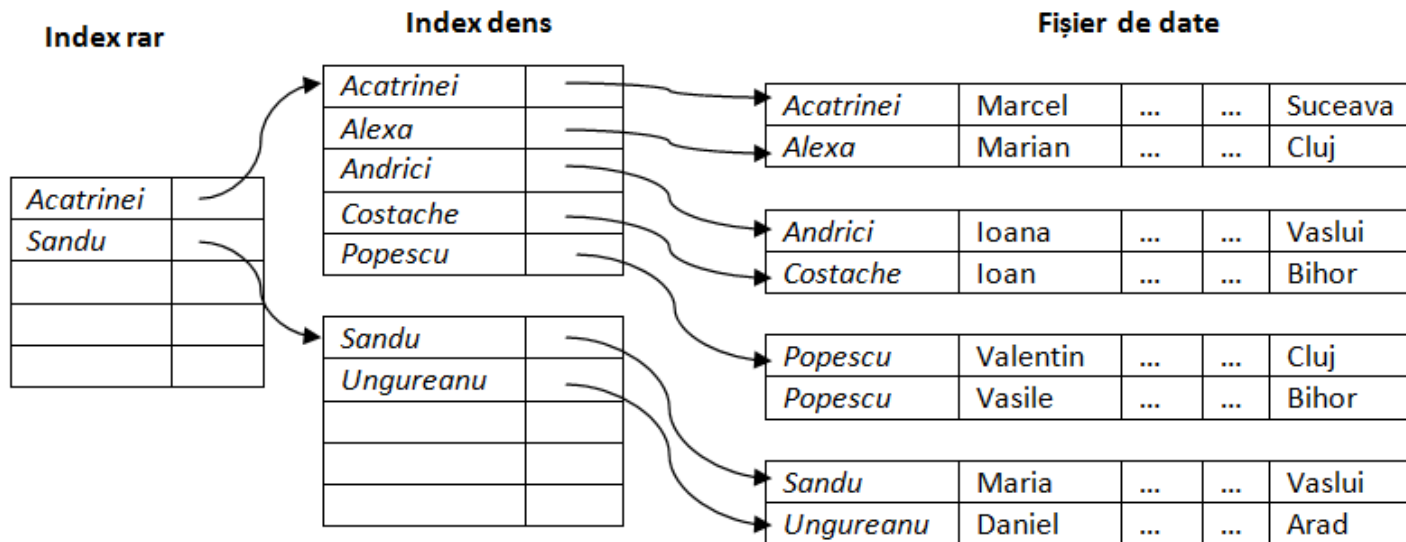


Index rar: cheia de cautare coincide **obligatoriu** cu cheia de sortare a fisierului de date

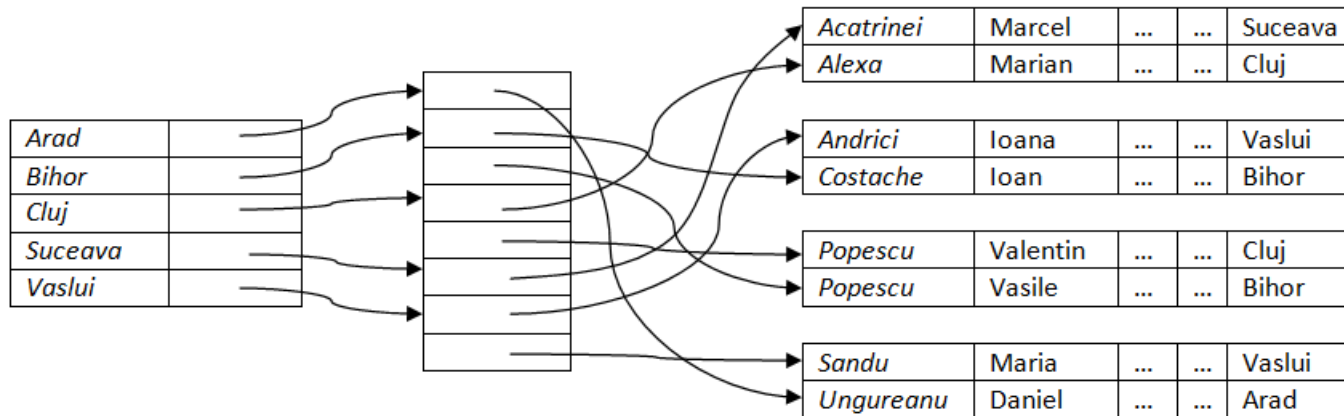
Indecși multi-nivel

- ▶ **Index multi-nivel:** index asociat unui alt index
 - ▶ Dacă indexul primar nu încapă în memorie accesul devine costisitor
 - ▶ Soluția: indexul primar păstrat pe disc este tratat ca un fișier secvențial și se construiește un index rar pentru el
- ▶ Indexul extern – un index rar al indexului primar
- ▶ Index intern – fișierul index primar
- ▶ Dacă și indexul extern este prea mare pentru a încăpea în memorie, se creează un index pe un nou nivel, etc...
- ▶ Indecșii de pe toate nivelurile trebuie actualizați la inserare și ștergere în fișierul cu date

Indecși multi-nivel



Indecși secundari



- ▶ În relația *Studenti* sortată după nume, care sunt studenții ce locuiesc în Cluj?
- ▶ Soluția: index secundar (dens!)
- ▶ Pentru a implementa relația de tip unu-la-multi dintre index și datele destinație se utilizează referințe la bucket-uri de pointeri

Actualizarea indecșilor secvențiali

Ștergere

- ▶ Ștergerea înregistrării din fișierul cu date atrage modificări asupra indexului
- ▶ Dacă înregistrarea ștearsă este singura care conține o valoare particulară a cheii de căutare, aceasta este ștearsă și din index
- ▶ Ștergerea în
 - ▶ Indecși denși: similară ștergerii din fișierul cu date
 - ▶ Indecși rari:
 - ▶ dacă există o intrare a cheii de căutare în index aceasta este înlocuită cu următoarea valoare a cheii de căutare din fișier (în ordinea cheii de căutare)
 - ▶ dacă următoarea valoare a cheii de căutare deja are o intrare în index, este efectuată ștergerea.

Actualizarea indecșilor secvențiali

Inserare

- ▶ Este necesară localizarea valorii cheii de căutare ce apare în tuplul inserat
- ▶ Indecși denși: dacă valoarea nu apare în index se va insera
- ▶ Indecși rari: dacă indexul păstrează o intrare pentru fiecare bloc al fișierului nu sunt necesare modificări decât dacă un nou bloc este creat (prima valoare a cheii de căutare ce apare în noul bloc este inserată în index)
- ▶ Inserarea în fișierul cu date ordonat și în indexul secvențial poate necesita crearea unor blocuri de exces -> degenerarea structurii secvențiale
- ▶ Inserarea (și ștergerea) pentru indecși multi-nivel sunt extensii simple a algoritmilor uni-nivel prezentați

Indecsi ordonati: B+-arbori

Fișiere index de tip B⁺-arbori

Motivație

- ▶ Organizarea secvențială a indecșilor se degradează pe măsură ce dimensiunea crește
- ▶ Reconstruirea indecșilor la intervale de timp e necesară însă costisitoare
- ▶ Indexul B⁺-arbore
 - ▶ mărește viteza de localizare și elimină necesitatea constantă de reorganizare
 - ▶ utilizați extensiv

Structura B⁺-arbore

(1)

- ▶ Un arbore balansat astfel încât fiecare drum de la rădăcină la frunze are aceeași lungime
- ▶ Un nod tipic

P ₁	K ₁	P ₂	K ₂	...	P _m	K _m	P _{m+1}
----------------	----------------	----------------	----------------	-----	----------------	----------------	------------------

- ▶ K_i sunt valori a cheii de căutare
- ▶ P_i sunt pointeri către
 - ▶ noduri copil/descendente (noduri care nu sunt frunze)
 - ▶ înregistrări sau bucket-uri de înregistrări (noduri frunză)
- ▶ Arborele este definit de o constantă m care specifică numărul maxim de valori dintr-un nod (numarul maxim de pointeri /descendenti este m+1)
 - ▶ De obicei dimensiunea unui nod e cea a unui bloc
- ▶ Valorile cheii de căutare într-un nod sunt ordonate

$$K_1 < K_2 < K_3 < \dots < K_m$$

Noduri care nu sunt frunze în B⁺-arbore

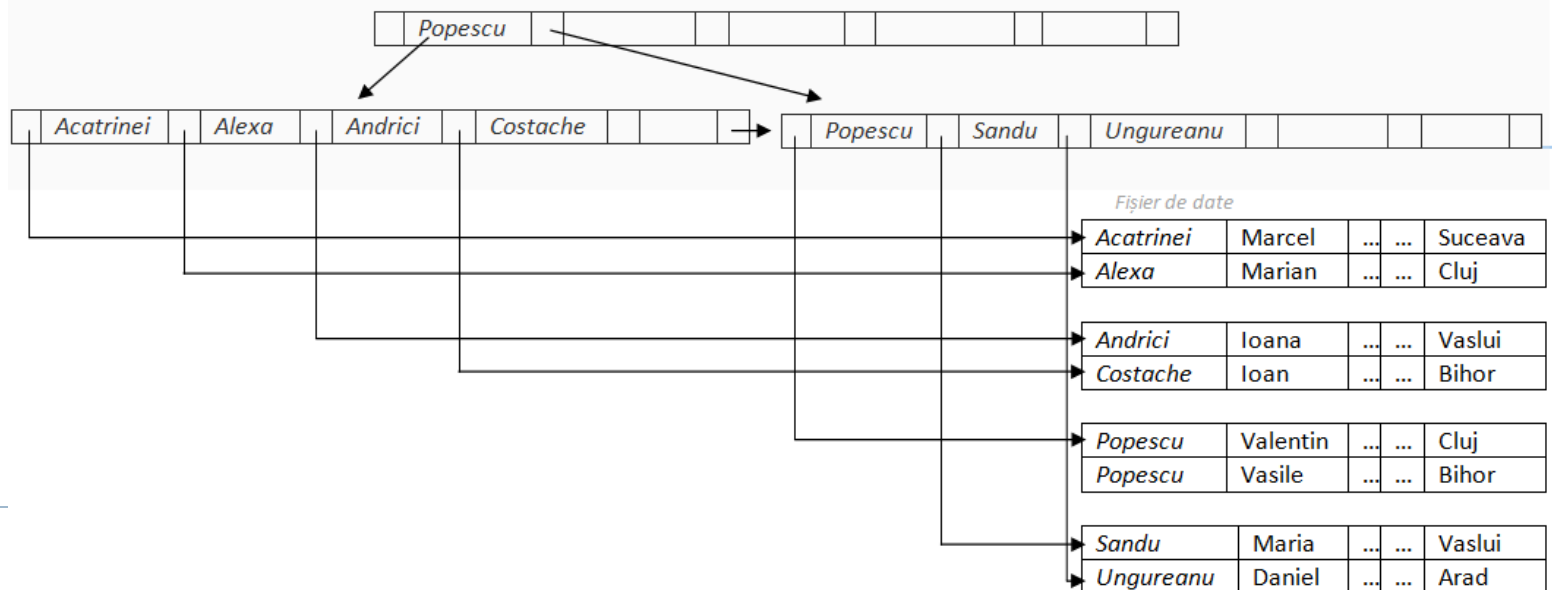
- ▶ formează un index rar multi-nivel pentru nodurile frunză
- ▶ Pentru un nod cu n pointeri:
 - ▶ Toate valorile cheii de căutare din subarborele spre care P_1 indică sunt mai mici decât K_1
 - ▶ Pentru P_i , $2 \leq i \leq n - 1$, toate valorile cheii de căutare din subarborele spre care indică sunt mai mari sau egale cu K_{i-1} și mai mici decât K_i
 - ▶ Toate valorile cheii de căutare din subarborele spre care indică P_n sunt mai mari sau egale cu K_{n-1}

P_1	K_1	P_2	K_2	...	P_m	K_m	P_{m+1}
-------	-------	-------	-------	-----	-------	-------	-----------

Structura B⁺-arbore

(2)

- ▶ Nodurile nu sunt complet ocupate pt. a evita reorganizari frecvente:
 - ▶ nodul radacina are cel putin doi si cel mult $m + 1$ pointeri/descendenti (corespunzator, cel putin una si cel mult m valori, ordonate crescator)
 - ▶ fiecare nod de pe un nivel intermediar are cel putin $\lceil (m+1)/2 \rceil$ si cel mult $m + 1$ pointeri/descendenti (corespunzator, cel putin $\lceil m/2 \rceil$ si cel mult m valori ordonate crescator)
 - ▶ fiecare nod frunza are cel putin $\lceil m/2 \rceil$ si cel mult m valori; toti pointerii fac trimitere catre fisierul de date, cu exceptia ultimului pointer care face trimitere catre urmatorul nod frunza (cu valori mai mari).



B⁺-arbore

Exemplu

- ▶ 1 bloc memorie = 1024 octeti
- ▶ Cheia de cautare = sir de max. 20 caractere (1 caracter=1 octet)
- ▶ 1 pointer = 8 octeti
- ▶ Numarul maxim de intrari in nod?
 - ▶ Cea mai mare valoare m cu proprietatea $20m + 8(m + 1) \leq 1024$.
 - ▶ $m=36$
- ▶ Radacina: cel putin una, cel mult 36 valori
- ▶ Nod intermediar: cel putin 19, cel mult 37 pointeri
- ▶ Nod frunza: cel putin 18, cel mult 36 valori = pointeri catre fisierul de date

B⁺-arbori

Observații

- ▶ Fiindcă conexiunile dintre noduri se realizează prin pointeri, blocuri apropiate logic nu trebuie să fie apropiate și fizic
- ▶ Nivelele diferite de nivelul frunză formează o ierarhie de indecși rari
- ▶ Ultimul nivel = index secvențial dens
- ▶ B⁺-arborele conține un număr relativ mic de nivele
 - ▶ Cel mult $\lceil \log_{\lceil (m+1)/2 \rceil}(K) \rceil$ pentru k valori a cheii de căutare
 - ▶ Nivelul imediat următor rădăcinii: cel puțin 2 noduri
 - ▶ Următorul: $2 * \lceil (m+1)/2 \rceil$ noduri
 - ▶ Următorul: cel puțin $2 * \lceil (m+1)/2 \rceil * \lceil (m+1)/2 \rceil$
 - ▶ etc...
- ▶ Inserările și ștergerile se fac eficient, restructurarea indexului necesitând timp logaritm

Interogări pe B⁺-arbori

Algoritm

Determinarea tuturor înregistrărilor cu valoarea k a cheii de căutare

1. N =rădăcina
2. Repetă
 - Caută în N cea mai mică valoare a cheii de căutare $>k$
 - Dacă aceasta există și e egală cu K_i , atunci $N = P_i$
 - Altfel $N = P_n$ ($k \geq K_{n-1}$)

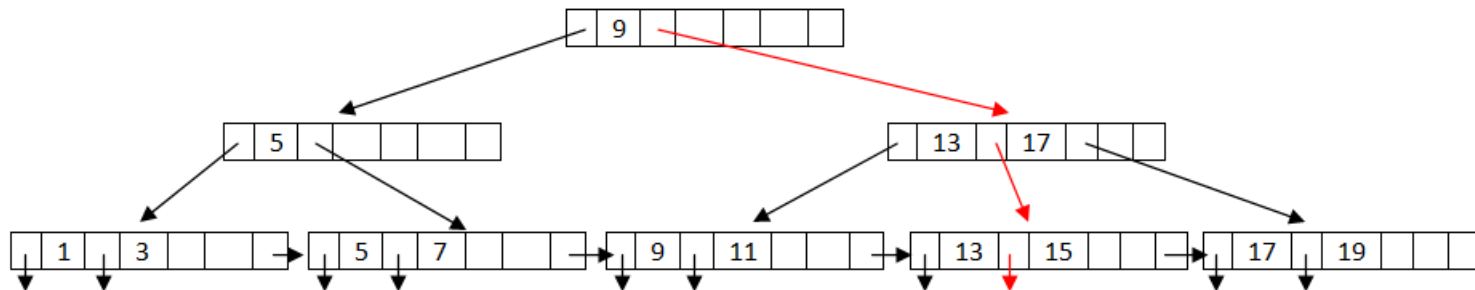
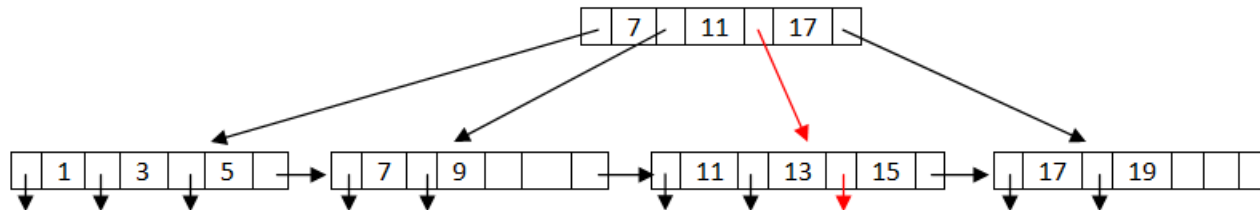
Până N este nod frunză

3. Dacă există $K_i = k$, pointerul P_i indică înregistrarea dorită
4. Altfel nu există înregistrarea cu cheia de căutare k

Interogari

Cheia de cautare - stocheaza numerele impare intre 1-19

Intrarea 15?



Interogări pe B⁺-arbori

Observații

▶ Ex:

- ▶ Un nod este în general de aceeași dimensiune ca a unui bloc pe disc - 4Kocteti
 - ▶ Pp. $m=100$
 - ▶ Fiecare acces al unui nod poate necesita o localizare pe disc (<20 milisecunde)
-
- ▶ *Pentru 1 milion valori a cheii de căutare, câte noduri (blocuri pe disc) sunt accesate la o căutare în B⁺-arbore? (4)*
 - ▶ *Dar dacă se utilizează un index secvențial? (20)*

Actualizări în B⁺-arbori

Inserarea

1. Determină nodul frunză în care va apărea valoarea cheii de căutare
2. Dacă valoarea e deja prezentă într-un nod frunză
 1. Adaugă înregistrarea în fișier/tabel/relație
 2. Adaugă un pointer în bucket
3. Dacă nu e prezentă valoarea
 1. Adaugă înregistrarea în fișier/tabel/relație
 2. Dacă e loc în nodul frunză inserează perechea (valoare cheie, pointer)
 3. Altfel divide nodul

Actualizări în B⁺-arbori

Inserarea: divizarea nodurilor

► Divizarea unui nod frunză

1. Se iau cele n perechi (inclusiv cea care urmează a fi inserată) ordonate. În nodul original se pun primele $\lceil n/2 \rceil$ perechi iar restul într-un nod nou
2. Fie p noul nod și k cea mai mică valoare din p . Inserează (k,p) în părintele nodului care se divide
3. Dacă părintele este plin acesta se divide la rândul său și divizarea se propagă în sus până când un nod nu este plin. În cel mai rău caz nodul rădăcină este divizat ceea ce crește înălțimea arborelui cu 1.

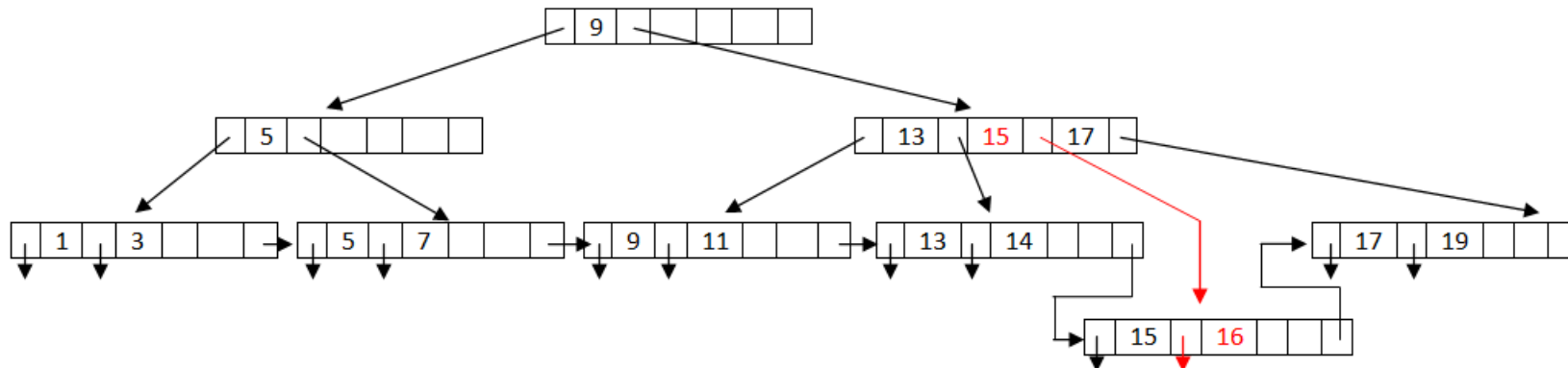
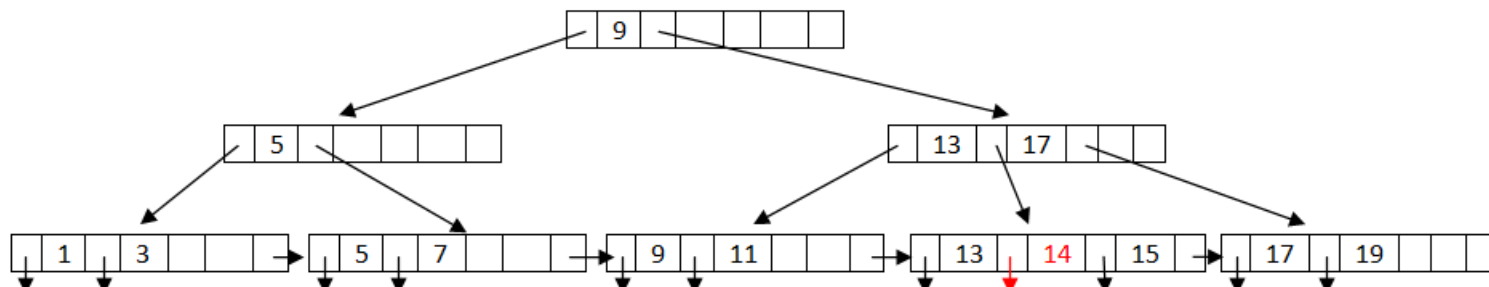
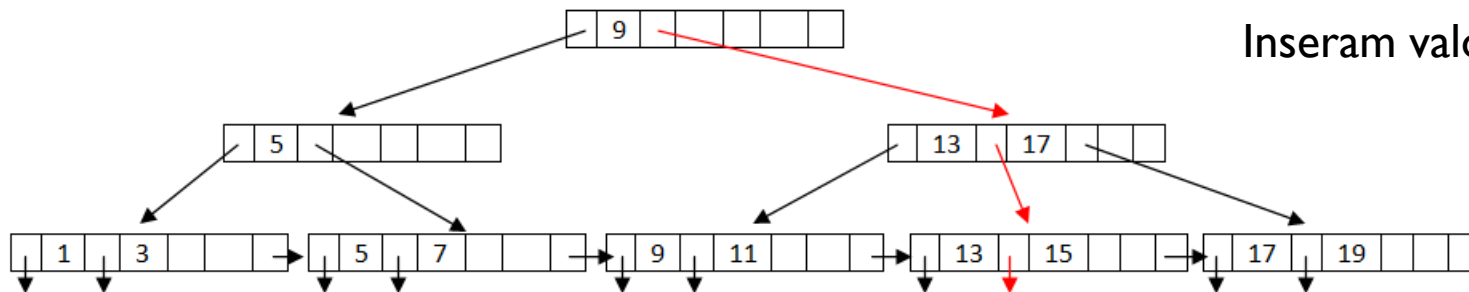
► Divizarea unui nod plin intern N la inserarea unei perechi (k,p)

1. Se creează un nod temporar M cu spațiu pentru $n+1$ pointeri și n valori în care se copie N și perechea (k,p)
2. Se copie $P_1, K_1, \dots, K_{\lceil n/2 \rceil - 1}, P_{\lceil n/2 \rceil}$ din M înapoi în N
3. Se copie $P_{\lceil n/2 \rceil + 1}, K_{\lceil n/2 \rceil + 1}, \dots, K_n, P_{n+1}$ din M într-un nou nod N'
4. Inserează $(K_{\lceil n/2 \rceil}, N')$ în părintele lui N

Actualizări în B⁺-arbori

Inserarea: Exemplu

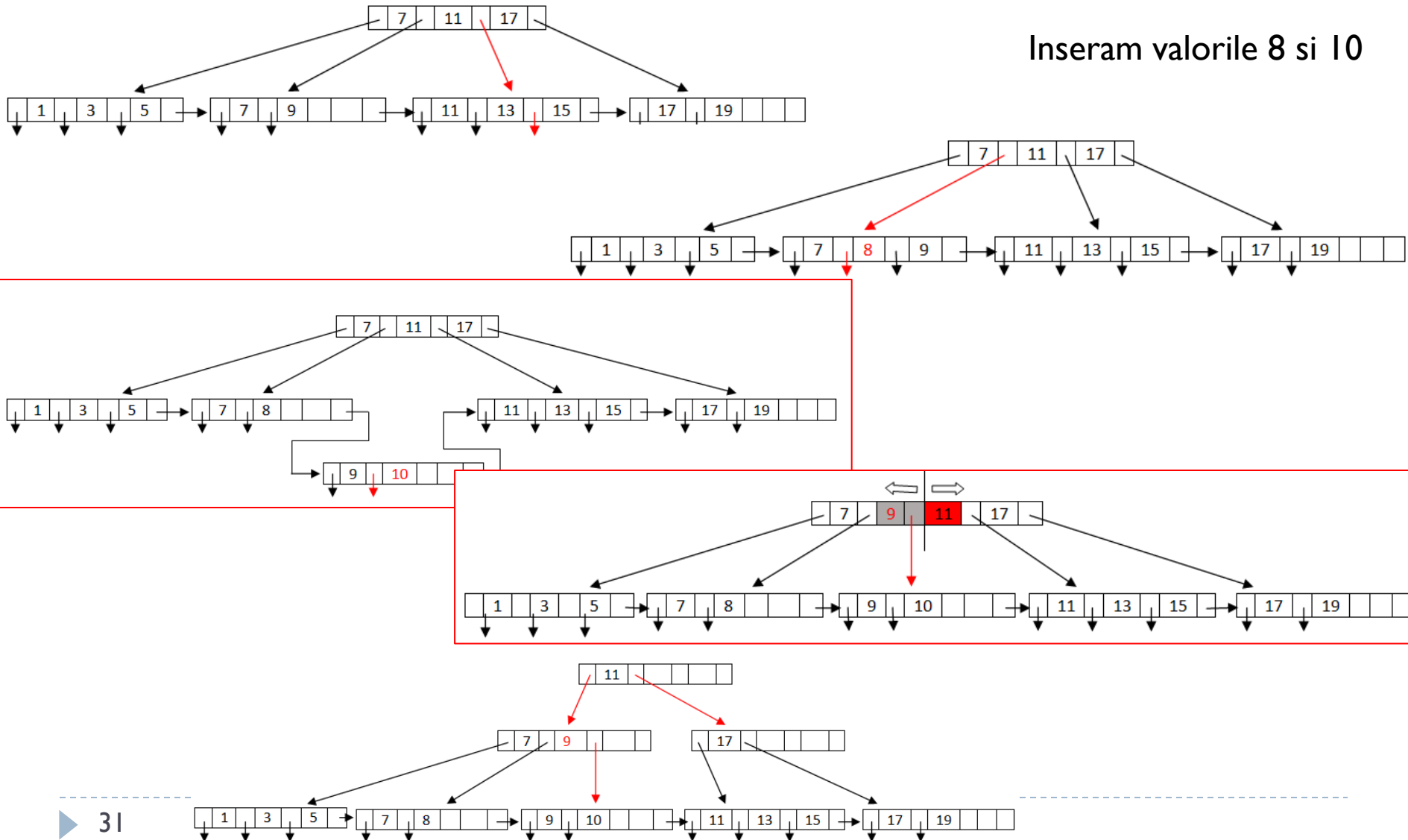
Inseram valorile 14 si 16



Actualizări în B⁺-arbori

Inserarea: Exemplu(2)

Inseram valorile 8 si 10



Actualizări în B⁺-arbori

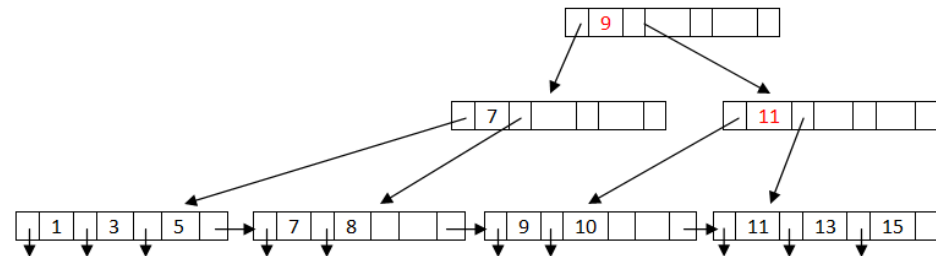
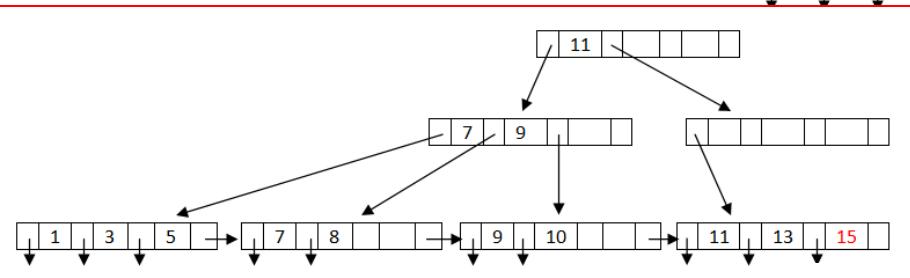
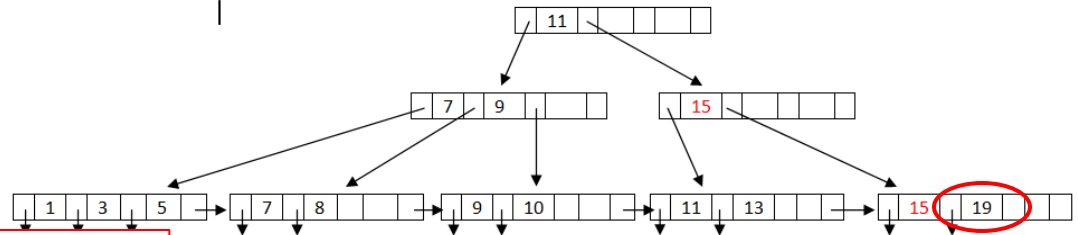
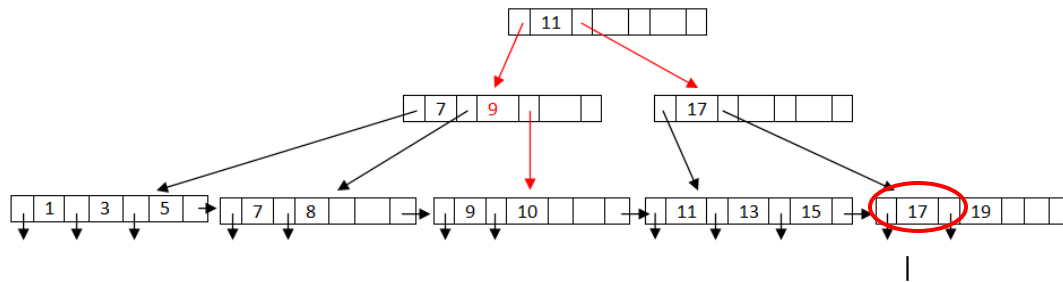
Ștergerea

1. Șterge înregistrarea din fișier/tabel/relație
2. Dacă înregistrarea face parte dintr-un bucket, e ștearsă din acesta. Altfel (sau dacă bucketul devine gol) șterge din nodul frunză perechea (pointer, valoare cheie)
3. Dacă nodul va avea prea puține intrări în urma ștergerii și ele încap într-un nod vecin va avea loc unirea:
 1. Se inserează toate intrările în nodul stâng și se șterge celălalt
 2. Se șterge perechea (K_{i-1}, P_i) , unde P_i este pointerul către nodul șters de la părinte. Dacă e necesar se propagă ștergerea recursiv în sus. Dacă nodul rădăcină rămâne cu un singur pointer va fi șters.
4. Altfel, dacă nodurile nu încap în vecin se redistribuie pointerii:
 1. Se redistribuie astfel încât avem satisfăcută condiția de minim în ambele noduri
 2. Se actualizează valoarea cheii de căutare corespunzătoare în părinte

Actualizări în B⁺-arbori

Ștergerea: Exemplu (1)

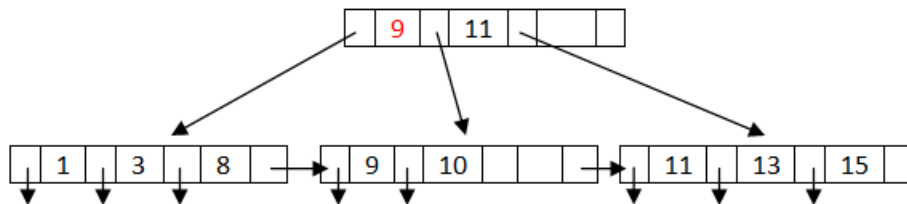
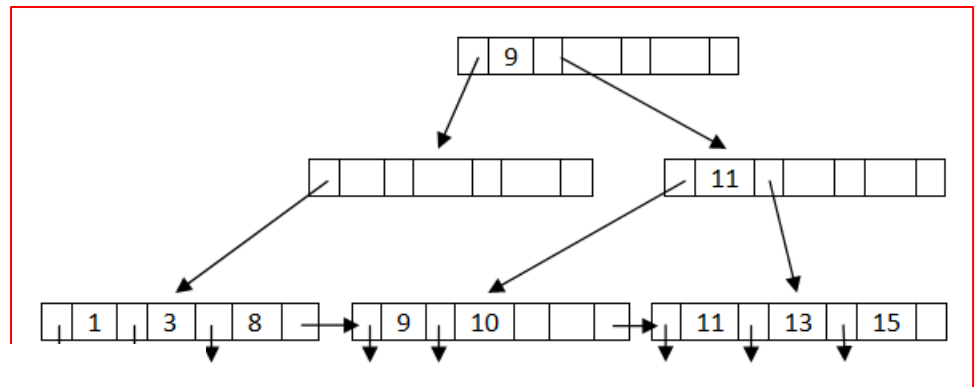
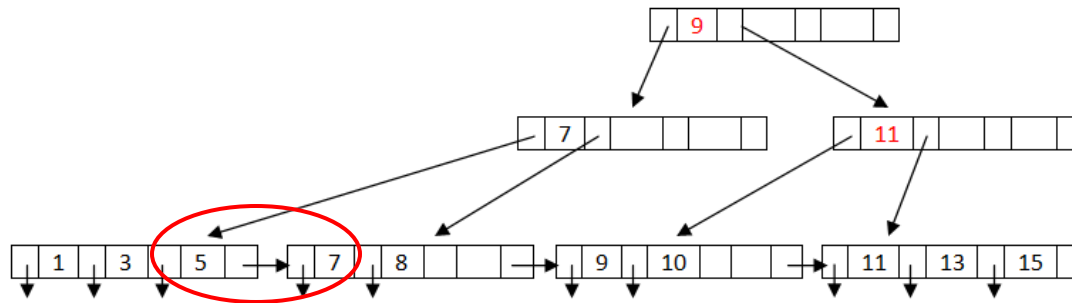
Stergem valorile 17 și 19



Actualizări în B⁺-arbori

Ștergerea: Exemplu (2)

Stergem valorile 5 și 7



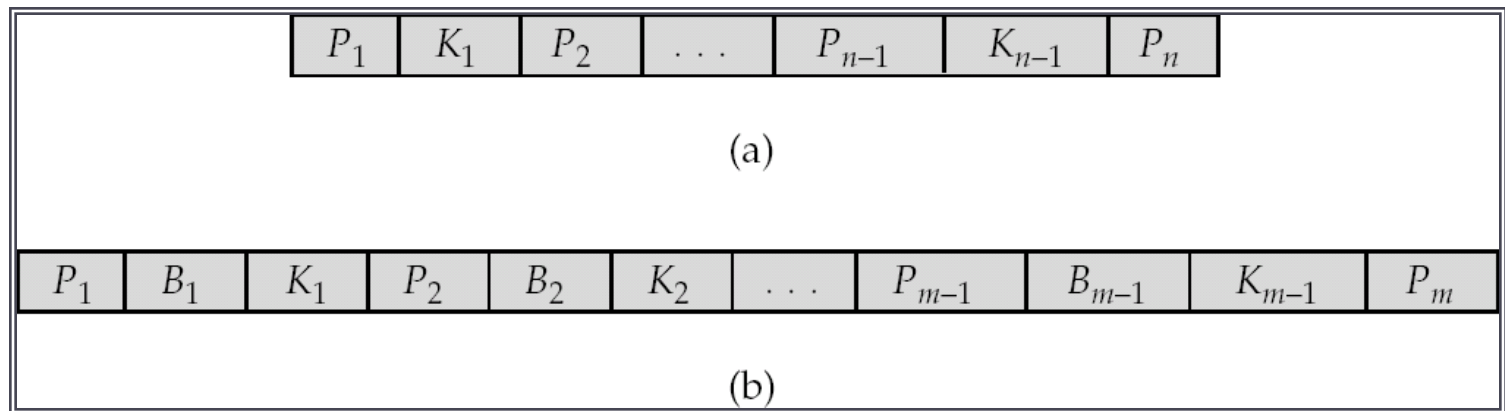
B⁺-arbori

Eficiența

- ▶ **Căutare:** cel mult $\lceil \log_{(m+1)/2}(K) \rceil$ blocuri transferate
 - ▶ Datorită înlănțuirii nodurilor frunză sunt eficienți și pt. Căutări în interval
- ▶ **Inserare, stergere:** cel mult $2 \lceil \log_{(m+1)/2}(K) \rceil$ blocuri transferate

Indecși B-arbore

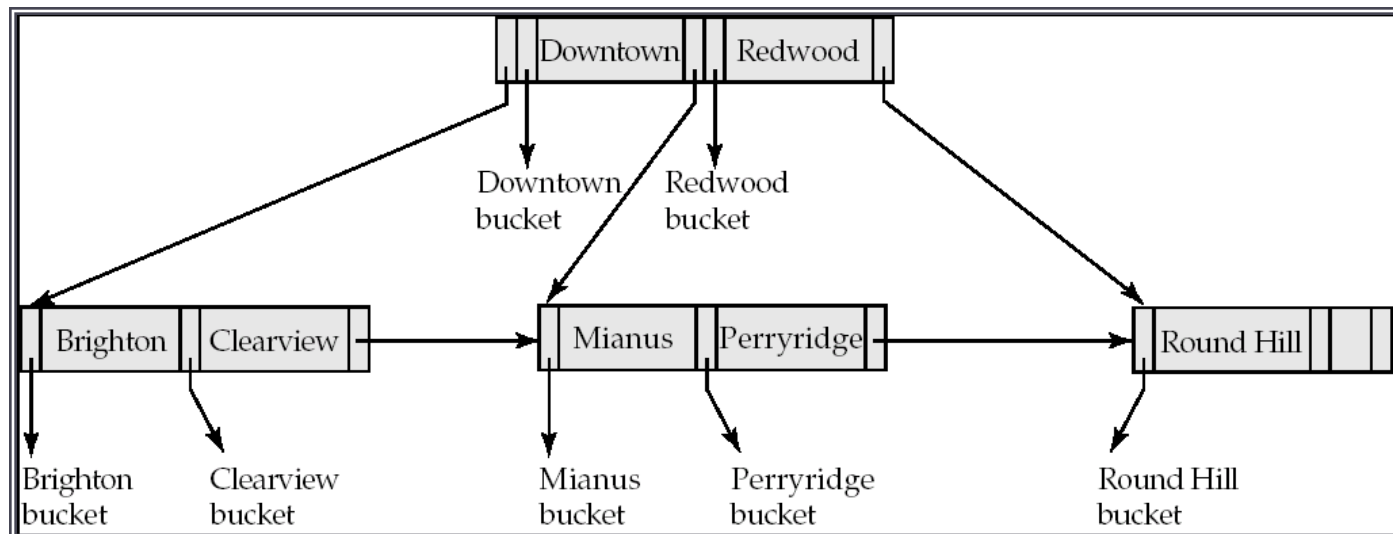
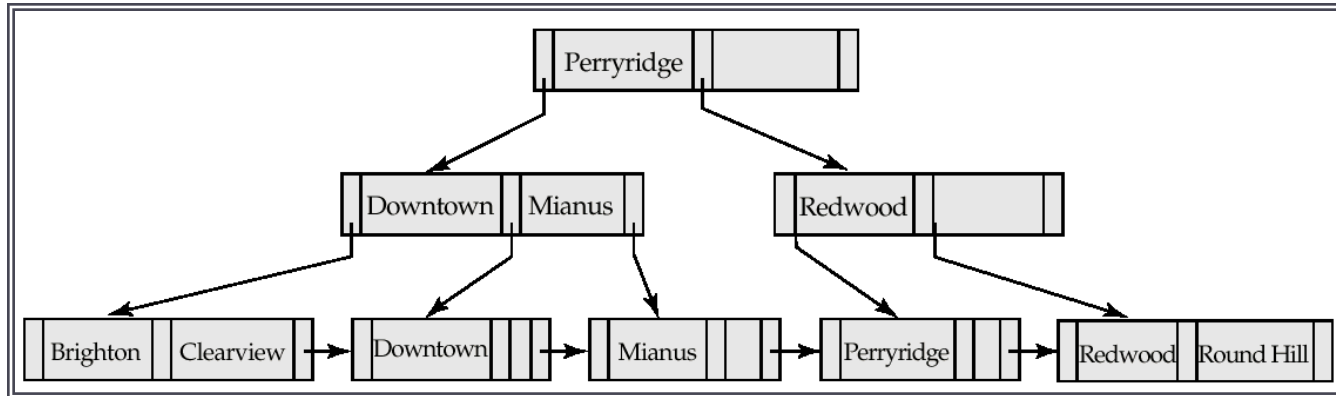
- ▶ Asemănători B⁺-arborilor însă permit o singură apariție a valorilor cheilor de căutare
- ▶ Cheile de căutare în nodurile care nu sunt frunză nu mai apar nicăieri în arbore ceea ce necesită introducerea unui pointer adițional



- ▶ Pointerii B_i sunt pointeri către înregistrări sau bucketuri

Indecși B-arbore

Exemplu*



Indecși B-arbore

Observații

▶ Avantaje

- ▶ Pot utiliza mai puține noduri decât B^+ -arborele corespunzător
- ▶ E posibil a se localiza valoarea căutată înainte de a ajunge la frunze

▶ Dezavantaje

- ▶ Nodurile care nu sunt frunze sunt mai mari ceea ce necesită reducerea numărului de valori stocate; înălțimea va fi mai mare
 - ▶ Inserările și ștergerile sunt mai complicate
 - ▶ Implementarea e mai dificilă
 - ▶ Nu e posibil a fi scanat un tabel doar cu ajutorul frunzelor
- ▶ Avantajele nu cântăresc mai mult decât dezavantajele, B^+ -arborii fiind preferați de către SGBD-uri

Indecsi ordonati:
indecsi multi-cheie

Acces multi-cheie

- ▶ Pot fi utilizați mai mulți indecși la o interogare

```
SELECT *  
FROM studenti  
WHERE judet = 'Bihor' AND an > 2010;
```

- ▶ Strategii posibile pentru utilizarea indecșilor uni-atribut:
 - ▶ Utilizarea indexului cu cheia de căutare *judet*
 - ▶ Utilizarea indexului cu cheia de căutare *an*
 - ▶ Utilizarea ambilor și efectuarea intersecției
- ▶ Dezavantaje:
 - ▶ Pot exista multe înregistrări ce satisfac numai una dintre condiții

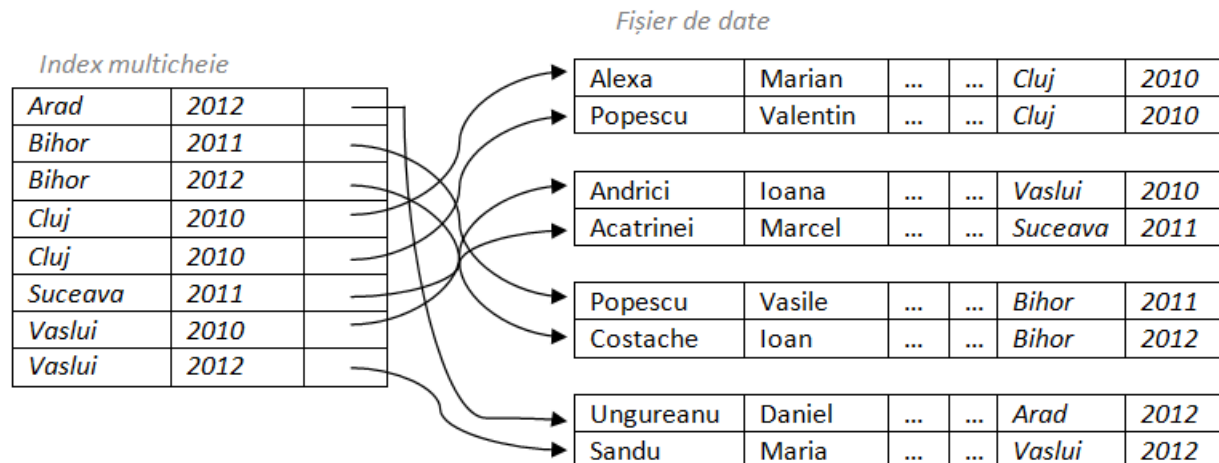
Indecși multi-cheie

- ▶ Cheile de căutare compuse sunt chei ce conțin mai mult de un atribut
- ▶ Ordinea lexicografică: $(a_1, a_2) < (b_1, b_2)$ dacă
 - ▶ $a_1 < b_1$ sau
 - ▶ $a_1 = b_1$ și $a_2 < b_2$

Ex. (*judet*, *an*)

Pot fi rezolvate eficient condițiile de mai jos?

- a) where *judet* = 'Bihor' AND *an* > 2010
- b) where *judet* > 'Bihor' AND *an* = 2010



Eficiența

- ▶ Ordinea atributelor într-un index multi-cheie este foarte importantă
- ▶ Eficiența depinde de selectivitatea atributelor

k-d arbori

- ▶ **Generalizare a arborelui binar de cautare:**
 - ▶ k = nr. de attribute ce formeaza cheia de cautare
 - ▶ Fiecare nivel corespunde unui atribut din cele k
 - ▶ Succesiunea nivelelor reprezintă iterări peste mulțimea celor k attribute
 - ▶ Pentru a obține arbori echilibrați, la nivelul fiecărui nivel în noduri este pusă valoarea mediană

Organizarea hash/Indexsi hash

Hashing

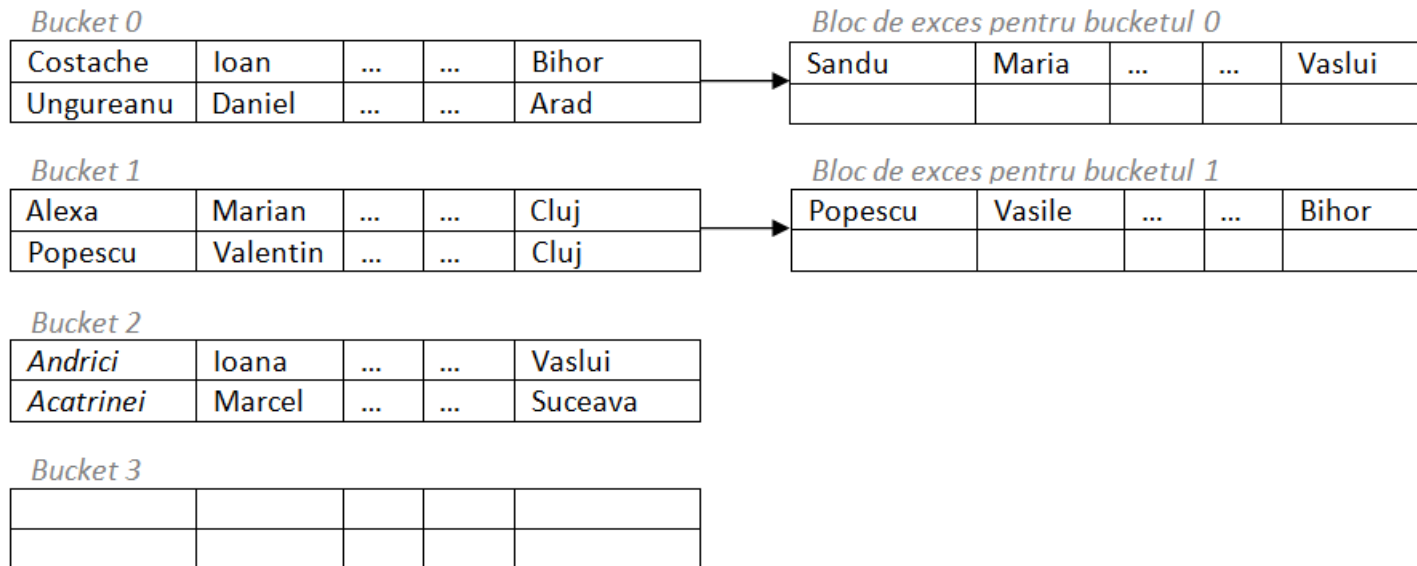
- ▶ În **organizarea de tip hash** a fișierului/tabelului/relației înregistrările sunt grupate în bucketuri care pot fi localizate pe baza valorilor cheii de căutare (un bucket ia dimensiunea unui bloc)
- ▶ **Funcția hash (de dispersie)** $h:K \rightarrow B$ este o funcție de la mulțimea valorilor cheii de căutare la mulțimea adreselor tuturor bucketurilor
 - ▶ Localizează înregistrările pentru acces, inserare, ștergere
- ▶ Înregistrări cu valori diferite ale cheii de căutare pot fi mapate la același bucket
 - ▶ Căutare secvențială în bucket

Funcții hash

- ▶ Cerințe
 - ▶ Uniformitate
 - ▶ Caracter aleatoriu
- ▶ Funcțiile hash tipice au la bază calcule pe reprezentarea binară internă a cheii de căutare
- ▶ Pot apărea situații de depășire a bucketului, caz în care se utilizează bucketuri de exces

Organizarea de tip hash

Exemplu



Organizarea de tip hash utilizând *nume* drept cheie:

Funcția hash: $h: \text{Dom}(\text{nume}) \rightarrow \{0, 1, 2, 3\}$ - suma reprezentărilor binare modulo 4

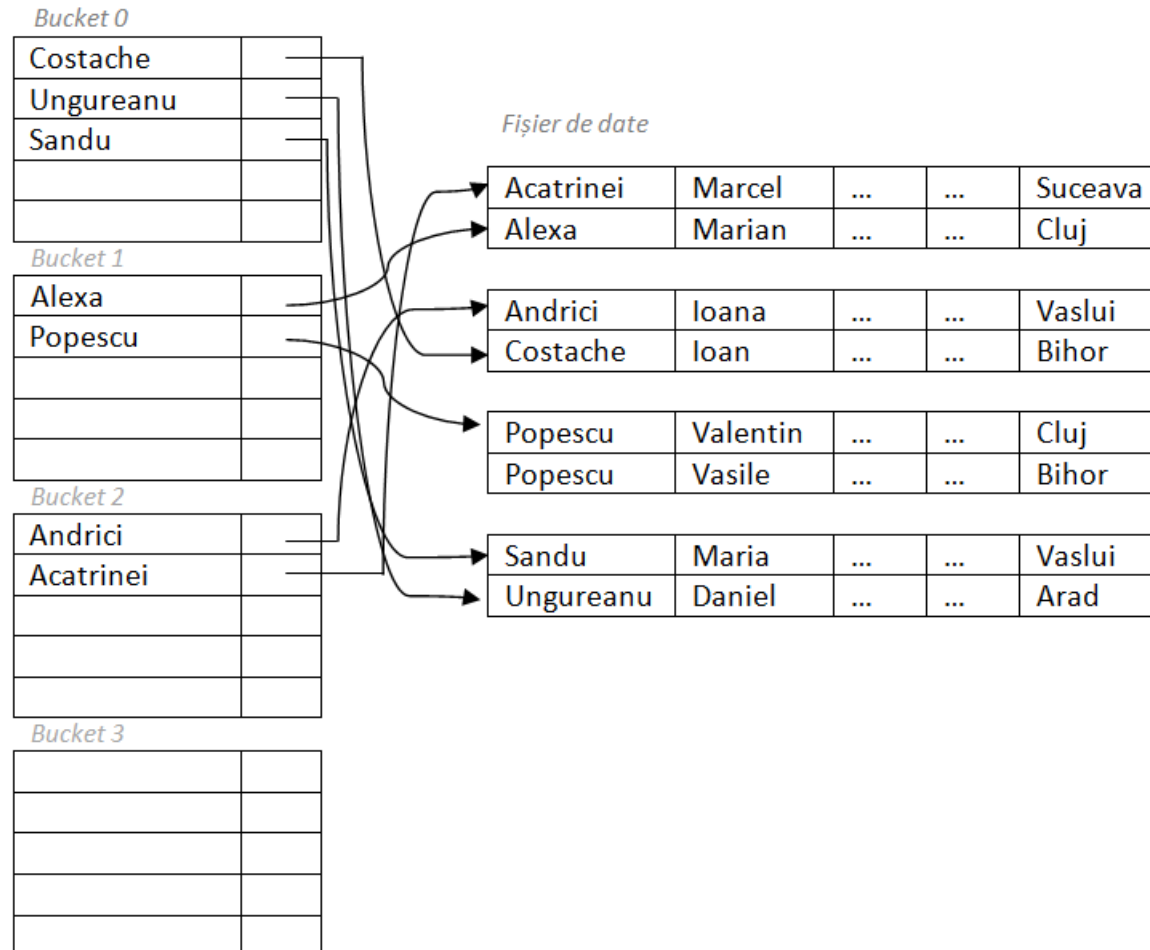
'Acatrinei':

'1000001 1100011 1100001 1110100 1110010 1101001 1101110 1100101 1101001'

$h(\text{'Acatrinei'}) = 34 \% 4 = 2.$

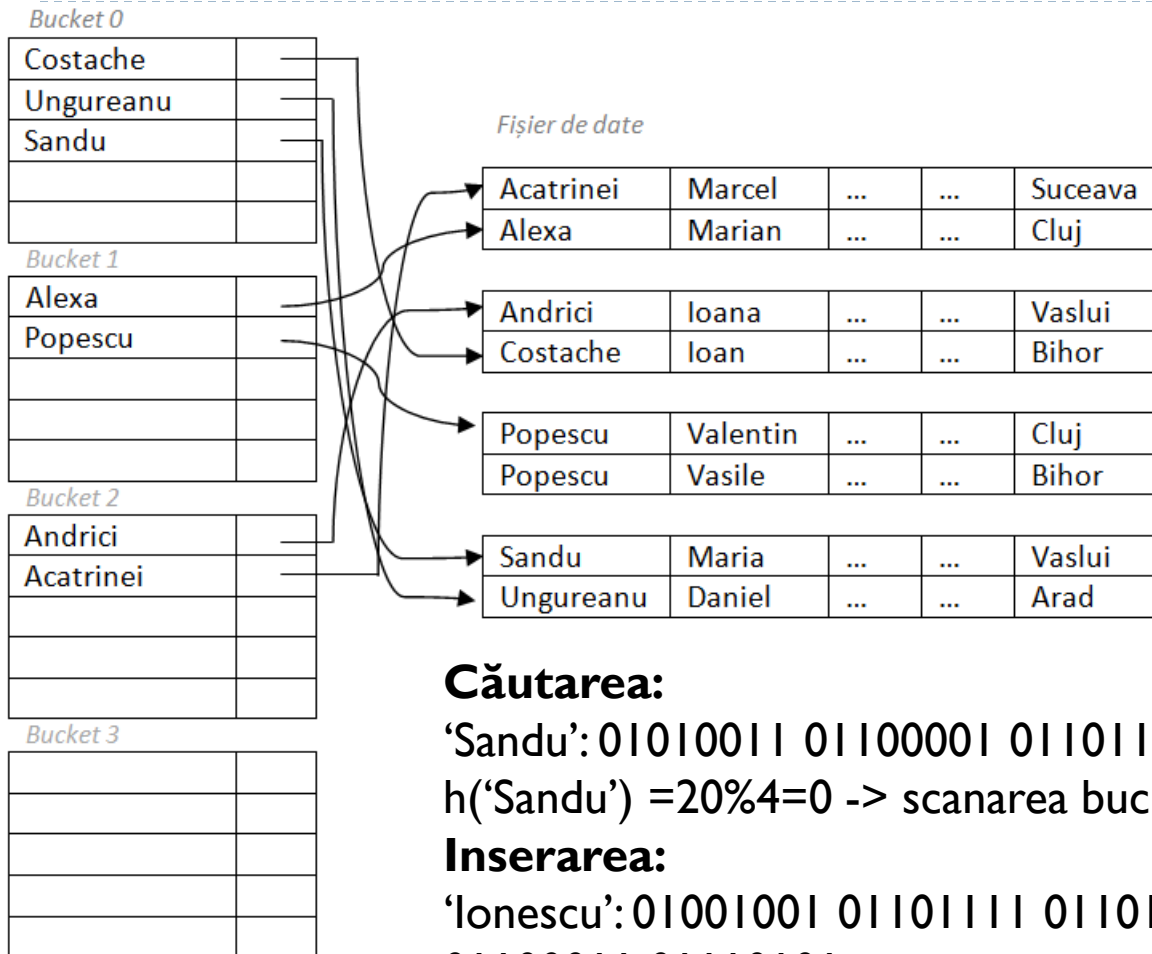
Indecși hash

- Organizează cheile de căutare cu pointerii asociați într-o structură de tip hash



Indecși hash

Operații



Căutarea:

'Sandu': 01010011 01100001 01101110 01100100 01110101
 $h(\text{'Sandu'}) = 20\%4 = 0 \rightarrow$ scanarea bucketului 0

Inserarea:

'Ionescu': 01001001 01101111 01101110 01100101 01110011
 01100011 01110101

$H(\text{Ionescu}) = 32\%4 = 0 \rightarrow$ inseram in fisierul de date si apoi in bucketul 0 in index

Ștergerea: calcul hash, scanare bucket, stergere



Indecși hash

Performanța

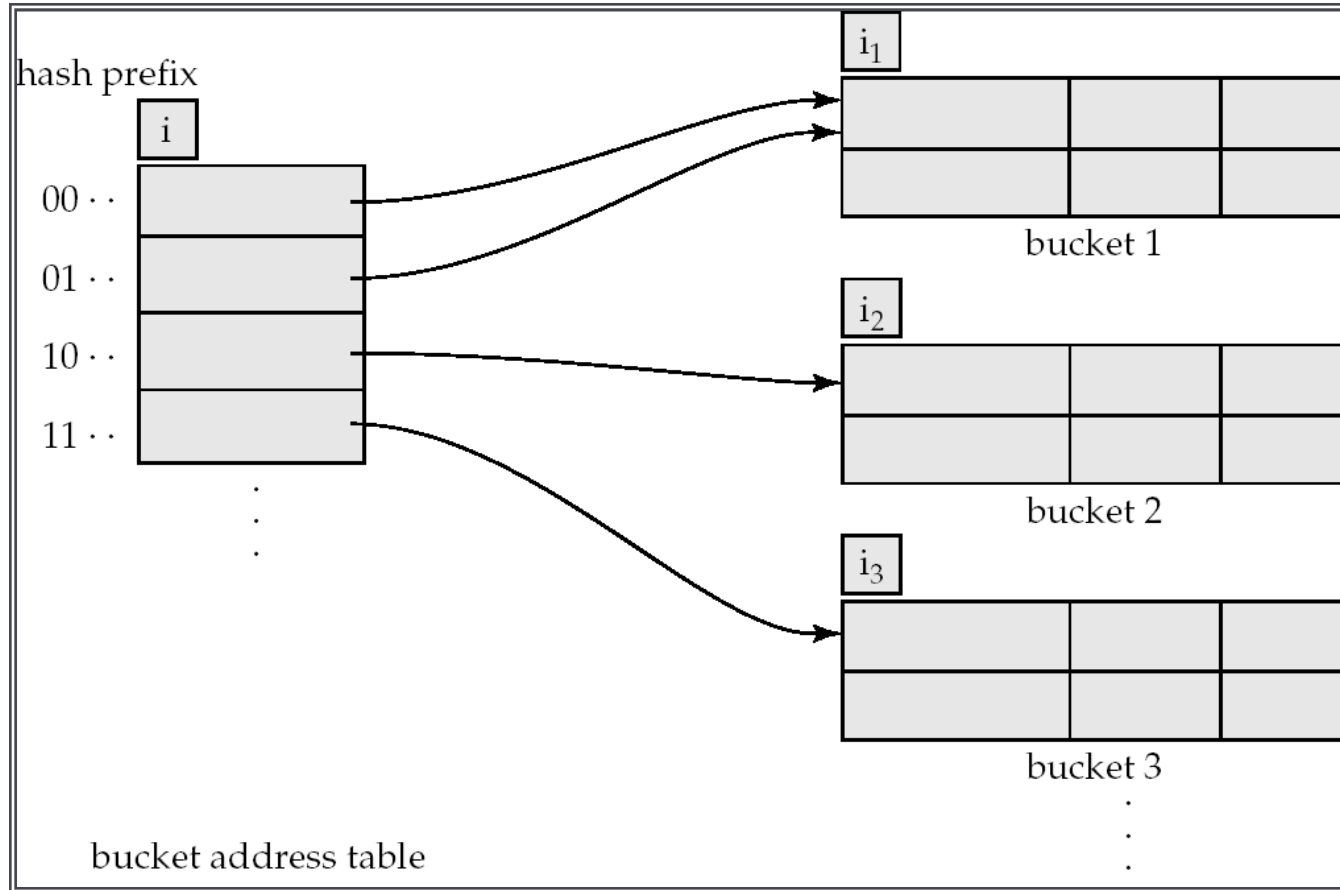
- ▶ Pentru căutări punctuale, în absența coliziunilor, localizarea unei valori necesită citirea unui singur bloc
- ▶ Pentru căutări în interval indecșii hash nu sunt eficienți, deoarece trebuie calculat hashul fiecărei valori posibile

Hash dinamic

- ▶ Funcția h mapează valori ale cheii de căutare la un set fix de adrese de bucketuri.
 - ▶ Dacă fișierul crește apar depășiri ale bucketurilor
 - ▶ Dacă fișierul se micșorează spațiu este alocat inutil
- ▶ Soluții:
 - ▶ Reorganizări periodice cu o nouă funcție hash (costisitoare, necesită întreruperea operațiunilor)
 - ▶ Numărul de bucketuri este modificat dinamic
- ▶ Hash extensibil: funcția hash e modificată dinamic
 - ▶ Generează valori într-o mulțime mare, tipic întregi pe 32 biți
 - ▶ La un anumit moment se utilizează doar un prefix al funcției hash (doar primii i biți) al cărui lungime scade sau crește după caz

Hash extensibil

Structura generală



$$i=2, i_2 = i_3 = i, i_1 = i - 1$$

Hash extensibil

Utilizare

- ▶ Fiecare bucket j stochează o valoare i_j
 - ▶ toate intrările care indică spre bucketul j vor avea aceeași valoare pe primii i_j biți
- ▶ Pentru a localiză bucketul ce conține cheia de căutare K_j :
 - ▶ Se calculează $h(K_j) = X$
 - ▶ Se utilizează primii i biți ai lui X și se urmează pointerul către bucketul potrivit
- ▶ Pentru a insera o înregistrare cu cheia de căutare K_j :
 - ▶ Se localizează bucketul j ca mai sus
 - ▶ Dacă este spațiu în bucket se inserează înregistrarea
 - ▶ Altfel bucketul este divizat și inserarea este reîncercată

Hash extensibil

Divizare bucket la inserare

Pentru a diviza bucketul j la inserarea unei valori K_j :

- ▶ Dacă $i > i_j$
 1. Se alocă un nou bucket z și $i_j = i_z = (i_j + 1)$
 2. Se actualizează a doua jumătate a tabelului de adrese a bucketurilor pentru a indica spre z
 3. Se scot înregistrările din j și sunt reinsertate în j sau z
 4. Se recalculează adresa bucketului pentru K_j și se inserează
- ▶ Dacă $i = i_j$
 1. Dacă se atinge o limită a lui i se utilizează bucketuri de exces
 2. Altfel
 1. Se incrementează i și se dublează dimensiunea tabelului de adrese
 2. Se înlocuiește fiecare intrare în tabel cu două intrări care indică spre același bucket
 3. Se recalculează adresa bucketului pentru K_j și se inserează (acum $i > i_j$)

Hash extensibil

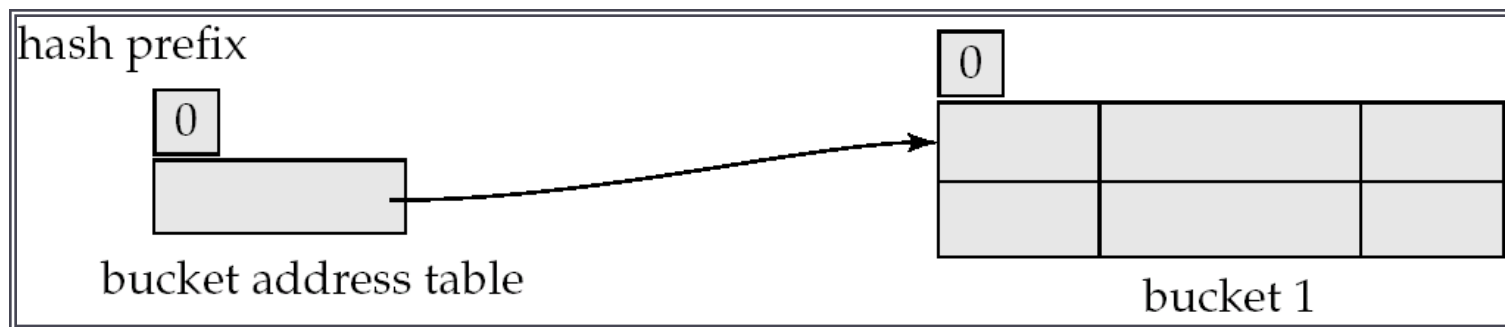
Ștergere

- ▶ **Pentru a șterge o înregistrare**
 - ▶ Se localizează bucketul și se șterge din el
 - ▶ Dacă bucketul devine gol acesta este șters cu modificările necesare în tabela de adrese
 - ▶ Pot fi contopite bucketuri care au aceeași valoare pentru i_j și același prefix $i_j - 1$
 - ▶ Descreșterea dimensiunii tabelului de adrese este posibilă

Hash extensibil

Exemplu*

<i>branch_name</i>	<i>h(branch_name)</i>
Brighton	0010 1101 1111 1011 0010 1100 0011 0000
Downtown	1010 0011 1010 0000 1100 0110 1001 1111
Mianus	1100 0111 1110 1101 1011 1111 0011 1010
Perryridge	1111 0001 0010 0100 1001 0011 0110 1101
Redwood	0011 0101 1010 0110 1100 1001 1110 1011
Round Hill	1101 1000 0011 1111 1001 1100 0000 0001

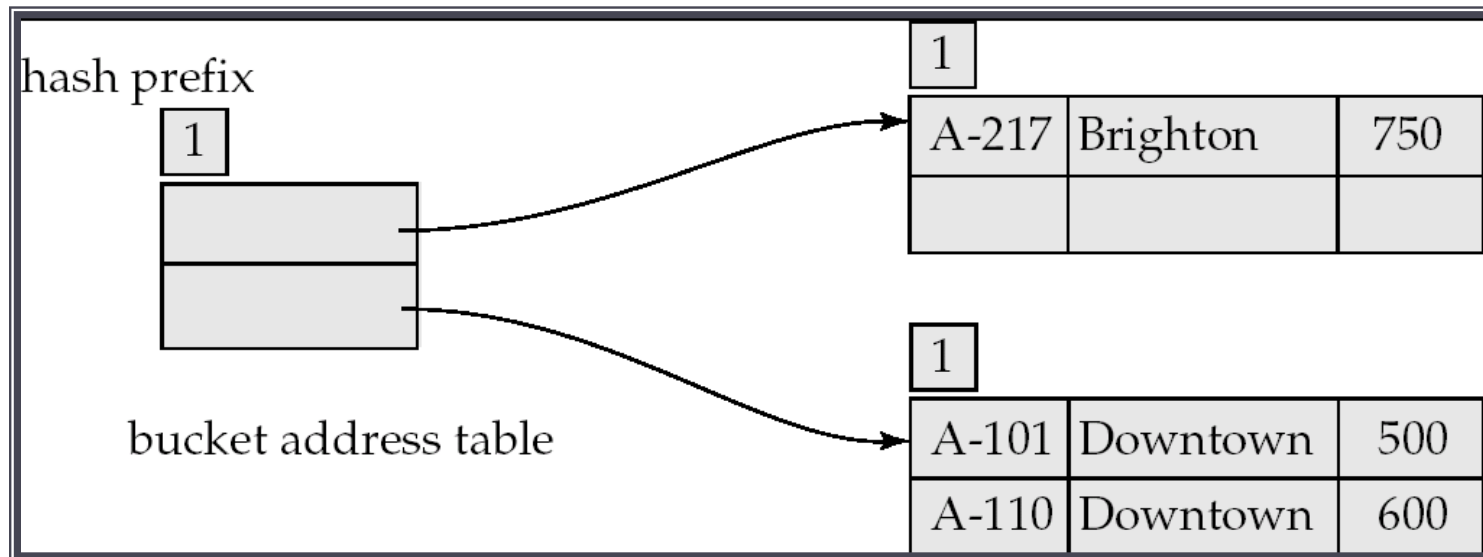


structura hash inițială, dimensiune bucket = 2

Hash extensibil

Exemplu

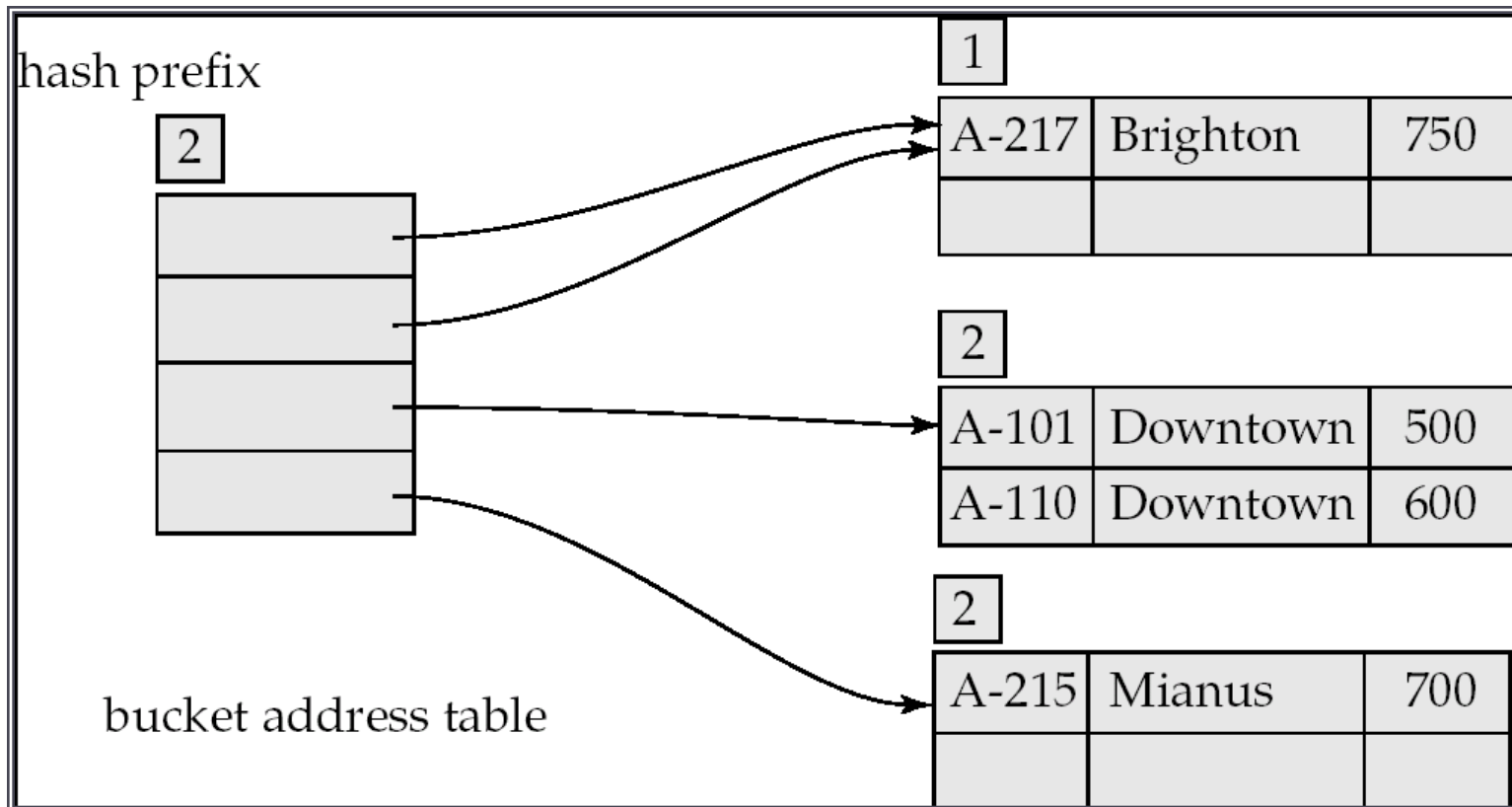
- Structura după inserarea unei înregistrări Brighton și a două înregistrări Downtown



Hash extensibil

Exemplu

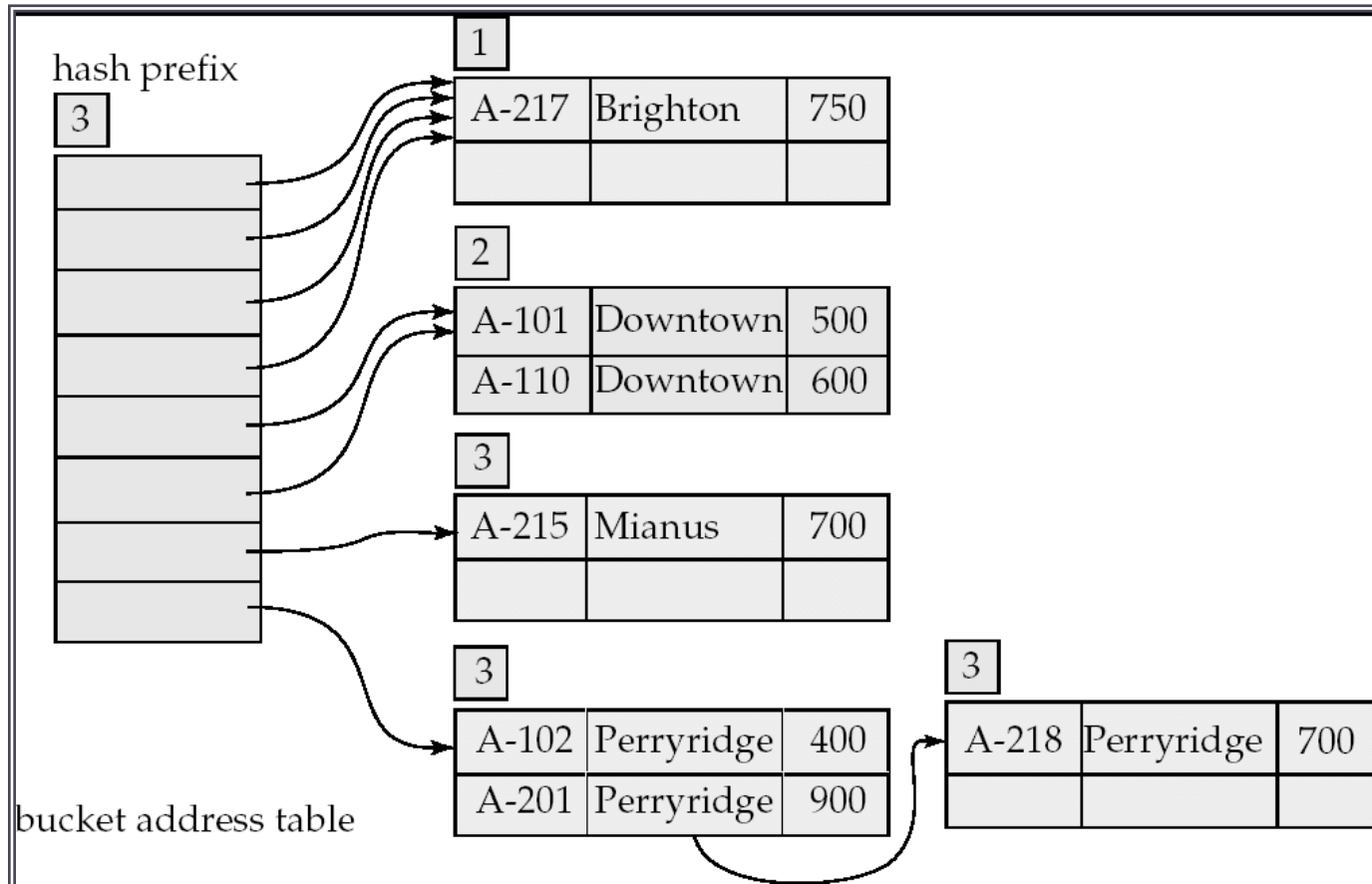
- După inserarea înregistrării Mianus



Hash extensibil

Exemplu

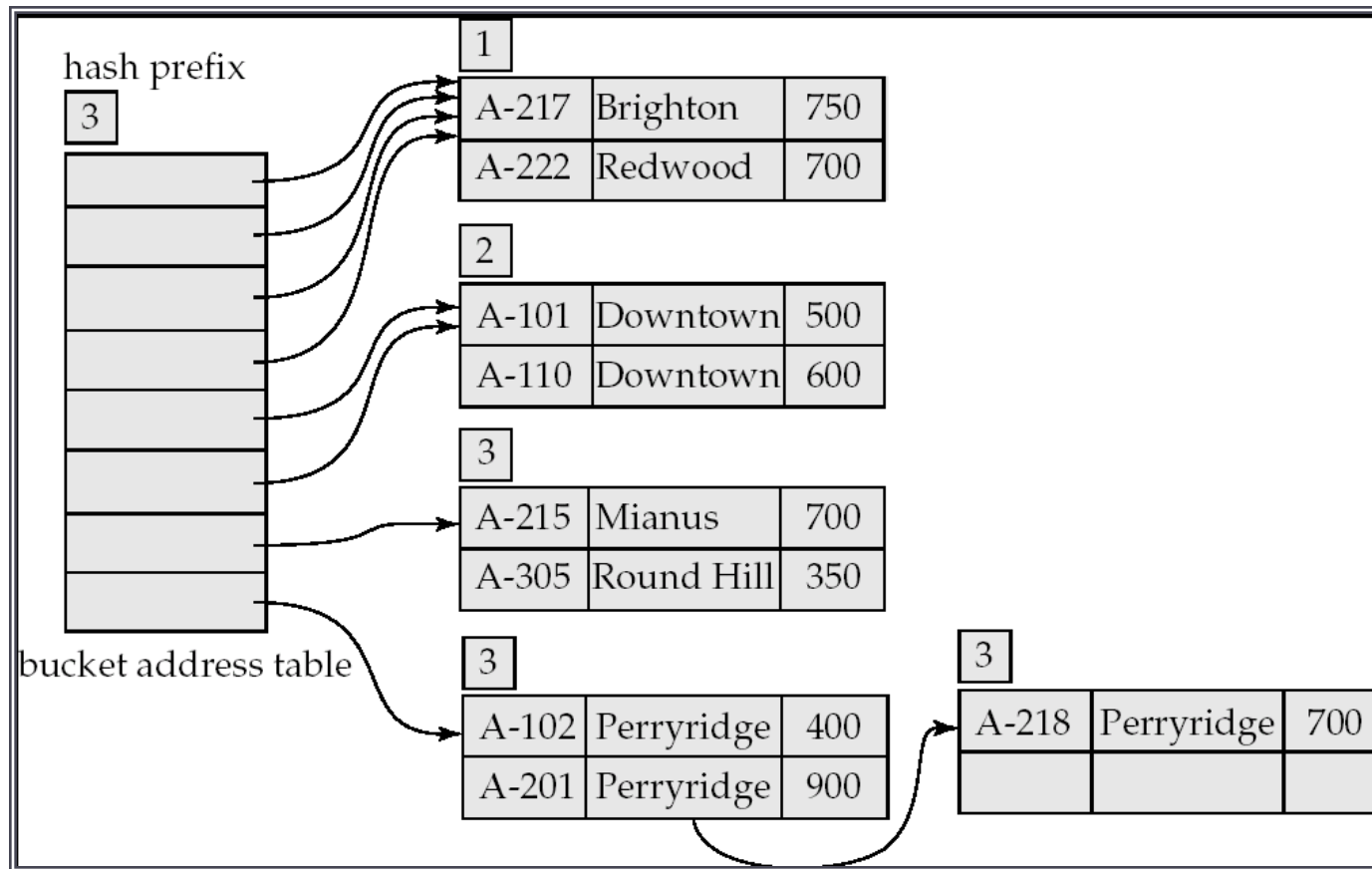
- După inserarea a trei înregistrări Perryridge



Hash extensibil

Exemplu

- După inserarea înregistrărilor Redwood și Round Hill



Hash extensibil

Observații

▶ Beneficii

- ▶ Performanța nu se degradează cu creșterea fișierului
- ▶ Minimizează consumul de memorie

▶ Dezavantaje

- ▶ Tabela de adrese a bucketurilor poate deveni foarte mare
 - ▶ Soluție: utilizarea unui B⁺-arbore pentru a localiza înregistrarea dorită în tabela de adrese
- ▶ Modificarea dimensiunii tabeli de adrese este costisitoare

▶ În funcție de tipul interogării:

- ▶ Hashingul e indicat când se specifică o valoare a cheii de căutare
- ▶ Dacă se lucrează cu intervale de valori e mai rapid indexul ordonat

▶ În practică:

- ▶ Postgres suportă indecșii hash
- ▶ Oracle suportă organizarea statică de tip hash, nu și indecși hash
- ▶ SQLServer suportă numai B⁺-arbori

Indeksi bitmap

Indecși bitmap

- ▶ Proiectați pentru a trata eficient interogările cu mai multe chei de căutare
- ▶ Aplicabili pentru attribute care iau un set redus de valori distincte
- ▶ Tuplele relației sunt considerate a fi numerotate
- ▶ Structura:
 - ▶ Un șir de biți pentru fiecare valoare a atributului
 - ▶ Șirul are lungimea numărului de înregistrări
 - ▶ Valoarea 1 semnifică egalitate cu valoarea căreia îi este asociat bitmapul

nume	prenume	str	loc	judet		
Alexa	Marian	Strada Florilor	Cluj Napoca	Cluj		Arad 0 0 0 0 0 0 1 0
Popescu	Valentin	Strada Unirii	Dej	Cluj		Bihor 0 0 0 0 1 1 0 0
Andrici	Ioana	Bulevardul Republicii	Vaslui	Vaslui		Cluj 1 1 0 0 0 0 0 0
Acatrinei	Marcel		Putna	Suceava		Suceava 0 0 0 1 0 0 0 0
Popescu	Vasile	Bulevardul Independentei	Oradea	Bihor		Vaslui 0 0 1 0 0 0 0 1
Costache	Ioan	Strada Teiului	Nucet	Bihor		
Ungureanu	Daniel	Aleea Amara	Arad	Arad		
Sandu	Maria	Strada Victoriei	Barlad	Vaslui		

Indecși bitmap

Observații

- ▶ Interogările sunt rezolvate utilizând operatori pe biți:

- ▶ Intersecția – AND
- ▶ Reuniunea – OR
- ▶ Complementarierea – NOT

SELECT *

FROM studenti

WHERE judet IN ('Arad', 'Cluj') AND an <> 2010;

- ▶ (Arad OR Cluj) AND NOT(2010)
- ▶ Nu e necesar accesul fișierului
- ▶ Utili când interogarea necesită numărare

- ▶ Implementare eficientă:

- ▶ La ștergere se preferă utilizarea unui bitmap de existență
- ▶ Bitmapurile sunt împachetate în cuvinte (tipul word) de 32 sau 64 biți (operatorul and pe un cuvânt – o singură instrucțiune CPU)

2010	1 1 1 0 0 0 0 0
	-----+
2011	0 0 0 1 1 0 0 0
	-----+
2012	0 0 0 0 0 1 1 1
	-----+
Arad	0 0 0 0 0 0 1 0
	-----+
Bihor	0 0 0 0 1 1 0 0
	-----+
Cluj	1 1 0 0 0 0 0 0
	-----+
Suceava	0 0 0 1 0 0 0 0
	-----+
Vaslui	0 0 1 0 0 0 0 1

Definirea indecsilor in SQL

Definirea indecșilor în SQL

- ▶ Standardul nu reglementează, dar în practică SGBD-urile aderă la aceeași sintaxă

- ▶ Creare:

create index <index-name> **on** <relation-name>
(<attribute-list>)

E.g.: **create index** *b-index* **on** *branch(branch_name)*

- ▶ Ștergere:

drop index <index-name>

- ▶ Majoritatea SGBD-urilor permit specificarea tipului de index
- ▶ Majoritatea SGBD-urilor creează implicit indecși la specificarea constrangerii *unique*
- ▶ Uneori sunt generați indecși la specificarea constrangerilor referențiale

Indexarea în Oracle

- ▶ Oracle suportă B⁺-arbori implicit la crearea indexului cu comanda SQL (in documentatia Oracle apar ca B-arbori dar corespund in teorie B⁺-arborilor)
- ▶ Indecșii sunt suportați pe:
 - ▶ Atribute și liste de atribute
 - ▶ Rezultatul unei funcții peste atribute
- ▶ Indecși bitmap sunt suportați cu declararea
create bitmap index <index-name> **on** <relation-name> (<attribute-list>)
- ▶ Indecși hash nu sunt suportați dar există suport pentru organizarea hash statică

Indexarea în Oracle

Considerente

- ▶ Crearea indecsilor e recomandată după încărcarea datelor – oricând e posibil
- ▶ Indexarea coloanelor potrivite:
 - ▶ Coloanele au valori (în proporție mare) unice
 - ▶ Dacă selecția filtrează un număr mic de tuple dintr-un tabel mare (selectivitate ridicată ~ 15%)
 - ▶ Coloanele sunt utilizate în join
- ▶ Alegerea tipului potrivit:
 - ▶ Coloane cu valori distincte puține -> structura bitmap
 - ▶ Range query (filtrare în interval) -> B+trees
 - ▶ Interogări punctuale frecvente: -> organizare hash a fișierului
 - ▶ Selecție cu condiții pe funcții -> indecsi definiți pe funcții

Bibliografie

- ▶ Capitolul 11 în *Avi Silberschatz Henry F. Korth S. Sudarshan. “Database System Concepts”. McGraw-Hill Science/Engineering/Math; 6 edition (January 27, 2010)*



BAZE DE DATE

Procesarea interogărilor

Mihaela Elena Breabăn

© FII 2015-2016

Cuprins

- ▶ Etapele procesării interogărilor
- ▶ Expresii în algebra relațională
 - ▶ Operatori (revizitat)
 - ▶ Expresii
 - ▶ Echivalența expresiilor
- ▶ Estimarea costului interogării
- ▶ Algoritmi pentru evaluarea operatorilor/expresiilor în algebra relațională

Etapele procesării interogărilor

► Compilarea interogării

► Analiza sintactică

► Parsare

- Arbore de parsare

► Analiza semantică

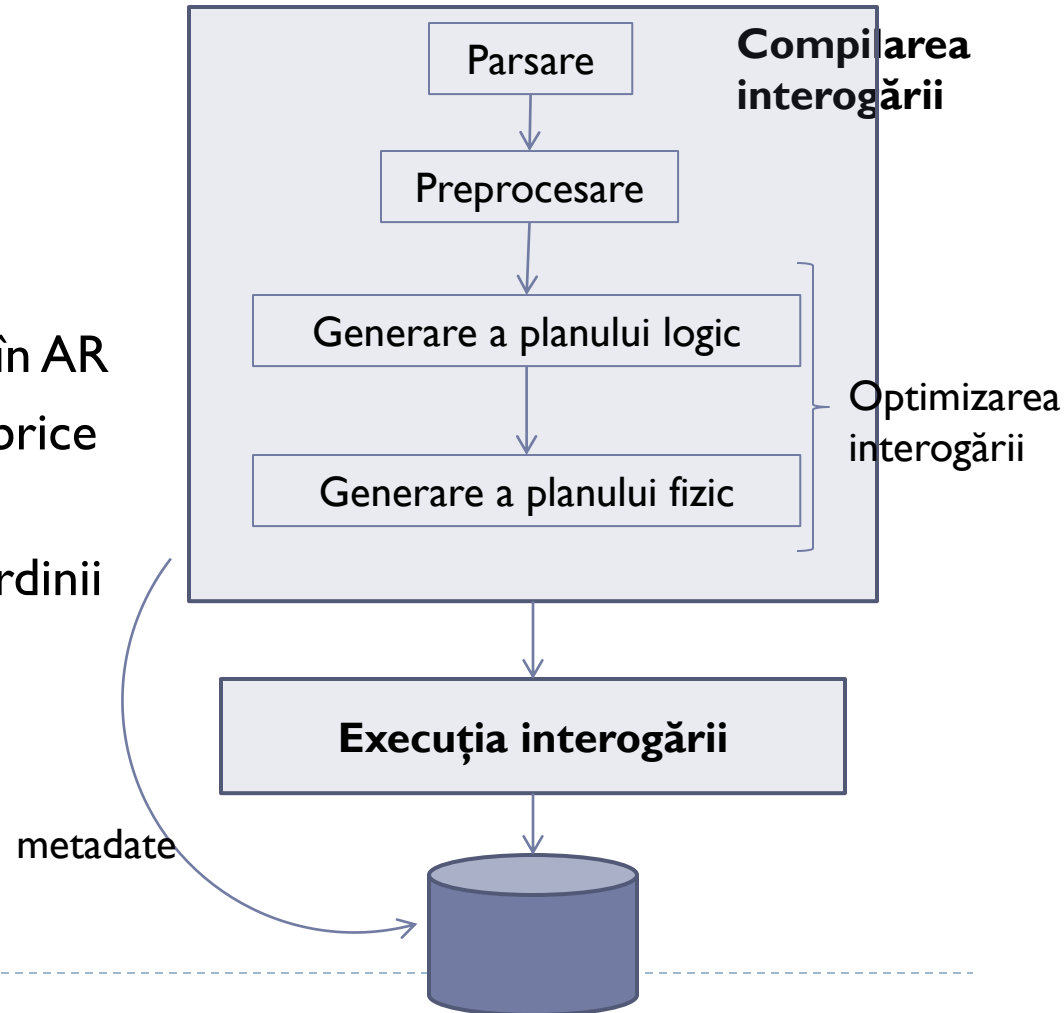
► Preprocesare și rescriere în AR

► Selecția reprezentării algebrice

- Plan logic

► Selecția algoritmilor și a ordinii

- Plan fizic



Analiza sintactică

- ▶ Gramatică independentă de context

$\langle \text{query} \rangle ::= \langle \text{SFW} \rangle \mid (\langle \text{query} \rangle)$

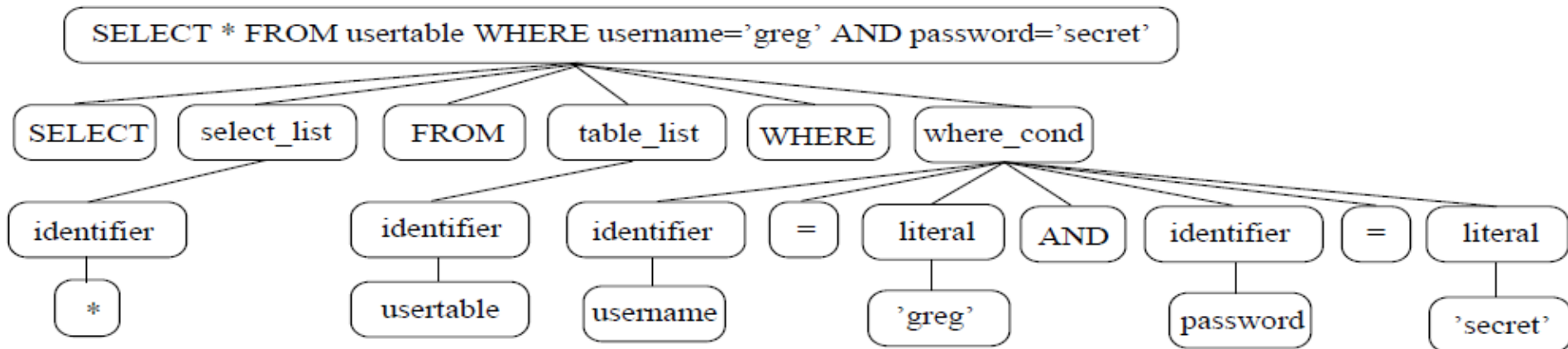
$\langle \text{SFW} \rangle ::= \text{SELECT } \langle \text{select_list} \rangle \text{ FROM } \langle \text{table_list} \rangle \text{ WHERE } \langle \text{where_cond} \rangle$

$\langle \text{select_list} \rangle ::= \langle \text{identifier} \rangle, \langle \text{select_list} \rangle \mid \langle \text{identifier} \rangle$

$\langle \text{table_list} \rangle ::= \langle \text{identifier} \rangle, \langle \text{table_list} \rangle \mid \langle \text{identifier} \rangle$

...

- ▶ Rezultatul parsării: arbore de parsare



- ▶ Gramatica SQL in forma BNF: <http://savage.net.au/SQL/index.html>

Analiza semantică

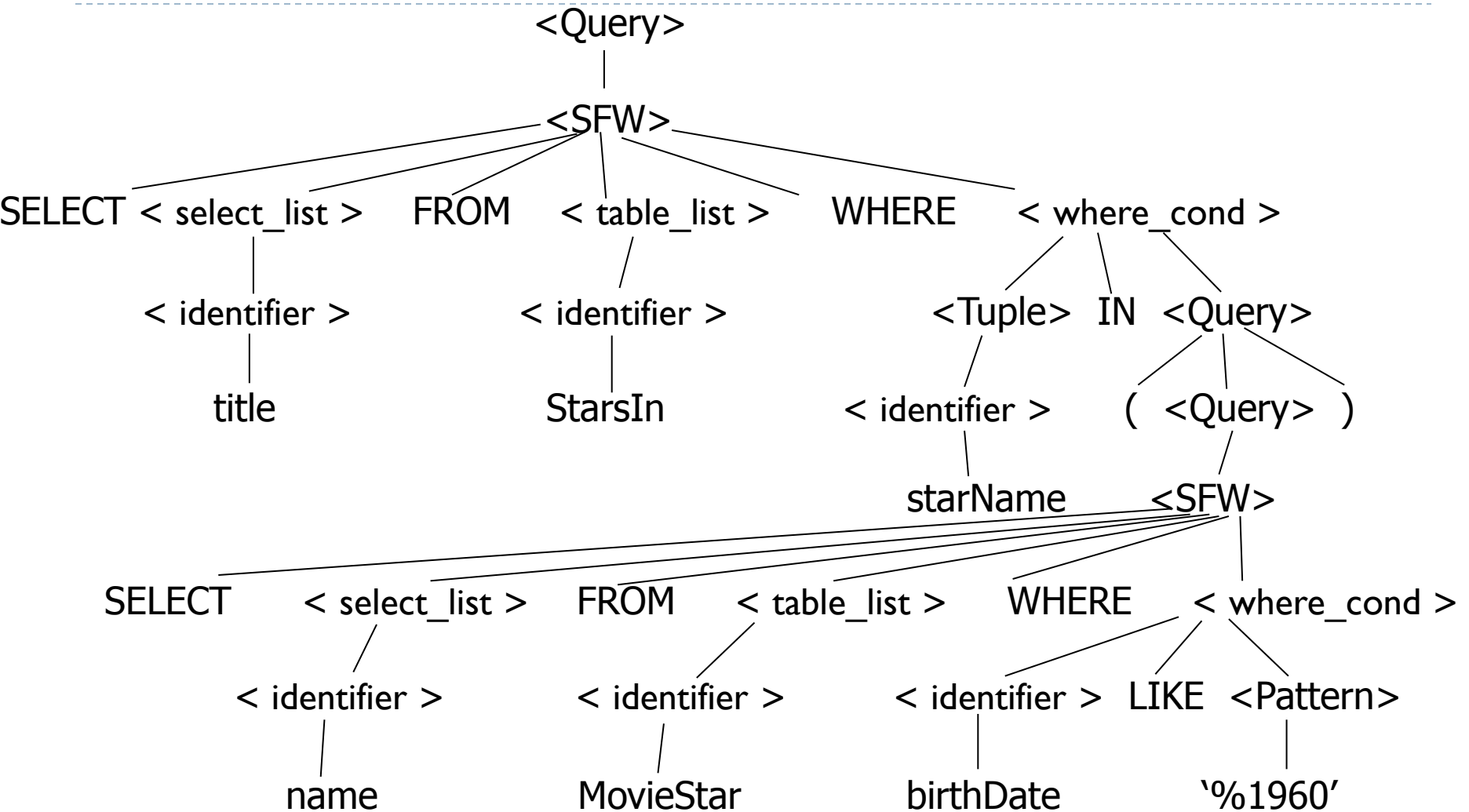
Preprocesare

- ▶ Rescrierea apelurilor la view-uri
- ▶ Verificarea existenței relațiilor
- ▶ Verificarea existenței atributelor și a ambiguității
- ▶ Verificarea tipurilor

Dacă arborele de parsare este valid el este transformat într-o expresie cu operatori din algebra relațională

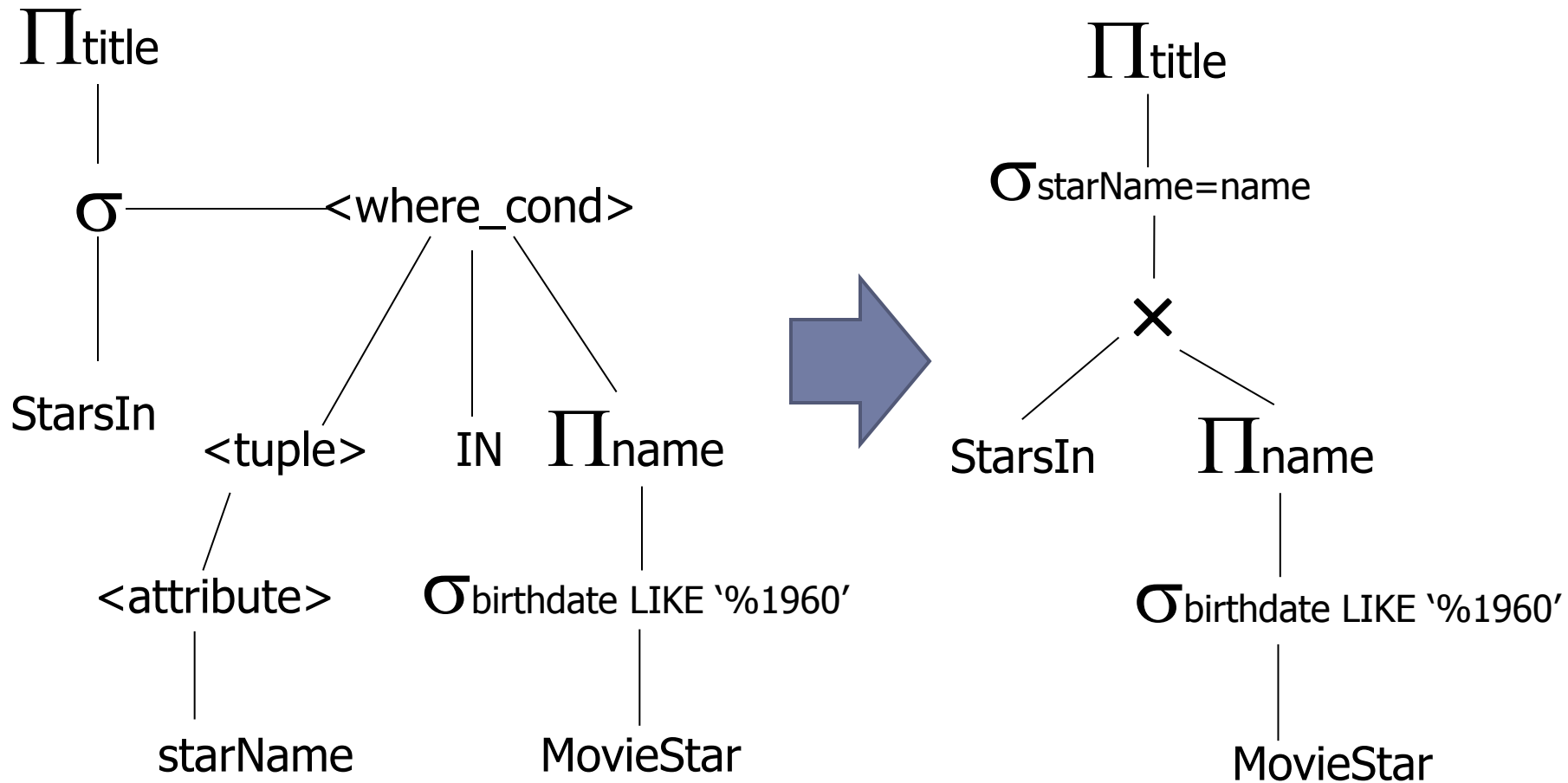
Analiza semantică

Rescriere în AR (1)



Analiza semantică

Rescriere în AR (2)

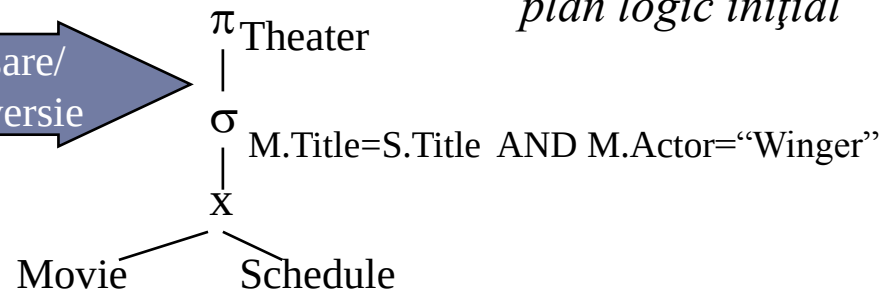


Analiza semantică

Optimizarea planului logic

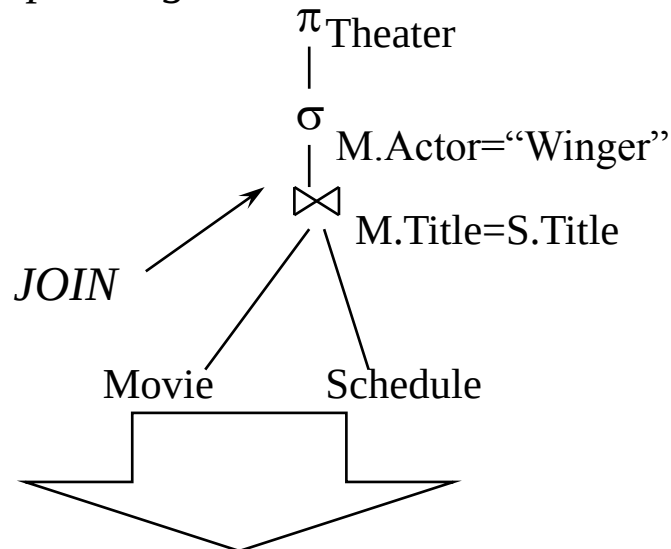
```
SELECT Theater
FROM Movie M, Schedule S
WHERE
  M.Title = S.Title
  AND M.Actor="Winger"
```

Parsare/
Conversie



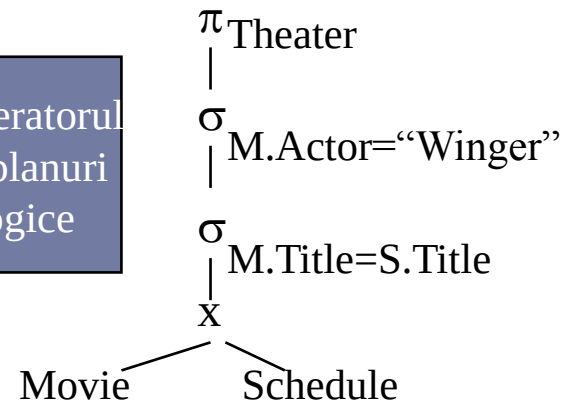
Generatorul de planuri logice
aplică rescrieri algebrice

alt plan logic



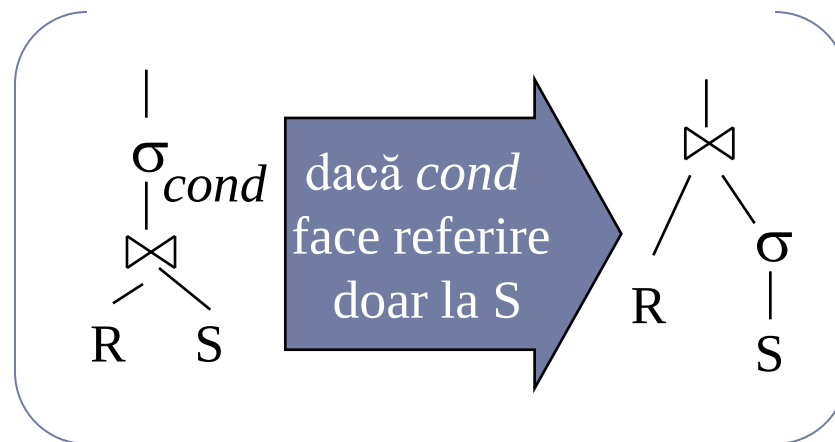
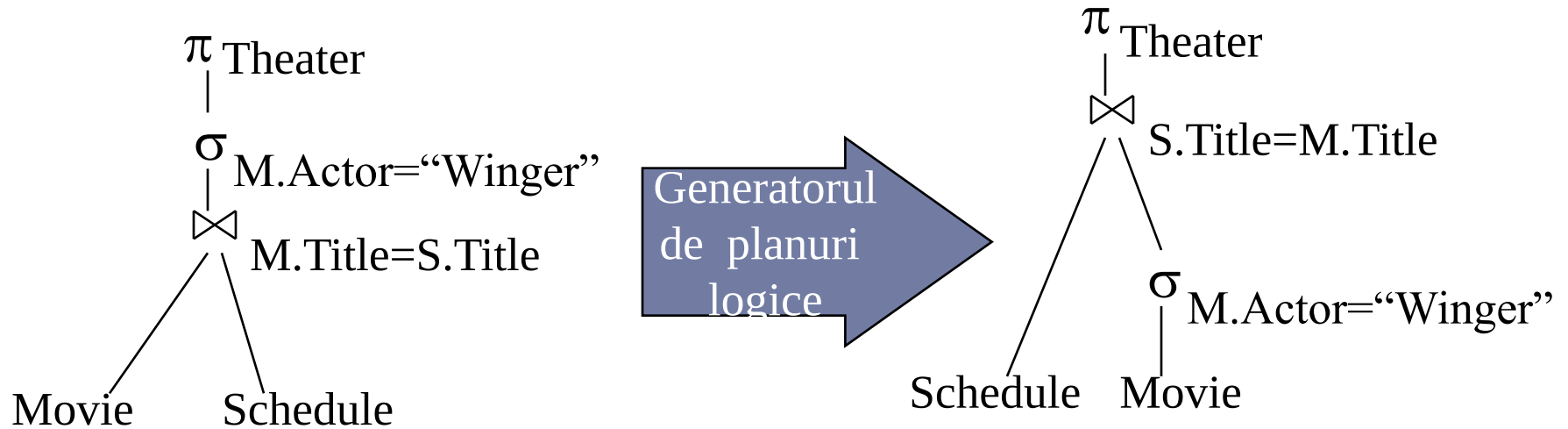
alt plan logic

Generatorul
de planuri
logice



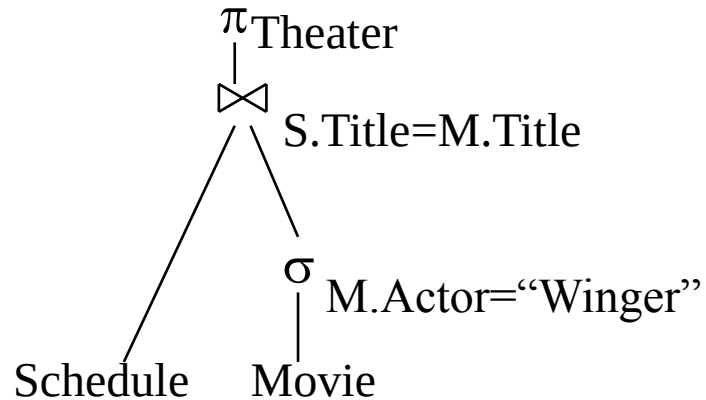
Analiza semantică

Optimizarea planului logic



Analiza semantică

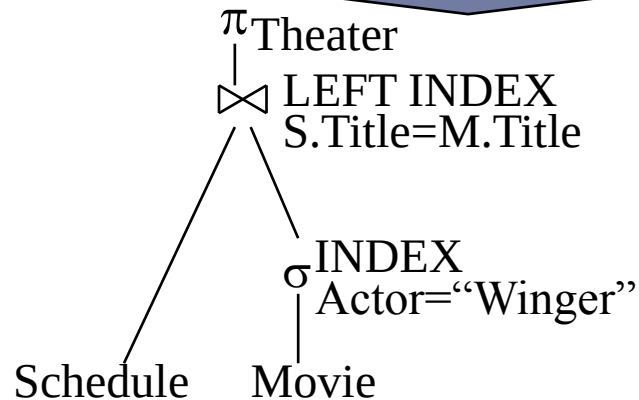
Optimizarea planului fizic



Generatorul de plan fizic alege
primitivele de execuție

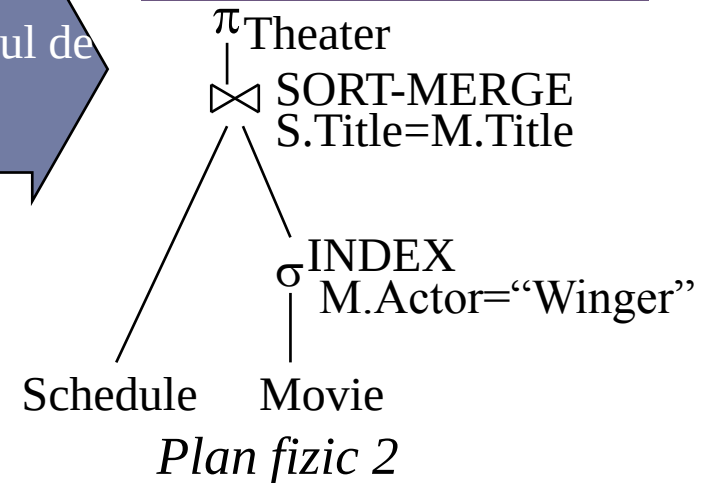
index pe Actor și
Title, tabele nesortate

Plan fizic1



Generatorul de
plan fizic

index pe Actor, tabelul
Schedule sortat pe Title,



Operatori în algebra relațională (revizitat)

- ▶ Șase operatori de bază
 - ▶ Selecția: σ
 - ▶ Proiecția: Π
 - ▶ Reuniunea: $\cup$
 - ▶ Diferența: $-$
 - ▶ Produsul cartezian: $\times$
 - ▶ Redenumirea: ρ
- ▶ Operatorii iau ca intrare una sau două relații și generează o nouă relație

Operatorul de selecție

► Relația r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

► $\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10

Operatorul de proiecție

► Relația r

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

► $\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

A	C
α	1
β	1
β	2

Operatorul reuniune

► Relațiile r și s

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

► $r \cup s$:

A	B
α	1
α	2
β	1
β	3

Operatorul diferență

► Relațiile r și s

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

► $r-s$

A	B
α	1
β	1

Produsul cartezian

► Relațiile r și s

A	B
-----	-----

α	1
β	2

r

C	D	E
-----	-----	-----

α	10	a
β	10	a
β	20	b
γ	10	b

s

► $r \times s$

A	B	C	D	E
-----	-----	-----	-----	-----

α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Operatorul de redenumire

- ▶ $\rho_X(E)$ - returnează expresia E sub numele X
- ▶ Dacă o expresie E în algebra relațională are aritate n atunci

$$\rho_{X(A_1, A_2, \dots, A_n)}(E)$$

returnează rezultatul expresiei E sub numele X și attributele redenumite în $A_1, A_2, \dots, A_n$.

Compunerea operatorilor

► $\sigma_{A=C}(r \times s)$

1. $r \times s$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

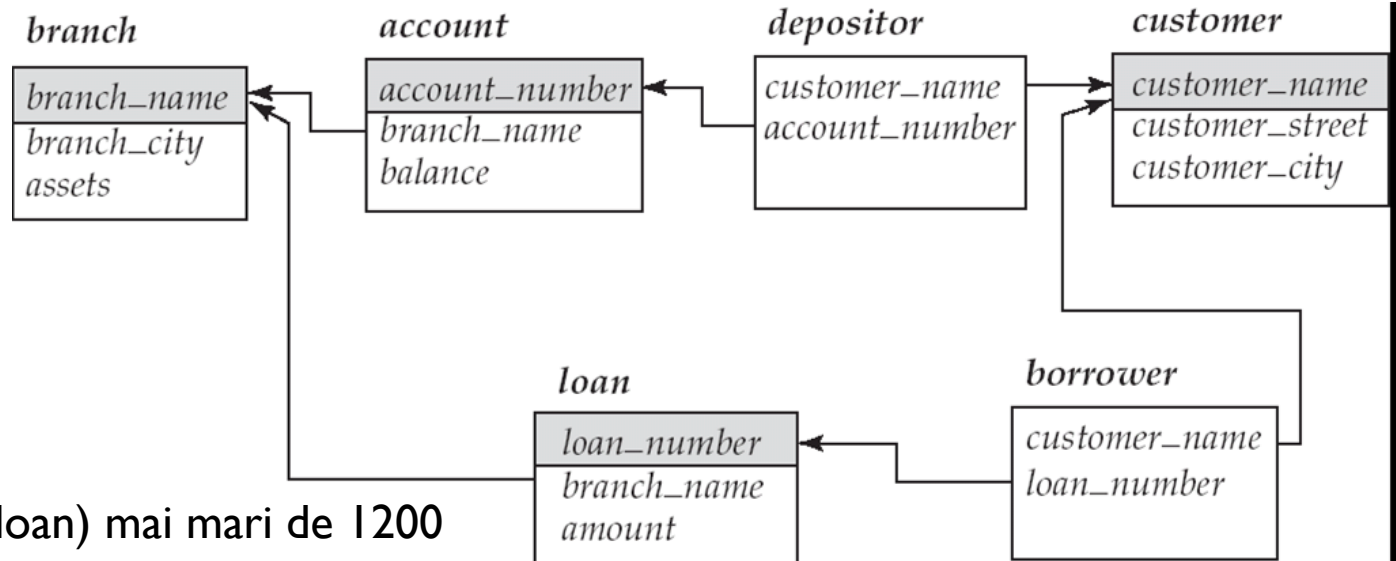
2. $\sigma_{A=C}(r \times s)$

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b

Expresii în algebra relațională

- ▶ Cea mai simplă expresie este o relație în baza de date
- ▶ Fie E_1 și E_2 expresii în algebra relațională; următoarele sunt expresii în algebra relațională:
 - ▶ $E_1 \cup E_2$
 - ▶ $E_1 - E_2$
 - ▶ $E_1 \times E_2$
 - ▶ $\sigma_P(E_1)$, P este un predicat peste atribute din E_1
 - ▶ $\Pi_S(E_1)$, S este o listă de atribute din E_1
 - ▶ $\rho_x(E_1)$, x este noul nume pentru rezultatul lui E_1

Exprimarea interogărilor în algebra relațională



- ▶ Împumuturile (loan) mai mari de 1200

$$\sigma_{amount > 1200} (loan)$$

- ▶ Numărul împrumutului (loan_number) pentru împrumuturi mai mari de 1200

$$\Pi_{loan_number} (\sigma_{amount > 1200} (loan))$$

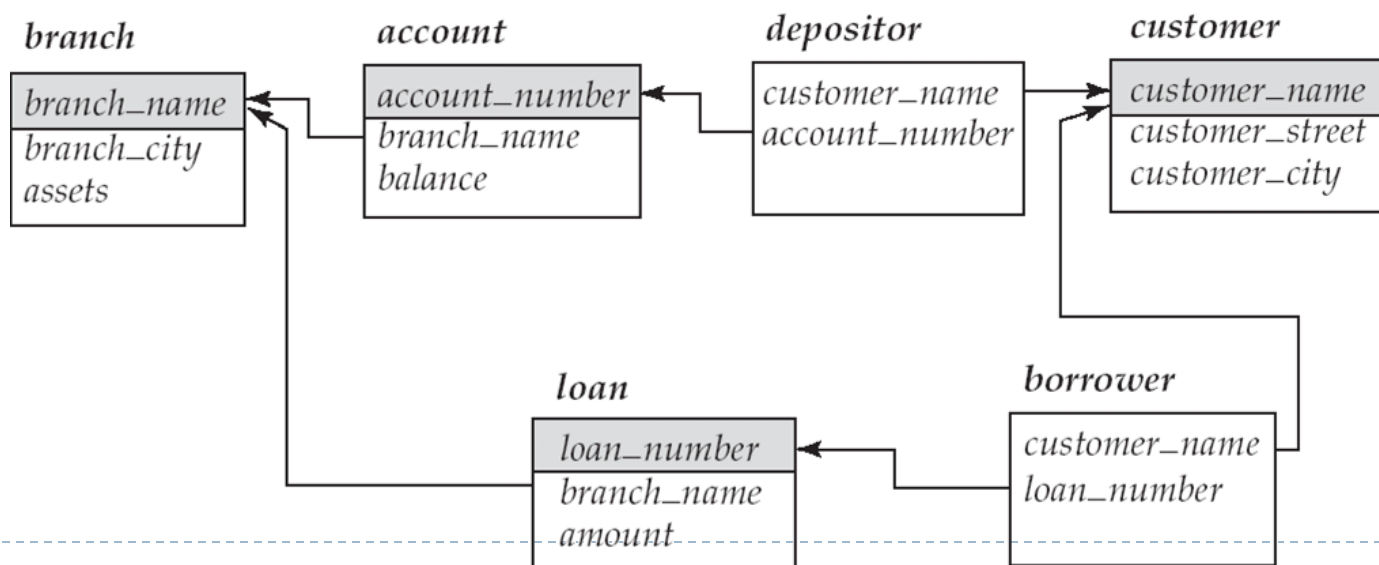
- ▶ Numele clienților care au un împrumut, un depozit sau ambele la bancă

$$\Pi_{customer_name} (borrower) \cup \Pi_{customer_name} (depositor)$$

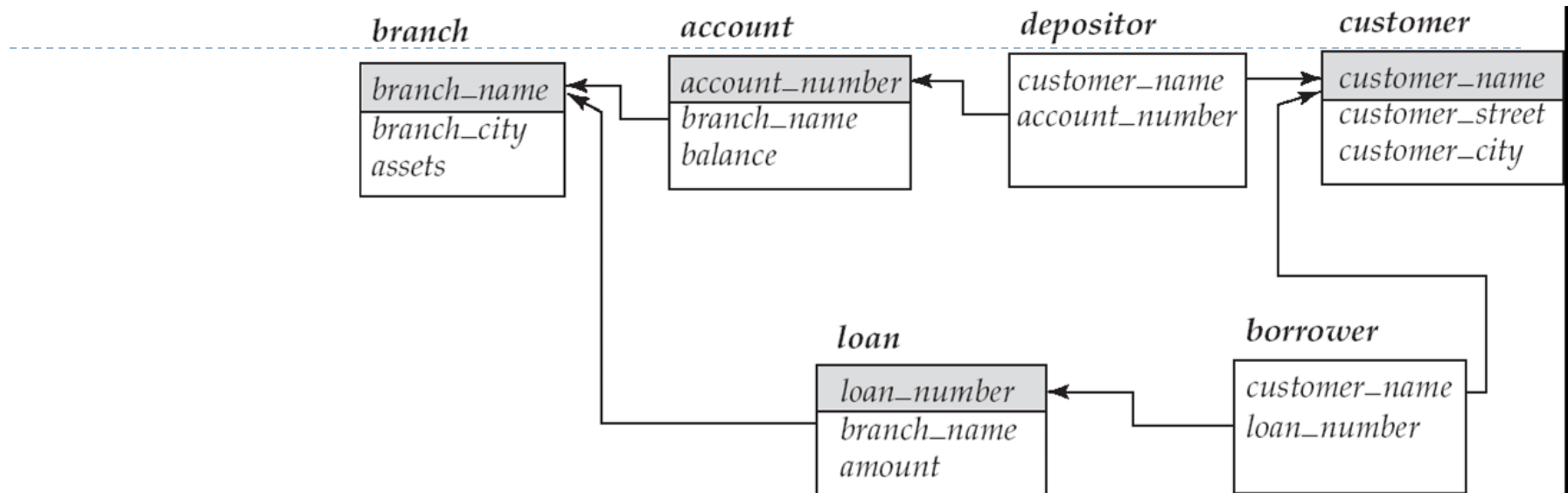
Interogări

► Numele tuturor clienților care au un împrumut la filiala Perryridge

- $\Pi_{\text{customer_name}} (\sigma_{\text{branch_name} = \text{"Perryridge"}} (\sigma_{\text{borrower.loan_number} = \text{loan.loan_number}} (\text{borrower} \times \text{loan})))$
- $\Pi_{\text{customer_name}} (\sigma_{\text{loan.loan_number} = \text{borrower.loan_number}} (\sigma_{\text{branch_name} = \text{"Perryridge"}} (\text{loan})) \times \text{borrower}))$



Interogări



- Numele tuturor clienților care au un împrumut la filiala Perryridge dar nu au un depozit la nici o filială a băncii

$\Pi_{customer_name} (\sigma_{branch_name = "Perryridge"}$

$(\sigma_{borrower.loan_number = loan.loan_number}(borrower \times loan))) -$
 $\Pi_{customer_name}(depositor)$

Operatori adiționali

- ▶ Intersecția pe mulțimi
 - ▶ Joinul natural
 - ▶ Agregarea
 - ▶ Joinul extern
 - ▶ Teta-joinul
-
- ▶ Toți cu excepția agregării pot fi exprimați utilizând operatori de bază

Intersecția pe mulțimi

► Relațiile r și s

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

► $r \cap s$

A	B
α	2

Joinul natural

► Relațiile r și s

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

► $r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

► $\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$

Agregare

Exemplu

- ▶ Cea mai mare balanță din tabela account

account

<i>account_number</i>
<i>branch_name</i>
<i>balance</i>

$$\Pi_{balance}(account) - \Pi_{account.balance}$$

$$(\sigma_{account.balance < d.balance} (account \times \rho_d(account)))$$

Funcții de agregare și operatori

- ▶ Funcții de agregare:

- ▶ **avg**
- ▶ **min**
- ▶ **max**
- ▶ **sum**
- ▶ **count**
- ▶ **var**

- ▶ Operatorul de agregare în algebra relațională

$$G_1, G_2, \dots, G_n \mathcal{G}_{F_1(A_1), F_2(A_2), \dots, F_n(A_n)}(E)$$

- ▶ E – expresie în algebra relațională
- ▶ $G_1, G_2, \dots, G_n$ o listă de attribute de grupare (poate fi goală)
- ▶ Fiecare F_i este o funcție de agregare
- ▶ Fiecare A_i este un atribut

Agregare

Exemplu

- ▶ relația r

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

- ▶ $g_{\text{sum}(c)}(r)$

sum(c)
27

- ▶ Care operații de agregare nu pot fi exprimate pe baza celorlalți operatori relaționali?

Join extern

relația loan

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

relația borrower

<i>customer_name</i>	<i>loan_number</i>
Jones	L-170
Smith	L-230
Hayes	L-155

► *loan* ⋈ *borrower* (join natural)

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith

► *loan* ⋈_l *borrower* (join extern stânga)

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	null

Join extern

➤ Join extern dreapta

loan ⋈_▷ *borrower*

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	<i>null</i>	<i>null</i>	Hayes

➤ Join extern plin

loan ⋈_◁ *borrower*

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	<i>null</i>
L-155	<i>null</i>	<i>null</i>	Hayes

Exemple interogări

- ▶ Numele clienților care au un împrumut și un depozit la bancă

$$\Pi_{customer_name} (borrower) \cap \Pi_{customer_name} (depositor)$$

- ▶ Numele clienților care au un împrumut la bancă și suma împrumutată

$$\Pi_{customer_name, loan_number, amount} (borrower \bowtie loan)$$

- ▶ Clienții care au depozite de la măcar cele două filiale Downtown și Uptown

$$\Pi_{customer_name} (\sigma_{branch_name = \text{“Downtown”}} (depositor \bowtie account)) \cap \\ \Pi_{customer_name} (\sigma_{branch_name = \text{“Uptown”}} (depositor \bowtie account))$$

Echivalența expresiilor

- ▶ Două expresii în algebra relațională sunt echivalente dacă acestea generează același set de tuple pe orice instanță a bazei de date
 - ▶ ordinea tuplelor e irelevantă
- ▶ Obs: SQL lucrează cu multiseturi
 - ▶ în versiunea multiset a algebrei relaționale echivalența se verifică relativ la multiseturi de tuple

Reguli de echivalență

1. selecția pe bază de conjuncții e echivalentă cu o secvență de selecții

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. operațiile de selecție sunt comutative

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. într-un șir de proiecții consecutive doar ultima efectuată e necesară

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

4. selecțiile pot fi combinate cu produsul cartezian și teta joinurile

a. $\sigma_{\theta}(E_1 \bowtie E_2) = E_1 \bowtie_{\theta} E_2$

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

Reguli de echivalență

5. operațiile de tetra-join și de join natural sunt comutative

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6. a) Operațiile de join natural sunt asociative

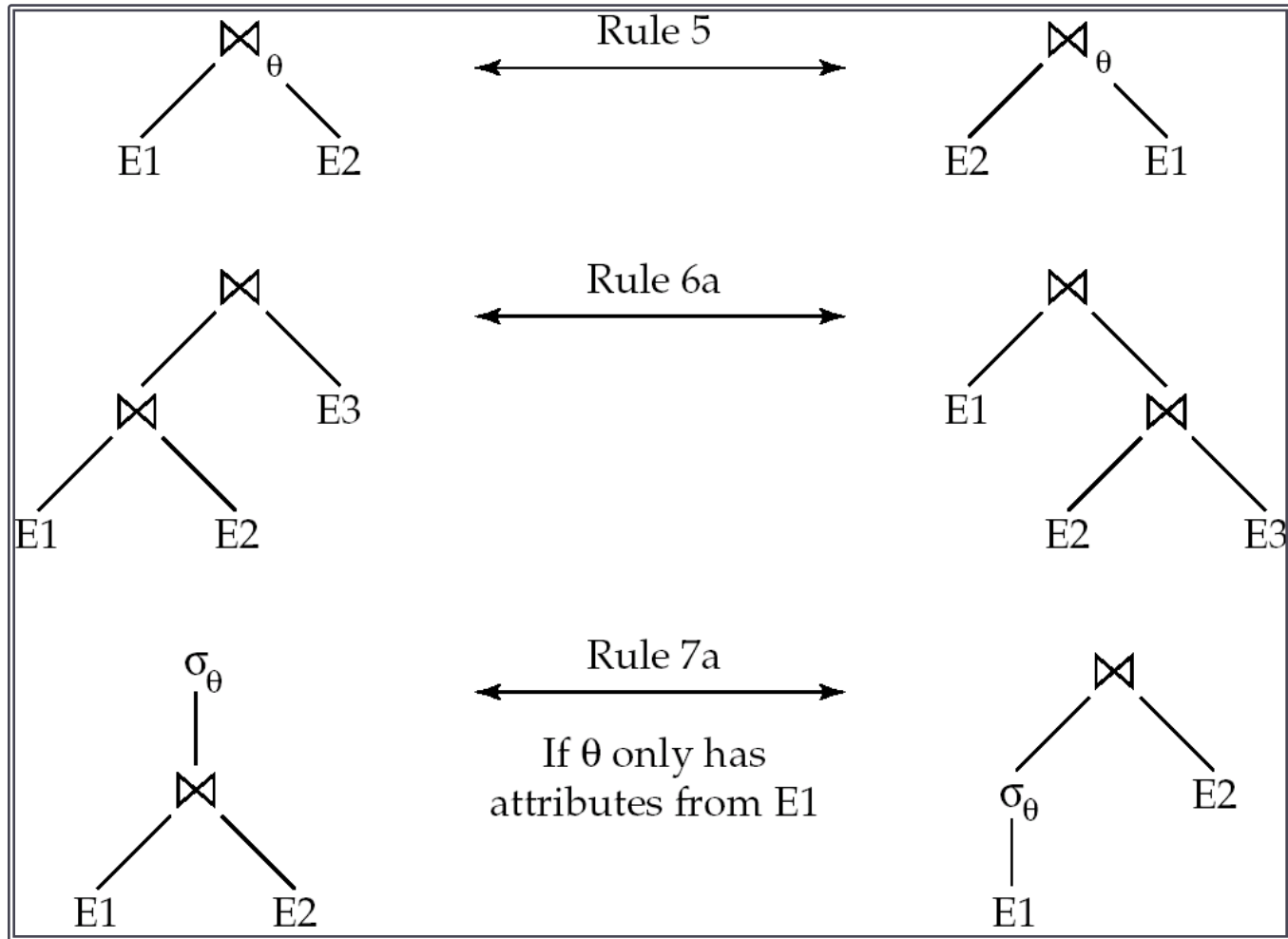
$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- b) Operațiile de tetra-join sunt asociative astfel:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

unde θ_2 implică attribute doar din E_2 și E_3

Reguli de echivalență



Reguli de echivalență

7. Distribuția selecției asupra operatorului de teta-join

- a) când θ_0 implică atribute doar din una dintre expresiile (E_1) din join:

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- b) când θ_1 implică numai atribute din E_1 și θ_2 implică numai atribute din E_2 :

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

Reguli de echivalență

8. Distribuția proiecției asupra teta-joinului

a) dacă θ implică numai attribute din $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

b) Fie joinul $E_1 \bowtie_{\theta} E_2$

Fie L_1 și L_2 mulțimi de attribute din E_1 și respectiv E_2

Fie L_3 attribute din E_1 care sunt implicate în condiția de join θ , dar nu sunt în $L_1 \cup L_2$,

Fie L_4 attribute din E_2 care sunt implicate în condiția de join θ , dar nu sunt în $L_1 \cup L_2$

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

Reguli de echivalență

9. Operațiile de reuniune și intersecție pe mulțimi sunt comutative

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

10. Reuniunea și intersecția pe mulțimi sunt asociative

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. Selecția se distribuie peste $\cup$, $\cap$ și $-$.

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta(E_2)$$

similar pentru $\cup$ și $\cap$ în locul $-$

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta(E_1) - E_2$$

similar pentru $\cap$ în locul $-$, dar nu pentru $\cup$

12. Proiecția se distribuie peste reuniune

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$



Optimizări

Împingerea selecțiilor

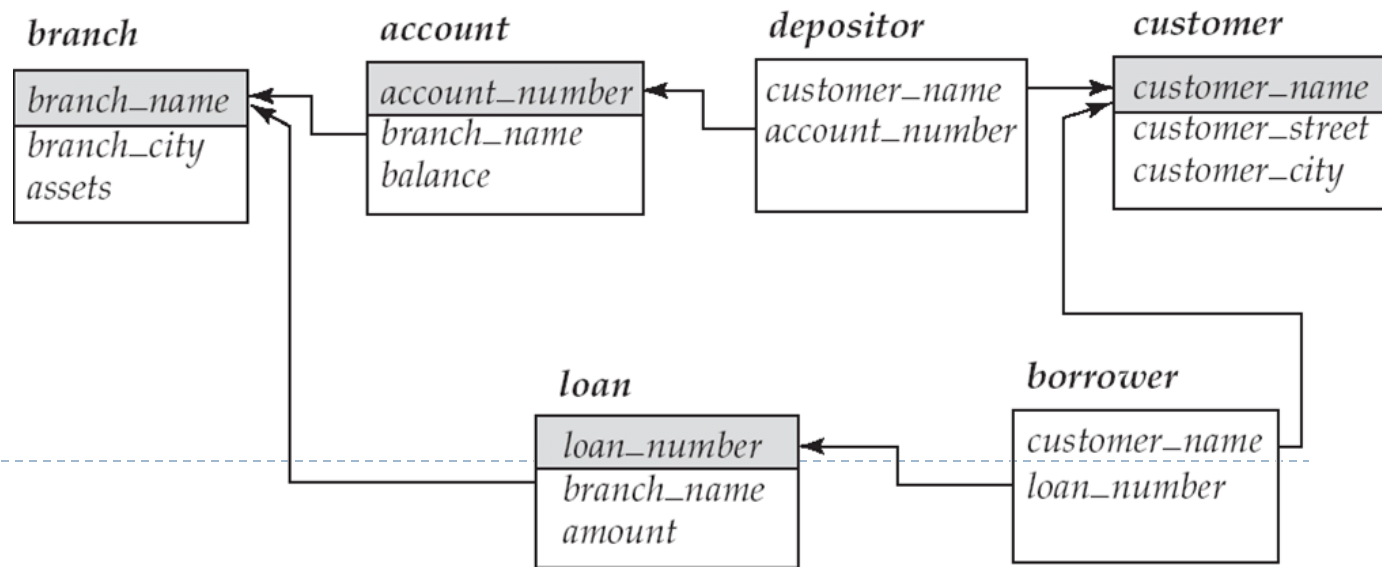
- Numele clienților care au un cont la o filială din Brooklyn

$\Pi_{customer_name}(\sigma_{branch_city = \text{"Brooklyn"}}(branch \bowtie (account \bowtie depositor)))$

- Pe baza regulii 7a

$\Pi_{customer_name}((\sigma_{branch_city = \text{"Brooklyn"}}(branch)) \bowtie (account \bowtie depositor))$

- Realizarea selecției în primele etape reduce dimensiunea relației care participă în join



Optimizări

Împingerea selecțiilor

- ▶ Numele clienților cu un cont la o filială din Brooklyn care are balanța peste 1000

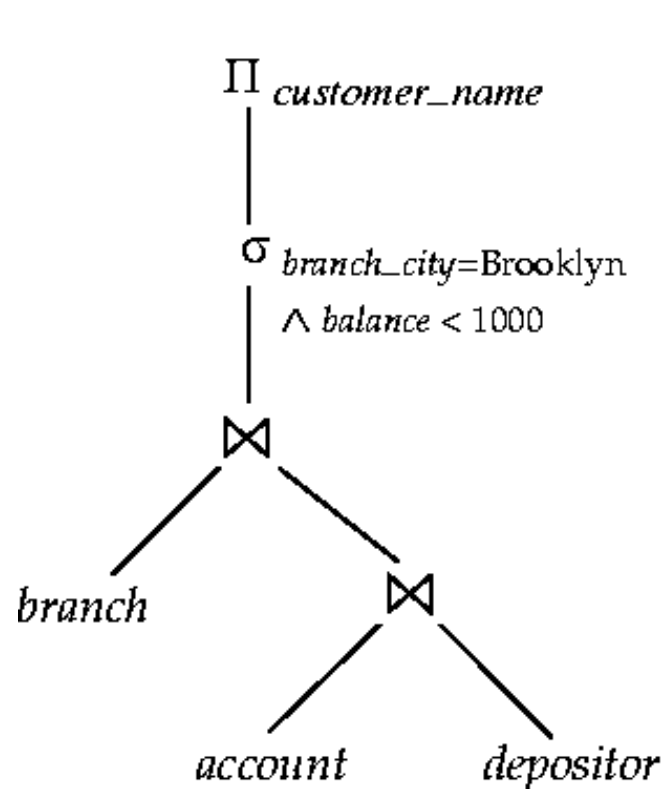
$$\Pi_{customer_name}(\sigma_{branch_city = \text{“Brooklyn”} \wedge balance > 1000} (branch \bowtie (account \bowtie depositor)))$$

- ▶ Regula 6a (asociativitatea la join)

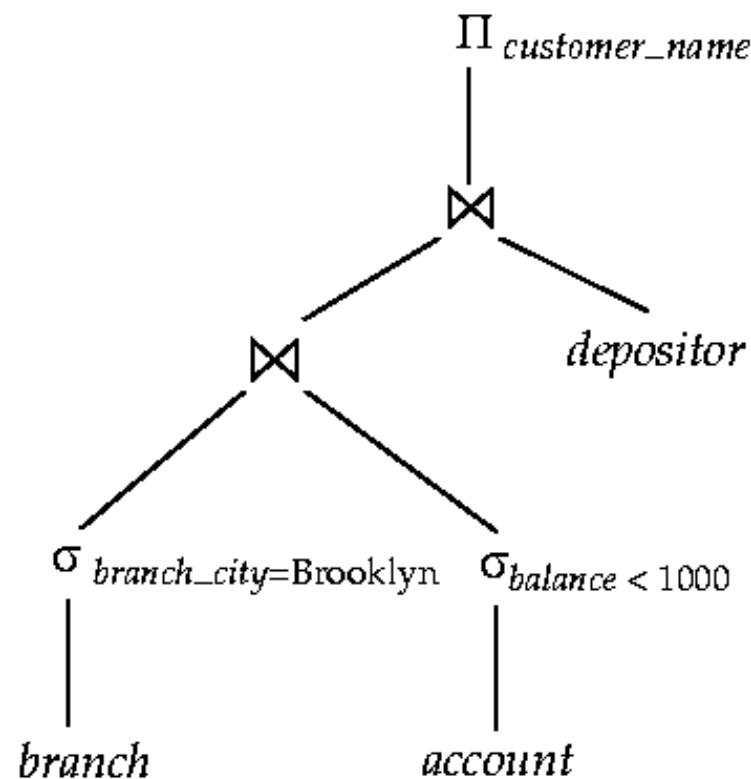
$$\Pi_{customer_name}((\sigma_{branch_city = \text{“Brooklyn”} \wedge balance > 1000} (branch \bowtie account)) \bowtie depositor)$$

- ▶ A doua formă furnizează oportunitatea de a efectua selecția devreme

$$\sigma_{branch_city = \text{“Brooklyn”}} (branch) \bowtie \sigma_{balance > 1000} (account)$$



(a) Initial expression tree



(b) Tree after multiple transformations

Optimizări

Împingerea proiecțiilor

$$\Pi_{customer_name}((\sigma_{branch_city = \text{“Brooklyn”}} (branch) \bowtie account) \bowtie depositor)$$

- ▶ Eliminarea atributelor care nu sunt necesare din rezultatele intermediare

$$\Pi_{customer_name} ((\Pi_{account_number} (\sigma_{branch_city = \text{“Brooklyn”}} (branch) \bowtie account) \bowtie depositor))$$

- ▶ Realizarea devreme a proiecției reduce dimensiunea relațiilor din join

Optimizări

Ordonarea joinurilor

- ▶ Pentru orice relații r_1, r_2 , și r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

- ▶ Dacă $r_2 \bowtie r_3$ are dimensiuni mari și $r_1 \bowtie r_2$ e de dimensiuni mai mici, alegem

$$(r_1 \bowtie r_2) \bowtie r_3$$

- ▶ Exemplu

$$\Pi_{customer_name} ((\sigma_{branch_city = \text{“Brooklyn”}}(branch)) \bowtie (account \bowtie depositor))$$

Numai un mic procent din clienți au conturi în filiale din Brooklyn deci e mai bine să se execute mai întâi

$$\sigma_{branch_city = \text{“Brooklyn”}}(branch) \bowtie account$$

- ▶ Pentru n relații există $(2(n-1))!/(n-1)!$ ordonări diferite pentru join.

- ▶ $n = 7 \rightarrow 665280$, $n = 10 \rightarrow 176$ miliarde!

- ▶ Pentru a reduce numărul de ordonări supuse evaluării se utilizează programarea dinamică

Estimarea costurilor

- ▶ l_r : dimensiunea unui tuplu din r .
- ▶ n_r : numărul de tuple în relația r .
- ▶ b_r : numărul de blocuri conținând tuple din r .
- ▶ f_r : *factorul de bloc* al lui r — nr. de tuple din r ce intră într-un bloc
- ▶ Dacă tuplele lui r sunt stocate împreună într-un fișier, atunci:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

- ▶ $V(A, r)$: numărul de valori distincte care apar în r pentru atributul A ; e echivalent cu dimensiunea proiecției $\Pi_A(r)$ (pe seturi).

Estimarea dimensiunii selecției

- ▶ $\sigma_{A=v}(r)$

- ▶ $n_r / V(A,r)$: numărul de înregistrări ce satisfac selecția
- ▶ pentru atribut cheie: 1

- ▶ $\sigma_{A \leq v}(r)$ (cazul $\sigma_{A \geq v}(r)$ este simetric)

- ▶ dacă sunt disponibile $\min(A,r)$ și $\max(A,r)$
 - ▶ 0 dacă $v < \min(A,r)$
 - ▶ $n_r \cdot \frac{v - \min(A,r)}{\max(A,r) - \min(A,r)}$ altfel
- ▶ dacă sunt disponibile histograme se poate rafina estimarea anterioară
- ▶ în lipsa oricărei informații statistice dimensiunea se consideră a fi $n_r / 2$.

Estimarea dimensiunii selecțiilor complexe

- ▶ Selectivitatea unei condiții θ_i este probabilitatea ca un tuplu în relația r să satisfacă θ_i
 - ▶ dacă numărul de tuple ce satisfac θ_i este s_i , *selectivitatea* e s_i / n_r
- ▶ Conjuncția (în ipoteza independenței)

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r): \quad n_r * \frac{s_1 * s_2 * \dots * s_n}{n_r^n}$$

- ▶ Disjuncția

$$\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r): \quad n_r * \left(1 - \left(1 - \frac{s_1}{n_r} \right) * \left(1 - \frac{s_2}{n_r} \right) * \dots * \left(1 - \frac{s_n}{n_r} \right) \right)$$

- ▶ Negația

$$\sigma_{\neg \theta}(r): \quad n_r - \text{size}(\sigma_{\theta}(r))$$

Estimarea dimensiunii joinului

- ▶ pentru produsul cartezian $r \times s$: $n_r * n_s$ tuple, fiecare tuplu ocupă $s_r + s_s$ octeți
- ▶ pentru $r \bowtie s$
 - ▶ $R \cap S = \emptyset$: $n_r * n_s$
 - ▶ $R \cap S$ este o (super)cheie pentru R: $\leq n_s$
 - ▶ $R \cap S = \{A\}$ nu e cheie pentru R sau S: $\frac{n_r * n_s}{V(A, s)}$ sau $\frac{n_r * n_s}{V(A, r)}$
 - ▶ minimul este considerat de acuratețe mai mare
 - ▶ dacă sunt disponibile histograme se calculează formulele anterioare pe fiecare celulă pentru cele două relații

Estimarea dimensiunii pentru alte operații

- ▶ Proiecția $\Pi_A(r) : V(A,r)$
- ▶ Agregarea: $_A g_F(r) : V(A,r)$
- ▶ Operații pe mulțimi
 - ▶ $r \cup s : n_r + n_s$
 - ▶ $r \cap s : \min(n_r, n_s)$
 - ▶ $r - s : n_r$
- ▶ Join extern
 - ▶ $r \bowtie s : \dim(r \bowtie s) + n_r$
 - ▶ $r \bowtie s = \dim(r \bowtie s) + n_r + n_s$
- ▶ $\sigma_{\theta_1}(r) \cap \sigma_{\theta_2}(r)$ echivalent cu $\sigma_{\theta_1 \wedge \theta_2}(r)$
- ▶ Estimatorii furnizează în general margini superioare

Optimizarea planului fizic

Estimarea costului la nivelul planului fizic

- ▶ Costul e în general măsurat ca durata de timp necesară pentru returnarea răspunsului
- ▶ Accesul la disc este costul predominant
 - ▶ Numărul de căutări * t_s (timpul pentru o localizare a unui bloc pe disc)
 - ▶ Numărul de blocuri citite/scrise * t_T (timpul de transfer)
 - ▶ costul CPU e ignorat pentru simplitate
- ▶ Costul pentru transferul a b blocuri plus S căutări:
$$b * t_T + S * t_s$$

Algoritmi pentru selectie

- ▶ **Căutare liniară (full scan)**
 - ▶ cost: $b_r * t_T + t_S$
 - ▶ dacă selecția e pe un atribut cheie, costul estimativ: $b_r/2 * t_T + t_S$
 - ▶ poate fi aplicată indiferent de condiția de selecție, ordonarea înregistrărilor în fișier, existența indecșilor
- ▶ **Căutarea binară**
 - ▶ aplicabilă pentru condiții de selecție de tip egalitate pe atributul după care e ordonat fișierul
 - ▶ costul găsirii primului tuplu ce satisface condiția: $\lceil \log_2(b_r) \rceil * (t_T + t_S)$; dacă există mai multe tuple se adaugă timpul de transfer al blocurilor
- ▶ **Scanarea indexului – condiția de selecție = cheia de căutare a indexului**
 - ▶ index primar pe cheie candidat, egalitate: $(h_i + 1) * (t_T + t_S)$
 - ▶ index primar pe non-cheie, egalitate: $h_i * (t_T + t_S) + t_S + t_T * b$
 - ▶ index secundar, egalitate, n tuple returnate: $(h_i + n) * (t_T + t_S)$
 - ▶ index primar, comparație: $h_i * (t_T + t_S) + t_S + t_T * b$

Algoritmi pentru selecții complexe

- ▶ **Conjunție:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
 - ▶ utilizarea unui index pentru θ_i și verificarea celorlalte condiții pe măsură ce tuplele sunt aduse în memorie
 - ▶ utilizarea unui index multi-cheie
 - ▶ intersecția identificatorilor (pointerilor la înregistrări) returnați de indecșii asociați condițiilor urmată de citirea înregistrărilor
- ▶ **Disjuncție:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$
 - ▶ reuniunea identificatorilor

Algoritmi pentru join

- ▶ Algoritmi:
 - ▶ join cu bucle imbricate (nested-loop join)
 - ▶ join indexat cu bucle imbricate
 - ▶ join cu fuziune (merge join)
 - ▶ join hash
- ▶ Alegerea se face pe baza estimării costului
- ▶ Sunt necesare estimări realizate la nivelul planului logic

Join cu bucle imbricate

- ▶ Pentru teta-join: $r \bowtie_{\theta} s$
for each tuplu t_r **in** r **do begin**
 for each tuplu t_s **in** s **do begin**
 if (t_r, t_s) satisface θ
 adaugă $t_r \cdot t_s$ la rezultat
 end
end
- ▶ relația interioară – s
- ▶ relația exterioară – r
- ▶ Costul estimat: $(n_r * b_s + b_r) * t_T + (n_r + b_r) * t_S$

Join indexat cu bucle imbricate

- ▶ Căutările în index pot înlocui scanarea fișierelor dacă:
 - ▶ e un echi-join sau join natural
 - ▶ există un index pe atributul de join al relației interioare
- ▶ pentru fiecare tuplu t_r în relația exterioară r se utilizează indexul pentru localizarea tuplurilor din s care satisfac condiția de join cu tuplul t_r
- ▶ costul: $b_r (t_r + t_s) + n_r * c$
 - ▶ c este costul parcurgerii indexului pentru a returna tuple din s care se potrivesc pentru un tuplu din r (echivalent cu selecția pe s cu condiția de join)
 - ▶ dacă există indecși pentru ambele relații, relația cu mai puține tuple va fi preferată drept relație exterioară în join
- ▶ Exemplu
 - ▶ $depositor \bowtie customer$, $depositor$ relație exterioară
 - ▶ $customer$ are asociat un index primar de tip B⁺-arbore pe atributul de join $customer-name$, cu 20 intrări pe nod
 - ▶ $customer$: 10,000 tuple ($f=25$), $depositor$: 5000 tuple ($f=50$)
 - ▶ costul: $100 + 5000 * 5 = 25,100$ blocuri transferate și căutări (corespondentul în joinul neindexat: 2,000,100 blocuri transferate și = 5100 căutări)

Join cu fuziune

- ▶ **Algoritm**

1. se sortează ambele relații în funcție de atributul de join
2. are loc fuziunea relațiilor

- ▶ **Poate fi utilizat doar pentru echi-joinuri**

- ▶ **Costul:**

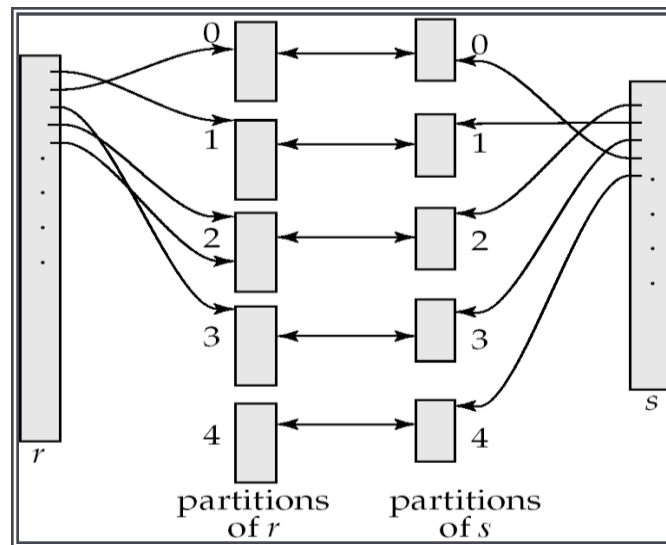
- ▶ $b_r + b_s$ blocuri transferate
- ▶ + costul sortării relațiilor

- ▶ **Join cu fuziune hibrid:** o relație este sortată iar a doua are un index secundar pe atributul de join de tip B⁺-arbore

- ▶ relația sortată fuzionează cu intrările de pe nivelul frunză al arborelui

Join hash

- ▶ aplicabil pentru echi-join
- ▶ o funcție hash h ce ia la intrare attributele de join partiționează tuplele ambelor relații în blocuri ce încap în memorie
 - ▶ $r_1, r_2, \dots, r_n$
 - ▶ $s_1, s_2, \dots, s_n$
- ▶ tuplele din r_i sunt comparate doar cu tuplele din s_i



Joinuri complexe

- ▶ **Condiție de tip conjuncție:** $r \bowtie_{\theta_1 \wedge \theta_2 \dots \wedge \theta_n} s$
 - ▶ bucle imbricate cu verificarea tuturor condițiilor sau
 - ▶ se calculează un join mai simplu $r \bowtie_{\theta_i} s$ și se realizează selecția pentru celelalte condiții
- ▶ **Condiție de tip disjuncție:** $r \bowtie_{\theta_1 \vee \theta_2 \dots \vee \theta_n} s$
 - ▶ bucle imbricate cu verificarea condițiilor sau
 - ▶ calculul reuniunii joinurilor individuale (aplicabil numai versiunii set a reuniunii)
 $(r \bowtie_{\theta_1} s) \cup (r \bowtie_{\theta_2} s) \cup \dots \cup (r \bowtie_{\theta_n} s)$

Eliminarea duplicatelor

- ▶ Sortarea tuplelor sau hashing
- ▶ Fiindca e costisitoare, SGBD-urile nu elimina duplicatele decat la cerere

Evaluare expresiilor

► Alternative:

- Materializarea: (sub)expresiile sunt materializate sub forma unor relații stocate pe disc pentru a fi date ca intrare operatorilor de pe nivele superioare
- Pipelining: tuple sunt date ca intrare operațiilor de pe nivele superioare imediat ce acestea sunt returnate în timpul procesării unui operator
 - nu e întotdeauna posibil (sortare, join hash)
 - varianta la cerere: nivelul superior solicită noi tuple
 - varianta la producător: operatorul scrie în buffer tuple iar părintele scoate din buffer (la umplerea bufferului există timpi de așteptare)

► Ex: Numele clientilor care au depozite >2000

Planuri de executie Oracle

- ▶ Inregistreaza planul:

EXPLAIN PLAN

[SET STATEMENT_ID = <id>]

[INTO <table_name>]

FOR <sql_statement>;

- ▶ Pentru orice comanda DML

- ▶ Vizualizeaza planul:

SELECT * FROM table(dbms_xplan.display);

sau

select * from plan_table [where statement_id = <id>];

<http://www.oracle.com/technetwork/database/bi-datawarehousing/twp-explain-the-explain-plan-052011-393674.pdf>

Planuri de executie

Oracle-statistici

- ▶ **Table statistics**
 - ▶ Number of rows
 - ▶ Number of blocks
 - ▶ Average row length
- ▶ **Column statistics**
 - ▶ Number of distinct values (NDV) in column
 - ▶ Number of nulls in column
 - ▶ Data distribution (histogram)
- ▶ **Index statistics**
 - ▶ Number of leaf blocks
 - ▶ Levels
 - ▶ Clustering factor
- ▶ **System statistics**
 - ▶ I/O performance and utilization
 - ▶ CPU performance and utilization

Planuri de executie

Colectarea statisticilor

- ▶ Proceduri Oracle din pachetul DBMS_STATS:
- ▶ GATHER_INDEX_STATS
 - ▶ Index statistics
- ▶ GATHER_TABLE_STATS
 - ▶ Table, column, and index statistics
- ▶ GATHER_SCHEMA_STATS
 - ▶ Statistics for all objects in a schema
- ▶ GATHER_DATABASE_STATS
 - ▶ Statistics for all objects in a database
- ▶ GATHER_SYSTEM_STATS
 - ▶ CPU and I/O statistics for the system
- ▶ http://docs.oracle.com/cd/B10500_01/server.920/a96533/stats.htm

Planuri de executie

Hints

- ▶ In cadrul unei comenzi DML este posibil a instrui optimizatorul Oracle asupra planului de executie:

```
SELECT /*+ USE_MERGE(employees departments) */ * FROM employees,  
departments WHERE employees.department_id =  
departments.department_id;
```

http://docs.oracle.com/cd/B19306_01/server.102/b14200/sql_elements006.htm

Bibliografie

- ▶ Capitolele 13 și 14 în *Avi Silberschatz Henry F. Korth S. Sudarshan. “Database System Concepts”. McGraw-Hill Science/Engineering/Math; 4th edition*